Учебник по DX Framework и космическому симулятору

Архитектура, математика, DirectX 11, Bullet Physics, NPC и разработка новых систем

Подробное руководство по текущему коду и практический план развития на 14 дней

DX_FrameWork

Издание 4.0 · обновлено по исходному коду 28 октября 2026 года

Содержание

Часть I. Паспорт движка
- 1. Что это за проект
- 2. Как пользоваться книгой после долгого перерыва
- 3. Как движок вырос: история по локальному git
- 4. Карта репозитория
- 5. Владение и жизненный цикл приложения
- 6. Сборка и запуск
- 7. Один кадр: фактический timeline
- 8. Время и численная устойчивость

Часть II. C++ и ECS именно в этом проекте
- 9. C++: значения, ссылки, указатели и RAII
- 10. Как устроена ECS
- 11. Как добавить компонент и систему

Часть III. Математика движка с проверяемыми примерами
- 12. Координаты, величины и проверка размерности
- 13. Векторы для движения, зрения и навигации
- 14. Матрицы и Transform
- 15. Углы, кватернионы и интерполяция
- 16. Камера, проекция и луч
- 17. Геометрия, границы и упражнения

Часть IV. DirectX 11 и рендер
- 18. Ресурсы и состояния DirectX 11
- 19. Фактический render pipeline
- 20. PBR: математика Cook–Torrance
- 21. Цвет, HDR и IBL
- 22. Тени
- 23. Прозрачность, reflection и refraction
- 24. Частицы и взрывы
- 25. Скелетная анимация

Часть V. Bullet и симуляция мира
- 26. Bullet Physics и синхронизация
- 27. Character controller
- 28. Давление, затопление и плавучесть
- 29. Комнаты и атмосфера
- 30. Энергетика
- 31. Damage, cutting и repair

Часть VI. NPC, данные и инструменты
- 32. NPC AI pipeline
- 33. StateSystem и исполнение

34. Recast/Detour NavMesh
35. JobSystem и асинхронные конвейеры
36. Scene save, runtime save и Entity remap
37. Editor, picking и диагностика

Часть VII. Срез качества и диагностика

38. Сильные стороны движка
39. Реестр слабых мест
40. Производительность без угадывания
41. Как диагностировать симптом
42. Практикум: безопасное добавление engine-функции
43. Карта чтения кода

Часть VIII. Работа с кодом и практикум по NPC

44. Как изучать собственный движок, а не просто читать файлы
45. Анатомия персонажа в DX Framework
46. Дружественный NPC в текущей архитектуре
47. Проектирование врага на существующем NPC pipeline
48. Практическое задание Первый полноценный дружественный NPC
49. Практическое задание Первый преследующий и атакующий враг
50. План одного рабочего дня и порядок сдачи

Часть IX. Боевой NPC как учебный проект

51. Текущее состояние врага и границы следующей версии
52. Практикум Удар направлен вбок
53. Восприятие и память цели
54. Преследование последней известной позиции
55. Укрытия как данные и запрос
56. Атака из укрытия и смена позиции
57. Обход и фланг
58. Несколько врагов и распределение ролей

Часть X. Атлас классов и маршруты данных

60. Как читать атлас и где проходят границы
61. Ядро, цикл приложения и инфраструктура
62. ECS, сцена, редактор и сериализация
63. Ресурсы и рендеринг
64. Игрок, ввод и перемещение персонажа
65. NPC: решение, команда и исполнение
66. Бой, анимация, инвентарь и взаимодействие
67. Физика подлодки, помещения, энергия и ремонт
68. Сквозные маршруты вызовов
69. Правила зависимостей и кандидаты на разделение

Часть XI. Практика разработки систем и NPC

70. Как читать одну систему от входа до результата
71. Дружественный NPC ремонтирует пробоину

72. Вражеский NPC обнаруживает и атакует
73. Добавление новой работы дружественного NPC
74. Добавление нового боевого поведения
75. Bullet Physics как часть симуляции
76. Вершинный и пиксельный шейдеры
77. Рецепт добавления системы в DX Framework
Часть XII. План развития космического симулятора на 14 дней
78. День 1. Базовая сцена и измеряемое поведение
79. День 2. Надёжность ремонтных задач
80. День 3. Доступность маршрута до назначения
81. День 4. Роли и ресурсы экипажа
82. День 5. Пожар как системная авария
83. День 6. NPC тушит пожар
84. День 7. Восприятие врага без всеведения
85. День 8. Надёжный огонь и перезарядка
86. День 9. Укрытия и выход из тупиков
87. День 10. Простой командный бой
88. День 11. Повреждения корабля как причина решений
89. День 12. Звук и сигналы тревоги
90. День 13. Сохранение и производительность
91. День 14. Вертикальный срез и выпуск
Приложение A. Горячие клавиши
Приложение B. Формулы на одной странице
Приложение C. Словарь
Приложение D. Что открыть через год

Как читать это издание

Учебник соединяет описание уже работающего кода с практическими улучшениями. Путь к файлу и имя метода обозначают текущую реализацию; главы с проектными задачами описывают то, что ещё предстоит добавить.

Математическая часть переписана с нуля: каждый приём привязан к коду и численному примеру. Разделы 70–77 подробно проходят ремонт, атаку, Bullet Physics, шейдеры и способ добавления систем. Разделы 78–91 содержат единственный план на две недели.

Важная поправка к прежним главам о боевом отряде: вызов NpcSquadCombatSystem в текущем ApplicationClass.cpp закомментирован. Индивидуальные укрытия и стрельба работают; командная тактика рассматривается как дальнейшая работа на день 10. При расхождении текста и исходников проверяй порядок вызовов в ApplicationClass.cpp и конкретный класс системы.

Часть I. Паспорт движка

1. Что это за проект

DX Framework — учебный игровой движок и одновременно основа космического симулятора. Он написан на C++20 для Windows и использует DirectX 11, DirectXMath, Bullet Physics, Assimp, TinyGLTF, Recast/Detour, ImGui, DirectWrite и JSON. Это уже не набор отдельных уроков: рендер, ECS, физика, редактор, импорт корабля, персонажи, NPC, атмосфера, энергетика, повреждения и сохранение сцены соединены одним жизненным циклом.

Главная ценность проекта — вертикальная интеграция. Например, пробоина — это не только картинка:

1. попадание создаёт событие повреждения;
2. повреждение меняет состояние панели или runtime-меша;
3. пробоина создаёт приток воды или утечку атмосферы;
4. помещение меняет параметры среды;
5. HUD и отладчик показывают последствия;
6. Utility AI может выбрать ремонт;
7. StateSystem строит исполнение задачи;
8. персонаж идёт по NavMesh и завершает ремонт;
9. состояние попадает в scene save.

Именно такие цепочки превращают отдельные технологии в движок.

1.1. Что реализовано сейчас

Область	Реальное состояние на срезе
Платформа	Windows x64, Visual Studio 2022, toolset v143
Язык	C++20 в Debug x64; конфигурация Release x64 настроена неполно
Графика	DirectX 11 forward renderer, PBR, тени, HDR sky, reflection/refraction, частицы, debug draw
Мир	ECS на `Entity = uint32_t` и типизированных component pools
Физика	Bullet rigid bodies, raycast, fixed step 60 Гц, capsule character controller
Навигация	Recast build и Detour pathfinding, локальное избегание динамических препятствий
Редактор	ImGui, picking, gizmo, инспектор компонентов, импорт и save/load
Контент	Assimp/TinyGLTF, GLB-корабль, материалы, skinned animation
Симуляция	комнаты, двери, кислород, давление, затопление, плавучесть, power grid, damage/repair
Параллельность	JobSystem, CPU-фаза фоновой загрузки, main-thread GPU upload
Тесты	есть `DXFrameworkCoreTests` и локальный build/shader gate; внешний CI пока не настроен

1.2. Уровень зрелости

Точный диагноз: **функционально богатый alpha-прототип собственного движка**.

Это выше «учебной демки», потому что подсистемы реально связаны. Это ещё не production engine, потому что:

- Debug x64 собирается, а Release x64 не воспроизводится из текущего project file;
- есть начальные core-contract tests, но coverage ещё узкое и внешний CI не настроен;
- есть несколько разных соглашений об углах и времени;
- крупные координаторы выросли до нескольких тысяч строк;
- часть физической и графической математики содержит подтверждённые расхождения;
- save/load не является транзакционным runtime-save;
- жизненный цикл ряда raw-ресурсов зависит от ручного порядка очистки.

Это нормальное состояние для быстро выросшего авторского движка. Следующий этап — не добавлять ещё десять подсистем, а сделать существующие правила явными, проверяемыми и повторяемыми.

2. Как пользоваться книгой после долгого перерыва

Не нужно читать её подряд. Сначала установи тип задачи.

Если нужно…	Открыть
вспомнить общий поток кадра	главы 6–7
добавить клавишу или кнопку мыши для игры	раздел 7.4
найти владельца объекта или GPU/Bullet-ресурса	главы 5 и 9
понять Transform, камеру или picking	главы 10–13
разобраться с PBR, HDR или тенями	главы 14–17
исправить дрожание и зависимость от FPS	главы 7, 18–20
понять затопление, давление и плавучесть	главы 21–23
добавить компонент или систему	главы 8 и 34
добавить задачу NPC	главы 27–29
понять scene save и Entity remap	глава 31
найти известный риск	глава 36

После перерыва ответь на семь вопросов:

1. Какие данные являются источником истины?
2. Какая система имеет право менять эти данные?
3. На каком потоке она работает?
4. В какой фазе кадра она вызывается?
5. В каких единицах хранятся значения?
6. Кто владеет внешним ресурсом и кто его освобождает?
7. Какая проверка доказывает корректность?

Если хотя бы один ответ неизвестен, изменение пока опасно.

3. Как движок вырос: история по локальному git

История коммитов показывает естественную эволюцию проекта.

Период	Этап
4–7 марта 2026	DirectX 11 каркас, multitexture, normal map, render-to-texture, ShaderManager, fog
20 марта	добавлены ECS и набор шейдеров
1 апреля	появились collision components
19–25 апреля	первый цикл скелетной анимации и animated mesh

Период	Этап
12 мая	отдельный проход по памяти и владению ресурсами
13–17 мая	skybox cache, камера, Editor/Play, player controller, PBR GLB, reflection, animation state, debug UI
26 мая – 14 июня	интеграционные исправления; последний коммит основной ветки `f31cfbb`
после 14 июня	большой незакоммиченный этап: перенос кода в `src`, NPC/NavMesh, корабль, power, atmosphere, particles, repair, асинхронные системы и новые документы

История объясняет нынешнюю форму архитектуры. `ApplicationClass` и `SceneManager` стали большими не из-за одной ошибки, а потому что каждый новый вертикальный срез естественно подключался к уже существующим координаторам. Теперь граница роста достигнута: дальнейшее качество зависит от выделения стабильных интерфейсов и тестов.

4. Карта репозитория

Рабочий проект находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork
```

Решение Visual Studio находится уровнем выше:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln
```

Основная документация находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork\docs
```

4.1. Ответственность каталогов

Каталог	Что является его задачей
`src/Core`	WinMain, окно, приложение, время, DirectX/Bullet bootstrap, JobSystem, файлы
`src/ECS`	registry, pools, базовые компоненты и физико-сценовые системы
`src/Scene`	создание сущностей, импорт, model cache, render traversal, JSON save/load
`src/Render`	D3D11, camera, renderer, models, materials, shaders, textures, culling, text
`src/Gameplay/Player`	input intent, movement, facing, capsule controller, vitals, first person
`src/Gameplay/AI`	Recast/Detour NavMesh и path queries
`src/Gameplay/Ship`	rooms, atmosphere, alerts, room culling, decompression, submarine data
`src/Gameplay/Power`	generators, consumers, battery и распределение мощности
`src/Gameplay/Combat`	damage events, weapons, explosions
`src/Gameplay/Repair`	repair state и runtime mesh restoration
`src/Gameplay/Effects`	CPU particles и particle renderer

Каталог	Что является его задачей
src/Editor	ImGui panels, gizmo, drag-and-drop, object debug
assets	модели, текстуры, HLSL и данные мира
docs	руководства, теория и планы

Некоторые новые файлы AI и world time пока находятся в корне проекта: NPCComponents.h, NpcCommandSystem.h, NpcUtilitySystem.h, StateSystem.h, GameTimer.h, WorldEvent.h, WorldEventLog.h. Это факт текущего среза, а не рекомендуемое окончательное размещение.

4.2. Горячие точки

Самые крупные авторские файлы на момент аудита:

Файл	Примерный размер	Почему важен
src/Editor/EditorUI.cpp	4,8 тыс. строк	почти весь редактор и множество component panels
src/Scene/SceneManager.cpp	4,4–4,9 тыс. строк	импорт, сцена, save/load и часть render traversal
src/Core/ApplicationClass.cpp	2,3 тыс. строк	bootstrap и точный порядок кадра
src/ECS/System/MeshCuttingTest.cpp	2,2 тыс. строк	cutting, damage history и асинхронные операции
StateSystem.h	1,8 тыс. строк	исполнение NPC-состояний и маршрутов
src/Render/Model/assimpmodelclass.cpp	1,8 тыс. строк	импорт, GPU buffers, animation, bone matrices
src/Gameplay/AI/NpcNavMeshSystem.cpp	1,4 тыс. строк	build/query/debug Recast/Detour
src/ECS/System/PhysicsSystem.cpp	1,2 тыс. строк	shapes, Bullet sync, forces, buoyancy

Большой файл не обязательно плох. Риск появляется, когда он одновременно владеет ресурсами, принимает архитектурные решения и выполняет детали нескольких подсистем.

5. Владение и жизненный цикл приложения

5.1. Верхний уровень

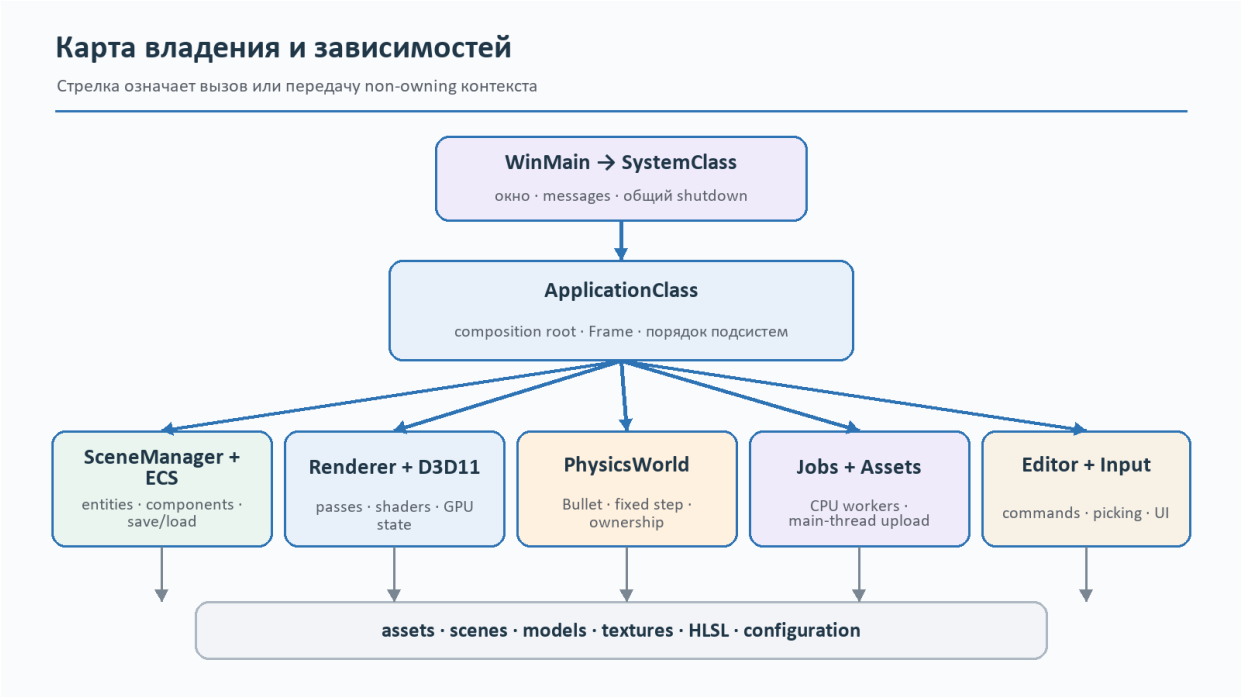


Рисунок 1. Владение и главные зависимости движка.

```
WinMain
  └─ SystemClass (stack)
       ├─ InputClass (unique_ptr)
       └─ ApplicationClass (raw owning pointer)
            ├─ D3DClass (unique_ptr)
            ├─ CameraClass (unique_ptr)
            ├─ SceneManager (value member)
            │    └─ ECSRegistry (value member)
            ├─ Renderer (raw owning pointer)
            ├─ JobSystem (value member)
            ├─ AssetLoadSystem (value member, depends on JobSystem)
            ├─ EditorUI (value member)
            └─ gameplay/render helper systems
```

`WinMain` создаёт `SystemClass` на стеке, вызывает `Initialize`, затем `Run`, затем `Shutdown`. Это хорошая корневая форма: даже при неуспешной инициализации вызывается общий shutdown.

5.2. Почему порядок shutdown важен

Внешние API хранят невладеющие ссылки:

- Bullet world помнит rigid bodies, ghost objects, actions и constraints;
- D3D context помнит связанные SRV, buffers, shaders и render targets;
- ECS-компоненты могут содержать указатели на Bullet и render resources;
- фоновые jobs могут держать `this`, Entity или CPU-результат для main-thread upload;
- материалы ссылаются на шейдеры ShaderManager.

Поэтому безопасный порядок выглядит так:

1. запретить новые фоновые задачи;
2. дождаться workers и выполнить разрешённые main-thread completions;
3. отвязать D3D state;
4. удалить Bullet actions/constraints/bodies;
5. очистить ECS и scene-owned materials/models;
6. удалить renderer helpers и render textures;
7. закрыть Editor/DirectWrite;
8. удалить ShaderManager;
9. освободить D3D device последним.

> **ОГРАНИЧЕНИЕ.** Текущий `SceneManager::LoadScene` напрямую заменяет `ECSRegistry`, тогда как character controllers и constraints требуют явного удаления из Bullet world. Полный application shutdown учитывает world лучше, чем горячая загрузка новой сцены.

5.3. Правила raw pointer

В проекте raw pointer имеет три разных смысла:

Вид	Пример	Правило
owning	`Renderer* m_renderer`	ровно один `delete`, поле сразу `nullptr`
borrowed	`ShaderBaseClass*` из ShaderManager	никогда не удалять в потребителе
API handle	`ID3D11Buffer*`, `btRigidBody*`	освобождать по контракту API и сначала отвязать от владельца

Если из объявления нельзя понять вид указателя, это будущая ошибка. Целевая модель: owning C++-объекты — `unique_ptr`; COM — `Microsoft::WRL::ComPtr`; невладеющие ссылки — pointer/reference с комментарием и ограниченным временем жизни.

6. Сборка и запуск

6.1. Подтверждённый baseline

`Debug|x64` собирается локальным MSBuild из Visual Studio 2022. Результат находится в:

```
C:\Users\Lakis\Desktop\DX_FrameWork\x64\Debug\DX_FrameWork.exe
```

Использованная команда:

```
& "C:\Program Files\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin\MSBuild.exe" `
  "C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln" `
  /m /p:Configuration=Debug /p:Platform=x64
```

6.2. Почему Release x64 сейчас падает

В `DX_FrameWork.vcxproj` Debug x64 получает:

- `EngineIncludeDirectories`;
- Bullet include paths;
- C++20;
- Assimp, Bullet Debug, Recast/Detour libraries;
- Windows subsystem.

Release x64 содержит Recast/Detour, но не повторяет include directories, C++20 и полный набор Assimp/Bullet libraries. Поэтому это конфигурационный дефект, а не доказательство ошибок каждой исходной подсистемы.

6.3. Runtime-зависимость от working directory

Многие пути записаны как `assets/...`. `ApplicationClass` создаёт файловую систему от `current_path()/assets`. Следовательно, запуск из Visual Studio работает, а запуск EXE из произвольной папки может не найти shaders/models/textures. Готовый движок должен вычислять runtime root от пути EXE или получать его явно.

6.4. Минимальный smoke-test

После любой архитектурной правки проверить:

1. запуск без shader compiler error;
2. загрузку сцены и HDR sky;
3. `F5` Editor/Play;
4. ходьбу, бег, прыжок и спокойный idle на полу;
5. NPC path через дверь;
6. пробоину и изменение среды;
7. ремонт;
8. save → очистка → load;
9. возврат в Editor;
10. чистый shutdown без live D3D objects и dangling Bullet objects.

7. Один кадр: фактический timeline

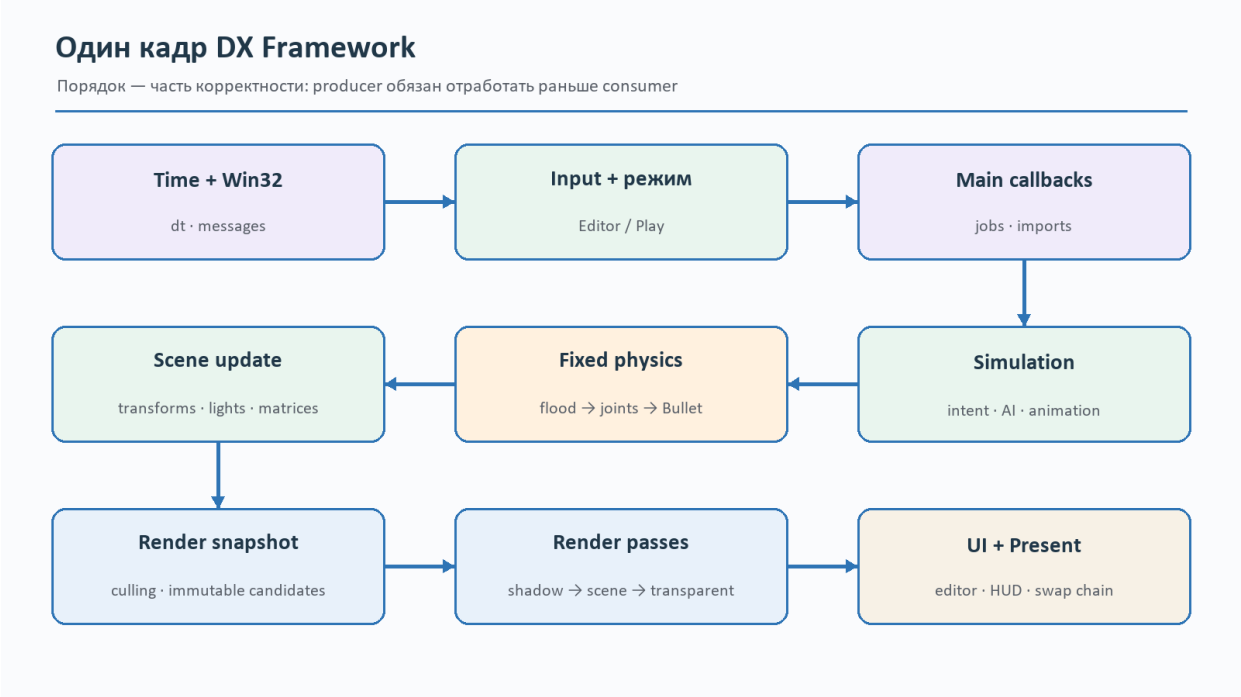


Рисунок 2. Фактическая последовательность фаз кадра.

Главная точка входа — `ApplicationClass::Frame(float dt)`. Порядок систем является частью поведения, а не косметикой.

```
Windows messages / TimeManager
        ↓
InputManager.Update + EventQueue.Clear
        ↓
Engine hotkeys (F1/F2/F5/F6)
        ↓
GameplayInputSystem.Sample (только действия Play)
        ↓
Main-thread jobs + asset upload + async ship import budget
        ↓
Editor camera  [Editor]
        ↓
Interaction → doors → weapon/repair  [Play]
        ↓
Player intent
        ↓
NPC Utility → StateSystem → Facing → CharacterController → Animation
        ↓
Damage → particles
        ↓
Sync compartments
        ↓
Flooding → joints → submarine → PhysicsSystem → pressure
        ↓
Post-physics camera
        ↓
SceneManager.UpdateAll
  animation / attachments / rooms / generator / grid / lights /
  oxygen / room environment / player vitals / ship alerts
        ↓
Camera matrices + debug/navmesh
        ↓
Renderer.RenderFrame
        ↓
```

```
Editor UI or Play UI / HUD
        ↓
Present
```

7.1. Зависимости порядка

- Utility должна принять решение до `StateSystem`, иначе команда исполнится на кадр позже.
- Facing читает `walkDirection`, поэтому идёт после AI, но до controller/animation.
- Камера Play следует после физики, иначе будет показывать предыдущую позицию.
- Power grid должен обновиться до потребителей и oxygen generator.
- Render видит только уже завершённое состояние кадра.

7.2. Editor и Play

`F5` переключает режим. При выходе из Play код явно сбрасывает movement intent и Bullet controller velocity, иначе кэшированное направление продолжило бы двигать персонажа в Editor. Это хороший пример архитектурного инварианта: **отключить producer недостаточно; нужно сбросить состояние consumer**.

7.3. Очередь Windows

`SystemClass::Run` использует `PeekMessage`, но на срезе обрабатывает одно сообщение за итерацию. При потоке mouse/window messages очередь может отставать. Целевая форма — внутренний `while (PeekMessage(...))`, после которого выполняется ровно один simulation/render frame.

7.4. Игровой ввод: от кнопки к действию

ФАКТ на 5 октября 2026 года. `SystemClass` получает `WM_KEYDOWN/UP` и сообщения левой, правой и средней кнопок мыши. `InputClass` хранит их физическое состояние. `InputManager` обслуживает редакторский ввод и события интерфейса. В `ApplicationClass::Frame` после решения о режимах F5 и F2 вызывается `GameplayInputSystem::Sample` ровно один раз. Он преобразует физические кнопки в `GameInputFrame`. Игровые системы читают этот снимок, а не коды клавиш.

```
WM_LBUTTONDOWN → InputClass::MouseLeftDown
               → GameplayInputSystem::Sample
               → GameInputFrame::Down(GameAction::Fire)
               → WeaponSystem::Update
```

`GameInputFrame` хранит три состояния для каждого `GameAction`:

Запрос	Смысл	Пример
`Down(action)`	кнопка удерживается в данном кадре	движение, ремонт, автоматический огонь
`Pressed(action)`	переход из отпущенного состояния в нажатое	взаимодействие, смена слота, пауза
`Released(action)`	переход из нажатого состояния в отпущенное	отпускание прицела или зарядки

Одна игровая команда может иметь несколько физических привязок: `Fire` сейчас принимает ЛКМ и `H`. Действия `Aim` (ПКМ) и `PrecisionAim` (средняя кнопка) уже описаны в слое ввода, но их игровую механику ещё нужно подключить; само наличие действия не переключает прицел. Дельта движения мыши для поворота камеры пока обрабатывается `CameraController::UpdateFollowLook` отдельно от дискретных действий. Клавиши редактора F1, F2, F5, F6 и свободная камера тоже остаются в контуре редактора.

Как добавить игровое действие, например перезарядку по `R`:

1. Добавь `Reload` в `GameAction` в `src/Gameplay/Player/GameplayInputSystem.h`, перед `Count`.
2. В конструкторе `GameplayInputSystem` добавь `SetBindings(GameAction::Reload, { GameInputBinding::Key('R') });`. Для мыши используй `GameInputBinding::Mouse(GameMouseButton::Right)` или другую кнопку. Чтобы сохранить несколько способов ввода, передай их в одном списке или вызови `AddBinding`.
3. В нужной системе прими `const GameInputFrame& input` и читай `input.Pressed(GameAction::Reload)`. Для действия, которое длится всё время удержания, используй `Down`. Если показываешь игроку подсказку, возьми подпись первой привязки через `GetBindingLabel`; так уже устроена подсказка `Interact` на HUD.
4. Передай снимок из `ApplicationClass::Frame` в эту систему. Не опрашивай там `InputClass::IsKeyDown('R')`: иначе привязка снова разойдётся по коду.
5. Проверь один короткий клик, удержание, отпускание, паузу, переключение Editor/Play и F2 с открытым UI. Проверка подтверждает, что событие приходит один раз и не попадает в игру, пока интерфейс владеет вводом.

`Sample` обнуляет игровые действия в Editor и при включённом F2 курсоре. Переключение вида, пауза, масштаб времени и HUD продолжают получать ввод в Play при F2. При возвращении курсора удерживаемая кнопка считается новым нажатием; это следствие вычисления `Pressed` по состоянию действий, поэтому для опасных действий после UI можно отдельно потребовать отпускания. Когда симуляция стоит на паузе, `Sample` всё равно обновляет границы нажатий, а игровой код, зависящий от `simulationDeltaTime`, не выполняется. Старые горячие клавиши `Space` и `K` из `SystemClass` управляют отладочным `TimeManager` только в Editor; в Play `Space` прыгает, а `Esc` закрывает открытую панель.

Где искать код: `src/Core/SystemClass.cpp` → `src/Input/InputClass.h` → `src/Gameplay/Player/GameplayInputSystem.h/.cpp` → `src/Core/ApplicationClass.cpp` → `CharacterMovementSystem`, `WeaponSystem`, `InteractionSystem`, `RepairSystem` или `GameHud`. Начинай чтение с конструктора `GameplayInputSystem`: там собраны текущие привязки.

8. Время и численная устойчивость

В проекте существуют два понятия времени:

- `TimeManager` — QPC, pause/step и delta игрового цикла;
- `GameTimer` — новый world/game total time для событий.

Большинство систем получает `dt` напрямую. Такое дублирование допустимо временно, но без контракта непонятно, что останавливается при pause и что масштабируется.

8.1. Variable step

Обычное интегрирование:

```
x_next = x + v · dt
v_next = v + a · dt
```

Оно подходит для камеры, UI и эффектов, если `dt` ограничен. После breakpoint или подвисания `dt` может стать огромным, поэтому безопасно:

```
dt_sim = clamp(dt_real, 0, dt_max)
```

8.2. Fixed step Bullet

`PhysicsWorld` использует:

```
h = 1 / 60 s
accumulator = min(accumulator + dt, 0.25 s)
while accumulator >= h:
    stepSimulation(h)
    accumulator -= h
alpha = accumulator / h
```

Ограничение 0,25 секунды предотвращает «спираль смерти»: движок не пытается бесконечно догнать потерянное время.

8.3. Интерполяция

После fixed tick визуальная позиция может быть:

```
x_render = lerp(x_previous, x_current, alpha)
```

В проекте character controller имеет отдельную стабилизацию/интерполяцию, а обычные rigid bodies в основном копируют текущий Bullet transform. Поэтому на высоком FPS часть тел может визуально обновляться ступенями 60 Гц.

8.4. FPS-независимое сглаживание

Формула `lerp(old, target, k · dt)` лишь приближённо независима от FPS и должна ограничивать `t` в `[0,1]`. Более устойчивый коэффициент экспоненциального приближения:

```
t = 1 - exp(-lambda · dt)
value = lerp(value, target, t)
```

Здесь `lambda` — скорость сходимости в 1/с. Период полураспада связан с ней:

```
lambda = ln(2) / halfLife
```

> **ОГРАНИЧЕНИЕ.** Сглаживание submersion в `PhysicsSystem` использует постоянные `0.3` на render frame. Это означает разную плавучесть при 30, 60 и 144 FPS.

8.5. Силы должны жить внутри fixed tick

На срезе buoyancy/drag/torque применяются один раз на render frame, после чего Bullet может выполнить ноль, один или несколько fixed steps. При низком FPS Bullet очистит силы после первого шага, и следующие шаги останутся без них; при высоком FPS силы могут накапливаться до следующего шага. Целевая модель: вычислять continuous forces для каждого fixed tick.

Часть II. C++ и ECS именно в этом проекте

9. C++: значения, ссылки, указатели и RAII

9.1. Значение

```
TransformComponent transform;
```

Объект живёт в своей области видимости. Для ECS-компонента это обычно означает хранение внутри `unordered_map<Entity, T>`.

9.2. Ссылка

```
void Update(ECSRegistry& registry, float dt);
```

Ссылка обязательна и не бывает `nullptr`. Она не передаёт владение.

9.3. Указатель

```
TransformComponent* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;
```

Указатель выбран, потому что компонент может отсутствовать. Проверка `nullptr` — часть контракта.

9.4. RAII

RAII связывает ресурс со временем жизни объекта. В движке это особенно важно для:

- COM resources (`ID3D11Buffer`, `ID3D11ShaderResourceView`);
- Bullet body/shape/motion state;
- worker threads;
- файлов;
- временных GPU states.

`RigidBodyComponent` пытается реализовать RAII и запрещает копирование. Это правильное направление. Но move-конструктор обязан переносить **каждое** поле владения; пропущенный raw pointer создаёт leak или double-delete.

9.5. Rule of zero / five

Если класс владеет ресурсом, есть два безопасных пути:

1. хранить ресурс в готовом RAII-типе и использовать rule of zero;
2. явно реализовать destructor, move constructor и move assignment, а copy удалить.

Пустой copy constructor хуже удалённого: он компилируется и создаёт объект с неинициализированным состоянием. На срезе такие конструкторы есть у некоторых legacy-классов, включая камеру/D3D/SystemClass.

10. Как устроена ECS

10.1. Entity

```
using Entity = std::uint32_t;
```

Registry выдаёт монотонно растущий ID. Entity не содержит данных; это ключ для component pools.

10.2. Component pool

Для обычного `T`:

```
ComponentManager<T>
  └ unordered_map<Entity, T>
```

Для `ActiveComponent` используется `unordered_set<Entity>`, потому что tag не хранит полезных полей.

10.3. Type erasure

Registry хранит разные pools в одной таблице:

```
unordered_map<type_index, unique_ptr<IComponentPool>>
```

`IComponentPool::Remove(Entity)` позволяет `DestroyEntity` удалить ID из каждого пула без знания конкретного `T`.

10.4. Добавление компонента

Есть copy-версия для копируемых типов и move-версия для owning-компонентов:

```
registry.AddComponent<TransformComponent>(entity, transform);
registry.AddComponent<RigidBodyComponent>(entity, std::move(body));
```

Для raw-resource компонента второй вариант обязателен.

10.5. Сложность

Операция	Ожидаемая сложность
`Get/Has/Add/Remove` одного компонента	O(1) average
`DestroyEntity`	O(number of component types)
`ForEach<T>`	O(count(T))
`ForEach<T1,T2>`	O(count(T1)) с lookup во втором pool

10.6. Подтверждённые ограничения

1. Нет generation counter. После будущего reuse старый Entity сможет указывать на новый объект.
2. Нет явного alive set; `DestroyEntity` только удаляет компоненты.
3. `GetComponent` и `HasComponent` создают пустой pool при первом чтении отсутствующего типа.
4. `ForEach` идёт напрямую по `unordered_map`; добавление/удаление того же типа из callback может инвалидировать iterator.
5. Нет deferred command buffer для структурных изменений.

Целевая Entity:

```
64 bits = generation | index
```

А структурные изменения должны накапливаться в command buffer и применяться после system phase.

11. Как добавить компонент и систему

11.1. Компонент — только данные

```
struct HeatComponent
{
    float kelvin = 293.15f;
    float heatCapacityJPerK = 1000.0f;
};
```

У полей должны быть единицы, диапазон и значение по умолчанию.

11.2. Система — одно правило изменения

```
class HeatSystem
{
public:
    static void Update(ECSRegistry& registry, float dt)
    {
        registry.ForEach<HeatComponent>(
            [dt](Entity, HeatComponent& heat)
            {
                heat.kelvin = std::max(0.0f, heat.kelvin - 0.2f * dt);
            });
    }
};
```

11.3. До подключения определить

- phase кадра;
- scaled или unscaled time;
- main thread или worker;
- источник истины;
- сериализацию;
- debug representation;
- минимум один детерминированный тест.

Часть III. Математика движка с проверяемыми примерами

Математика здесь нужна для конкретных задач: поставить объект в мире, повернуть персонажа к цели, проверить обзор NPC, провести луч из оружия, построить камеру и передать вершину в шейдер. Все примеры ниже используют метры, секунды и радианы там, где явно не сказано иное. В шейдере `src/Render/Shader/Vs/pbr.vs` позиция умножается как вектор-строка: `mul(position, worldMatrix)`, затем на `viewMatrix` и `projectionMatrix`. Это важнее любой заученной формулы из книги: порядок умножения обязан совпадать с контрактом конкретного кода.

12. Координаты, величины и проверка размерности

12.1. Точка, направление и смещение

Точка отвечает на вопрос «где?», направление — «куда?», смещение — «на сколько?». Если NPC стоит в A = (2, 0, 3), а пробоина в B = (8, 0, 11), то вектор до цели d = B − A = (6, 0, 8) м. Его длина |d| = √(6² + 8²) = 10 м. Направление без единиц: n = d / |d| = (0,6, 0, 0,8). При скорости 2 м/с и шаге 0,25 с желаемое смещение равно n · 2 · 0,25 = (0,3, 0, 0,4) м. Новая желаемая точка — (2,3, 0, 3,4). В реальном движении окончательное положение определяет Bullet после столкновений; простой расчёт не имеет права «телепортировать» капсулу сквозь стену.

В однородных координатах точке соответствует w = 1, а направлению w = 0. Поэтому перенос действует на точку и не действует на направление. Ошибка `w = 1` для нормали или направления заставит переносить их вместе с объектом.

12.2. Локальное и мировое пространство

Локальная координата задана относительно модели или кости, мировая — относительно сцены, view — относительно камеры, clip — результат проекции. Нельзя сравнивать muzzle socket в пространстве кости с мировой позицией цели. Сначала переведи оба значения в одно пространство. Для точки модели p путь в текущем PBR vertex shader такой: `p_world = p_model · W`, `p_view = p_world · V`, `p_clip = p_view · P`. После перспективного деления x/w и y/w становятся нормализованными экранными координатами. Не дели на w раньше проекции.

Пример: локальная точка (1, 0, 0) у объекта с одним только переносом на (5, 0, 2) становится (6, 0, 2). Если объект повернуть или масштабировать, сначала уточни, вокруг какого pivot это происходит. Один и тот же набор чисел Transform с другим pivot даст другое мировое положение вершины.

12.3. Почему проверять единицы

`скорость · deltaTime` имеет размерность `(м/с) · с = м` и годится для смещения. `скорость + deltaTime` складывает м/с и с; физического смысла у выражения нет. Ускорение интегрируется как `v_new = v_old + a · dt`; позиция затем меняется через скорость. Параметр `repairRate` в `RepairableComponent` имеет смысл единиц прогресса в секунду, поэтому код игрока добавляет

repairRate · dt. Для NPC путь другой: его прирост определяется requiredProgress · repairSkill · dt / repairDuration. Смешивать эти две скорости без оговорки нельзя.

13. Векторы для движения, зрения и навигации

13.1. Длина, нормализация и нулевой вектор

Длина трёхмерного вектора v = (x, y, z) равна √(x² + y² + z²). Нормализация делит компоненты на длину и делает длину равной единице. Перед делением проверяй малую длину: при A = B направление не определено. Условие lengthSquared > epsilon² дешевле вычисления корня и надёжнее точного сравнения с нулём. Для дистанций до цели сравнивай квадраты: distanceSquared <= range * range. Тогда корень не нужен.

На палубе часто нужен горизонтальный вектор (dx, 0, dz). В StateSystem::UpdateRepairTask рабочая дистанция до пробоины считается горизонтально, потому что персонаж ходит по полу. Для выстрела или пространственного радиуса нельзя автоматически выбрасывать Y: цель на лестнице или другом уровне может оказаться значительно дальше, чем показывает проекция на пол.

13.2. Скалярное произведение

dot(a, b) = ax·bx + ay·by + az·bz = |a|·|b|·cos θ. Для единичных векторов результат сразу равен cos угла. Если forward NPC = (0, 0, 1), а направление на цель = (0,8, 0, 0,6), то dot = 0,6 и угол приблизительно 53,13°. Для полного поля зрения 90° допустима половина 45°: порог cos 45° ≈ 0,707. Значит, эта цель за пределами конуса, даже если находится достаточно близко.

Почему сравнивать с cos половинного угла, а не с самим углом? Dot даёт косинус, и acos для каждой цели не нужен. Почему нормализовать оба вектора? Без этого dot включает их длины; дальняя цель может пройти порог только из-за большого расстояния. В NpcCombatSystem горизонтальная проверка обзора использует компоненты X и Z. Отдельные лучи затем проверяют, не закрыта ли цель геометрией. Попадание в конус не равно реальной видимости.

Dot полезен и для движения: dot(velocity, directionToTarget) > 0 означает движение в сторону цели. При диагностике застревания одной этой проверки мало: капсула может смотреть вперёд и упираться в дверь. Поэтому измеряй фактический прогресс позиции за интервал времени.

13.3. Векторное произведение и знак поворота

cross(a, b) перпендикулярен обоим векторам. Для a = (1, 0, 0), b = (0, 0, 1) результат в обычной правой алгебраической системе равен (0, −1, 0). Не меняй порядок молча: cross(b, a) = −cross(a, b). Чтобы понять, с какой стороны от взгляда цель на горизонтальной плоскости, удобно взять Y-компонент cross(forward, toTarget), но знак обязан быть проверен на известном тестовом случае для выбранной системы координат и направления осей.

Для треугольника cross(edge1, edge2) даёт геометрическую нормаль. Перестановка двух вершин меняет нормаль и winding; после этого back-face culling может скрыть поверхность. Нормаль нужно нормализовать, если длины рёбер произвольны.

13.4. Проекция и остановка перед целью

Проекция вектора v на единичное направление n равна `dot(v, n) · n`. Для движения вдоль плоскости можно убрать вертикальную компоненту и нормализовать остаток. При подходе к точке не требуй равенства координат: float и физический контроллер редко дают точное совпадение. Введи радиус прибытия, зону замедления и гистерезис. Например, переход в `Repairing` при расстоянии ≤ 1,4 м, а возврат в `MoveToRepair` только при > 1,7 м. Разные пороги предотвращают дрожание состояния на границе; конкретные числа должны согласовываться с `brain.repairDistance` и размером капсулы.

14. Матрицы и Transform

14.1. Что делает матрица 4 на 4

Матрица объединяет масштаб, вращение, сдвиг и однородную координату. Для контракта вектор-строка точка проходит через `p · W · V · P`. Матрицы не коммутируют: `A · B` обычно не равно `B · A`. Проверка на бумаге: точка (1, 0, 0), масштаб 2, перенос +5 по X. «Сначала масштаб, затем перенос» даёт 7; «сначала перенос, затем масштаб» даёт 12. Если объект летит в другую точку после смены порядка, это не небольшая погрешность, а другая операция.

Матрица `W` переводит из model в world. `V` переводит world в пространство камеры; геометрически это обратное преобразование камеры. `P` переводит view в clip и задаёт перспективу. В коде `pbr.vs` последовательность записана явно отдельными `mul`, что удобно для диагностики. На CPU `PBRShaderClass.cpp` перед записью матриц в constant buffer вызывает `XMMatrixTranspose`. Это вопрос размещения матрицы в памяти и соглашения HLSL, а не разрешение менять геометрический порядок на `P · V · W`.

14.2. Обратная матрица

Обратная матрица M^{-1} отменяет действие M: $(p \cdot M) \cdot M^{-1} = p$, если M обратима. Для преобразования мирового луча в локальные координаты модели применяют inverse world к началу луча с w = 1 и к направлению с w = 0, затем нормализуют направление. Матрица с нулевым масштабом необратима; такую ситуацию нужно обнаружить и отвергнуть. Не путай обратную матрицу с транспонированной: перенос и масштаб не отменяются простым обменом строк и столбцов.

14.3. Нормали и неравномерный масштаб

При равномерном масштабе и вращении нормаль часто можно преобразовать вращательной частью W. При масштабе (2, 1, 0,5) так делать нельзя: новая нормаль может перестать быть перпендикулярной поверхности. Правильный общий оператор — inverse transpose для линейной части world matrix с последующей нормализацией. Проверка: касательная t после W и нормаль n после normal matrix должны удовлетворять `dot(t', n') ≈ 0`. Трансляцию к нормали не применяют. Для конкретного PBR пути посмотри `normal_transform.hlsli` и `BuildWorldTangentBasis`: TBN также должен оставаться согласованным после skinning и масштаба.

14.4. Родитель и ребёнок

Мировое преобразование кости или дочернего объекта включает родительское. При соглашении вектор-строка локальное преобразование применяется раньше родительского: `p_world = p_local · M_child · M_parent`. Если вращать корабль, а предмет внутри остаётся на месте, проверь именно составление и обновление родительской матрицы. Для физического тела отдельно реши, кто владеет позицией: Transform, Bullet или вычисленный socket. Запись в оба источника в одном кадре вызывает рывки.

15. Углы, кватернионы и интерполяция

15.1. Градусы и радианы

Полный оборот — 360° = 2π радиан, прямой угол — π/2. `sin`, `cos` и функции DirectXMath ожидают радианы, если API не оговаривает другое. В коде есть показательный переход `XMConvertToRadians(m_cameraController.GetFollowYaw())`: значение yaw камеры переводится перед записью в поворот Transform. Если передать 90 как радианы, получится множество оборотов, а не 90°. В параметрах редактора явно указывай единицы и конвертируй у границы ввода.

15.2. Почему три угла не универсальны

Euler yaw, pitch, roll удобны для редактирования, но результат зависит от порядка осей. Один и тот же набор чисел при `Rx · Ry · Rz` и `Rz · Ry · Rx` даёт разные ориентации. При некоторых сочетаниях две оси совпадают, теряется степень свободы. Для камеры с отдельным yaw и pitch это часто приемлемо; для плавного перехода между произвольными ориентациями и костной анимации удобнее кватернион.

15.3. Кватернион без мистики

Поворот на угол θ вокруг единичной оси u задаётся `q = (u·sin(θ/2), cos(θ/2))`. Для 180° вокруг Y: q = (0, 1, 0, 0). Нормализуй q после накопления операций, иначе длина дрейфует. У q и −q одинаковая ориентация; перед интерполяцией выбери короткую дугу. `slerp(q0, q1, t)` сохраняет равномерный угловой ход лучше простого покомпонентного lerp. Не используй обычный lerp углов через границу −π/π: переход от 179° к −179° должен быть 2°, а не 358°.

15.4. Интерполяция времени

`lerp(a, b, t) = a + (b − a)t` при t от 0 до 1. Для FPS-независимого экспоненциального сглаживания коэффициент кадра `α = 1 − exp(−λ·dt)`. При λ = 4 c^{-1} и dt = 0,0167 с α ≈ 0,0646. При dt = 0,0333 с α ≈ 0,1247: редкие кадры делают больший шаг, поэтому скорость сближения за реальную секунду остаётся похожей. Постоянный коэффициент 0,1 за кадр изменяет поведение при другом FPS.

16. Камера, проекция и луч

16.1. View и перспектива

Камера в мире имеет позицию и направление; view matrix переводит мир так, будто камера находится в начале координат. Перспективная проекция уменьшает удалённые предметы через деление на w. В левой DirectX-конвенции перед камерой обычно положительная ось Z view space, глубина после проекции лежит в [0, 1]. Но конечный знак и оси нужно подтверждать по созданию конкретной camera/view matrix движка, а не переносить из OpenGL или учебника по иной конвенции.

16.2. Построение picking ray

Экранную точку сначала переведи из пикселей viewport в NDC: `x_ndc = 2x/width − 1`, `y_ndc = 1 − 2y/height`. Для центра экрана 1920×1080 при x=960, y=540 получаем (0, 0). Создай ближнюю и дальнюю точки, преобразуй их inverse projection и inverse view, выполни деление на w. Луч имеет origin в мире и направление `normalize(farWorld − nearWorld)`. Не используй просто `(mouseX, mouseY, 1)` как мировое направление: оно ещё в экранных координатах. Физический raycast должен получить длину, фильтр владельца и корректную маску; попадание возвращает геометрию, а не автоматически игровой объект.

16.3. Луч зрения NPC и луч выстрела

Это два разных вопроса. Луч зрения: видна ли точка цели от глаза NPC с учётом препятствий и FOV? Луч выстрела: свободен ли путь от дула и нет ли союзника на линии огня? Глаз над стенкой может видеть цель, а дуло всё ещё упирается в укрытие. Поэтому `NpcCombatSystem` выбирает видимую цель, а `NpcRangedCombatSystem` позже отдельно проверяет дуло и линию огня. При отладке записывай origin, direction, максимальную длину и первую коллизию каждого луча.

17. Геометрия, границы и упражнения

17.1. AABB, sphere и frustum

AABB задаётся минимальными и максимальными координатами по осям. Сфера задаётся центром и радиусом. Сфера удобна для грубого отсева и не требует пересчёта ориентации; AABB плотнее охватывает многие объекты, но после поворота его мировые границы надо обновить. Frustum состоит из плоскостей камеры. Для сферы достаточно проверить расстояние центра до каждой плоскости: если оно меньше −radius хотя бы для одной, объект вне объёма. Эти проверки управляют видимостью рендера и не должны останавливать симуляцию далёких пробоин, NPC или электросети.

17.2. Контрольные задачи с ответами

Задача A. NPC в (−2, 0, 1), цель в (4, 0, 9). Смещение (6, 0, 8), горизонтальная дистанция 10 м, единичное направление (0,6, 0, 0,8). При 5 м/с за 0,1 с желаемое смещение (0,3, 0, 0,4) м. Если получилось (3, 0, 4), забыли умножить на dt.

Задача B. Взгляд (0, 0, 1), цель по направлению (1, 0, 1)/√2. Dot = 1/√2 ≈ 0,7071. При полном FOV 90° цель лежит на границе. Для устойчивого поведения добавь небольшой гистерезис или отдельный порог потери, иначе колебание float и поворота может переключать видимость каждый кадр.

Задача C. Урон от оружия не должен вычисляться из экранной длины луча. Луч в мире длиной 20 м проверяет физические препятствия. Если target за стеной на 5 м, raycast должен сначала вернуть стену; видимый на экране силуэт не отменяет препятствие.

Задача D. Вершина (1, 0, 0) после масштаба 2 и переноса +5 по X должна стать 7. Если получила 12, поменян порядок. После добавления view/projection проверь не только экранную точку, но и промежуточную world position, чтобы локализовать ошибку.

17.3. Как проверять собственную формулу

Запиши пространство каждого входа и выхода; подпиши единицы; проверь нулевой и единичный случай; возьми числа, которые легко посчитать вручную; проверь симметрию или сохранение величин, когда она должна быть. После этого сравни с debug overlay. Например, для ремонта проверь `progress` при двух разных dt за одну и ту же секунду, а для поворота — известные 0°, 90° и 180°. Это быстрее, чем пытаться понять ошибку по одной сложной сцене.

17.4. Поворот в плоскости на числах

Для обычной двумерной записи вектор-столбец повернётся против часовой стрелки по формулам `x' = x·cos θ − z·sin θ`, `z' = x·sin θ + z·cos θ`. При точке (x, z) = (1, 0) и θ = 90° получаем (0, 1). В коде с вектором-строкой матрица для этого же геометрического результата будет транспонирована относительно столбцовой записи. Именно поэтому не стоит копировать матрицу вращения из статьи, где не указано, с какой стороны умножается вектор. Проверка на (1, 0) при 90° сразу раскрывает ошибку знака или транспонирования.

В трёхмерной сцене Y направлена вверх, поэтому горизонтальный поворот идёт в плоскости XZ. Для NPC сначала построй `toTarget = target − position`, обнули Y, проверь длину, нормализуй. Для визуальной модели учти её исходное направление: если импортированная модель смотрит вдоль −Z, геометрически верный yaw движения может требовать дополнительный поворот π. Такой offset лучше задокументировать один раз рядом с моделью или контроллером, чем добавлять разные «магические» 180° в AI, анимации и оружии.

17.5. Движение с ускорением и ограничением скорости

Пусть NPC стартует с v = 0, желаемая скорость 4 м/с, ускорение ограничено 2 м/с². За 0,5 с скорость может вырасти максимум на 1 м/с. Нельзя сразу выставить 4 м/с, если механика требует инерцию. Формула для вектора скорости: `Δv = desiredVelocity − currentVelocity`, затем ограничь длину Δv величиной `maxAcceleration · dt` и добавь к currentVelocity. Если NPC должен тормозить перед рабочей точкой, желаемую скорость уменьши по расстоянию до цели, но оставь минимальный радиус прибытия. Для движений, которые исполняет Bullet controller, этот расчёт задаёт намерение; итоговое смещение может быть меньше из-за столкновения.

Проверка размерности: ускорение 2 м/с², dt 0,5 с, изменение скорости 1 м/с. Если в результате получилось 1 м, смешаны скорость и перемещение. При двух тиках по 0,25 с итоговая скорость должна

приблизительно совпасть с одним тиком 0,5 с при постоянном желаемом направлении; контактная физика при этом может дать небольшие отличия.

17.6. Проекция скорости на нормаль контакта

При скольжении вдоль стены раздели скорость v на нормальную и касательную составляющие. Если единичная нормаль стены n, то `v_normal = dot(v, n)·n`, `v_tangent = v – v_normal`. Например, v = (3, 0, 4) м/с и n = (1, 0, 0): нормальная часть (3, 0, 0), касательная (0, 0, 4). Это объясняет, почему персонаж, упёршийся в стену, может продолжить двигаться вдоль неё. Не применяй формулу без понимания ориентации нормали и без Bullet contact resolution: она полезна как учебная модель, а физический контроллер уже содержит свои правила столкновений, трения и step-up.

17.7. Навигационная и прямая дистанция

Два NPC могут находиться в 3 м по прямой через стену и в 18 м по маршруту через дверь. Utility score по евклидову расстоянию предпочтет ближайшую геометрически цель, хотя она фактически дальше. Сначала можно оставить прямую дистанцию как дешёвый фильтр, но перед окончательным claim проверить достижимость и оценить длину пути. Для A* стоимость `g` — уже пройденный путь, `h` — нижняя оценка остатка, `f = g + h`. Если h равна прямому расстоянию и рёбра имеют не меньшую реальную стоимость, она обычно допустима: не завышает остаток. Но дверь с отдельным временем открывания требует добавлять эту цену в стоимость пути, а не искажать h произвольным штрафом, если нужна гарантия оптимальности.

17.8. Законы сохранения в системах корабля

Математика движка — не только геометрия. Если воздух переходит из комнаты A в B без источника и утечки, сумма масс до и после шага должна совпасть с точностью float. При потоке `q` кг/с за dt секунд из A уходит `q·dt` кг и в B приходит столько же. Если обе комнаты потеряли по `q·dt`, ошибка в знаке. Для энергии аналогично: мощность P в ваттах за dt секунд даёт энергию `E = P·dt` джоулей. Генератор 1000 Вт за 10 с отдаёт 10 000 Дж; аккумулятор в Вт·ч нужно переводить через 1 Вт·ч = 3600 Дж. Нельзя складывать текущую мощность с запасённой энергией как числа одного типа.

При изменении давления разделяй давление, объём, температуру и массу газа. Для простой модели допустимо взять приближение, но напиши его допущения и проверь предельные случаи: закрытая дверь не создаёт поток, одинаковое давление не создаёт направленный поток, утечка выводит массу из системы. Так легче отличить ошибку формулы от ошибки порядка вызовов `PressureSystem` и `FloodingSystem`.

Часть IV. DirectX 11 и рендер

18. Ресурсы и состояния DirectX 11

DirectX 11 — stateful API. Результат draw call зависит не только от переданных вершин, но и от всего состояния context: render targets, depth, blend, rasterizer, shaders, buffers, SRV и samplers.

18.1. Базовые ресурсы

Ресурс	Роль
`IDXGISwapChain`	очередь backbuffers и Present
`ID3D11RenderTargetView`	поверхность, куда пишет pixel shader
`ID3D11DepthStencilView`	depth/stencil testing
`ID3D11Buffer`	vertices, indices, constants
`ID3D11ShaderResourceView`	чтение texture/buffer в shader
`ID3D11SamplerState`	фильтрация и addressing
rasterizer state	culling, fill, bias, scissor
depth-stencil state	depth compare/write и stencil
blend state	смешивание результата pixel shader

Текущий backbuffer — `R8G8B8A8_UNORM`, depth — `D24_UNORM_S8_UINT`, MSAA выключен, основной rasterizer использует `CULL_NONE`.

18.2. Constant buffer alignment

HLSL constant buffer упаковывается блоками по 16 байт. CPU-структура должна совпадать по размеру и layout. Ошибка «создали 64 байта, записали 144» не обязана сразу упасть: она повреждает mapped memory и проявляется случайно.

> **ОГРАНИЧЕНИЕ.** В `SkyShaderClass` существует неиспользуемый legacy-метод, который может записать структуру больше созданного matrix buffer. Текущий рабочий path пишет только матрицу, поэтому дефект латентный.

18.3. COM lifetime

COM-объект освобождается `Release`, но ручной код уязвим к раннему `return`. Целевая форма:

```
Microsoft::WRL::ComPtr<ID3D11Buffer> buffer;
```

`ComPtr` автоматически балансирует reference count и делает partial initialization безопаснее.

18.4. Полностью заданное состояние

Каждый render pass должен явно отвечать:

1. какой RTV/DSV связан;
2. какой viewport;
3. depth test и depth write;
4. blend;
5. culling/fill;
6. какие SRV и samplers заняты;
7. какие slots очищаются после pass.

Иначе следующий pass наследует невидимый остаток предыдущего.

19. Фактический render pipeline

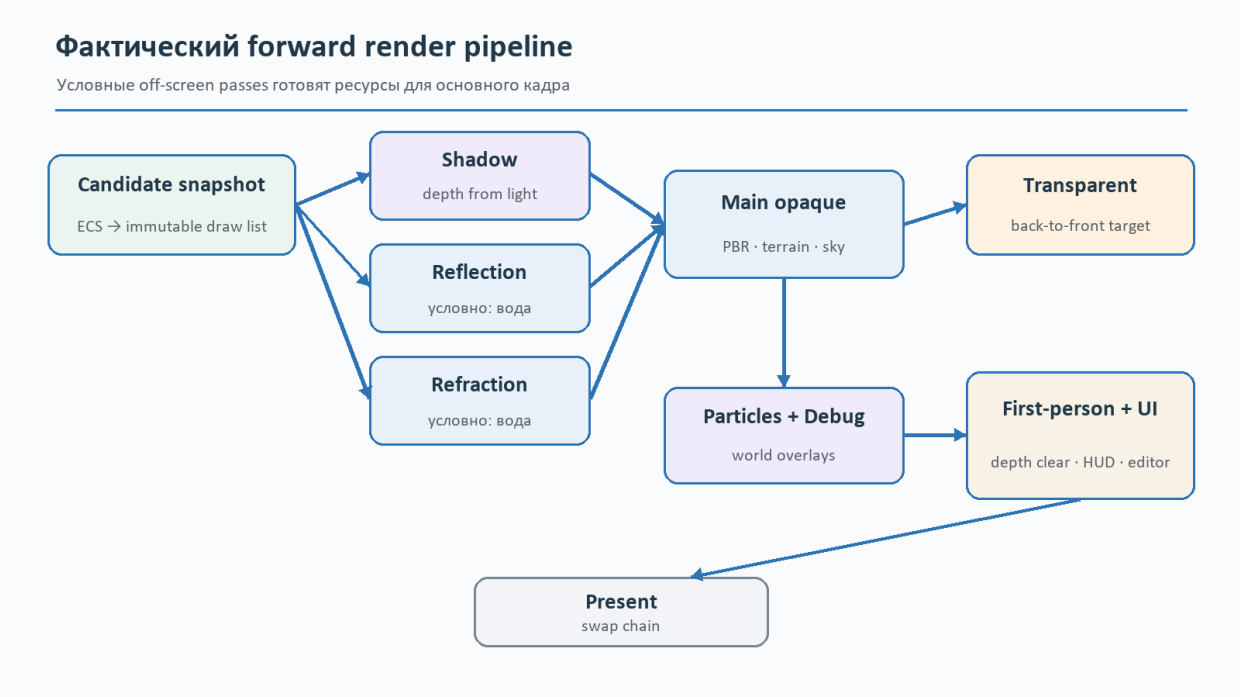


Рисунок 4. Основные проходы forward-рендера.

`Renderer::RenderFrame` реализует forward renderer. G-buffer/deferred lighting в проекте нет.

```
RenderCandidate snapshot/request
  ↓
Sun + environment update
  ↓
Directional shadow map
  ↓
Reflection texture (если нужна)
  ↓
Refraction texture (если нужна)
  ↓
Editor preview (если нужен)
  ↓
Backbuffer clear + equirectangular sky
  ↓
Opaque scene + grass
  ↓
World sprites
```

```
  ↓
Transparent scene
  ↓
Particles
  ↓
Debug geometry
  ↓
Depth clear + first-person geometry
  ↓
UI/HUD + Present
```

19.1. Почему first-person рисуется отдельно

Оружие и руки не должны проваливаться в стены мира. Отдельный pass после очистки depth и с собственной projection делает view model визуально стабильной. Цена — нужны отдельные правила shadows, FOV и clipping; объект first-person нельзя случайно отрисовать ещё раз в world pass.

19.2. Candidate culling

Main thread копирует минимальные render-данные из ECS, worker проверяет frustum, main thread использует только подходящий по camera matrices результат. Это хороший пример безопасной параллельности: worker не трогает D3D immediate context и не обходит живой ECS.

19.3. Room culling

Кроме frustum, `RoomCullingSystem` определяет видимые помещения через связи комнат/дверей. Это portal-like gameplay culling. Оно не должно влиять на саму симуляцию: закрытая за дверью комната может не рисоваться, но oxygen/power/NPC продолжают обновляться.

19.4. Draw context как целевой интерфейс

Сейчас функции получают длинные наборы matrices, arrays и pointers. Безопаснее один объект:

```
struct RenderViewContext
{
    XMMATRIX view;
    XMMATRIX projection;
    XMFLOAT3 cameraPosition;
    LightPacket lights;
    FogPacket fog;
    RenderPassKind pass;
};
```

Так компилятор и validation layer не позволят refraction забыть camera position или light packet.

20. PBR: математика Cook–Torrance

Основной `pbr.ps` реализует metallic-roughness Cook–Torrance с GGX, Fresnel-Schlick и Smith geometry.

20.1. Векторы

```
N = surface normal
V = direction surface → camera
L = direction surface → light
H = normalize(V + L)
```

Все dot products ограничиваются снизу нулём:

```
NdotL = max(dot(N,L), 0)
NdotV = max(dot(N,V), epsilon)
```

20.2. Roughness и GGX distribution

Код использует:

```
a = roughness²
D_GGX = a² / (π · ((N·H)²(a² - 1) + 1)²)
```

`D` описывает долю микрограней, ориентированных как `H`. При малой roughness пик узкий; при большой — широкий.

20.3. Fresnel-Schlick

```
F = F0 + (1 - F0)(1 - cosTheta)⁵
F0 = lerp(0.04, albedo, metalness)
```

Диэлектрик отражает около 4% при прямом взгляде; металл берёт цвет отражения из albedo.

20.4. Smith/Schlick-GGX geometry

```
k = (roughness + 1)² / 8
G1(x) = x / (x(1-k) + k)
G = G1(N·V) · G1(N·L)
```

`G` учитывает masking/shadowing микрограней.

20.5. Specular BRDF

```
specular = D · G · F / max(4(N·V)(N·L), epsilon)
```

Diffuse сохраняет энергию:

```
kS = F
kD = (1 - kS)(1 - metalness)
diffuse = kD · albedo / π
```

Итог одного света:

```
Lo += (diffuse + specular) · radiance · (N·L)
```

20.6. Точечное и spot-затухание

Точечный свет использует художественную модель конечного радиуса:

```
attenuation(d) = (1 - d/R)², если d < R; иначе 0
```

Это не физический `1/d²`, зато свет строго заканчивается на `R` и удобен редактору.

Spot factor:

```
smoothstep(cosOuter, cosInner, coneDirection · L)
```

20.7. Ограничение количества lights

Shader поддерживает одно солнце и четыре point/spot lights. `SceneManager::GatherLightsForShader` берёт первые четыре активных ECS-источника, а не ближайшие/важнейшие. При пятом источнике результат зависит от порядка `unordered_map` и может «прыгать». Целевая политика: score по влиянию на объект/камеру и стабильная сортировка.

21. Цвет, HDR и IBL

21.1. Linear и sRGB

Физика света выполняется в linear space. Обычная цветовая текстура хранится в sRGB и декодируется при sample; normal/metal/roughness — linear data.

Правильный pipeline:

```
sRGB textures → decode to linear
lighting/blending in linear HDR
exposure + tone mapping
linear → sRGB output
```

21.2. Текущий pipeline

PBR shader выполняет ACES-like tone mapping и gamma внутри каждого draw, а пишет сразу в UNORM backbuffer. Следствия:

- прозрачность смешивается после gamma;
- невозможно корректно применить общий exposure;
- bloom/postprocess не видит исходную HDR-энергию;
- разные shader families могут иметь разные color transforms.

Целевая модель: общий HDR scene target и ровно один финальный postprocess.

21.3. Environment map

Текущий sky — один equirectangular HDR. PBR читает его как environment reflection, но не использует:

- diffuse irradiance convolution;
- prefiltered specular mips;
- BRDF integration LUT.

Roughness поэтому в основном ослабляет отражение, а не физически размывает его.

Split-sum IBL:

```
ambientDiffuse  = irradiance(N) · albedo
ambientSpecular = prefiltered(R, roughness) · (F · brdf.x + brdf.y)
```

21.4. Несовпадение ориентации HDR

Sky shader добавляет к U примерно `+0.25`, а PBR direction→equirect UV — нет. Видимая панорама и отражение повернуты относительно друг друга примерно на 90°. Должна существовать одна общая функция mapping и один rotation parameter.

22. Тени

22.1. Текущая модель

- один directional light;
- orthographic shadow map 1024×1024;
- 2×2 PCF;
- receiver slope bias.

Light matrices:

```
V_light = LookAtLH(center - direction·distance, center, up)
P_light = OrthoLH(size, size, near, far)
```

Receiver bias:

```
bias = max(0.00015, 0.0008 · (1 - N·L))
```

22.2. Подтверждённые ограничения

1. Shadow volume центрирован возле world origin, а не camera frustum.
2. Casters предварительно отсекаются main-camera frustum; объект за камерой может давать видимую тень, но быть удалён.
3. Нет texel snapping — возможен shimmering.
4. Shadow pass не skinning-aware — animated mesh даёт bind-pose shadow.
5. Нет alpha cutout.
6. PCF предполагает фиксированное разрешение.
7. Grass не включён полностью в shadow model.

22.3. Путь развития

Сначала camera-fitted stabilized single cascade. Только после стабильности — CSM:

```
near cascade: высокая детализация
middle: средняя
far: большая площадь
```

23. Прозрачность, reflection и refraction

23.1. Правильная alpha transparency

Для обычного `srcAlpha / (1-srcAlpha)`:

```
opaque:       depth test ON, depth write ON
transparent:  depth test ON, depth write OFF, sort back-to-front
```

На срезе transparent objects не сортируются и продолжают использовать depth-write state. Если модель имеет хотя бы одну прозрачную embedded part, весь объект может уйти в transparent pass, включая непрозрачные части.

23.2. Refraction defect

В refraction path локальные arrays lights объявлены, но не заполняются до передачи в render. Camera position также остаётся default `(0,0,0)`. PBR specular/Fresnel получают случайный или неверный input. Это P0-дефект, потому что Debug/Release могут показывать разные картинки.

23.3. Reflection camera

Для горизонтальной плоскости `y=h`:

```
C'_y = 2h - C_y
```

View matrix отражается, но PBR должен получить и отражённую camera position. Текущий исключающий scene path теряет входную позицию и передаёт ноль.

23.4. Вода

Текущий water shader смешивает reflection/refraction почти постоянным коэффициентом. Более полная модель использует Fresnel:

```
F = F0 + (1-F0)(1-N·V)⁵
color = lerp(refraction · absorption, reflection, F)
```

Дальше добавляются depth/thickness absorption, normal waves и foam. Но сначала нужно исправить cameras, clipping и render states.

24. Частицы и взрывы

24.1. CPU simulation

Максимум около 4096 частиц. Интегрирование:

```
v_next = (v + a·dt) · exp(-drag·dt)
x_next = x + v_next·dt
```

Emission accumulator:

```
A += rate·dt
spawn = floor(A)
A -= spawn
```

Это сохраняет дробную часть и делает среднюю скорость emission устойчивой к FPS.

24.2. Billboard

Right/up извлекаются из inverse view. На каждую частицу строится quad из шести CPU-вершин. Это просто и понятно, но масштабируется хуже instancing.

24.3. Случайное распределение

Нормализованная точка равномерного куба **не** равномерна на сфере. Корректный способ:

```
z = 1 - 2u
phi = 2πv
dir = (sqrt(1-z²)cos(phi), z, sqrt(1-z²)sin(phi))
```

Для равномерного объёма шара радиус:

```
r = R · cbrt(u)
```

а не `R·u`.

24.4. Сортировка

Renderer сначала сортирует alpha particles far-to-near, но затем группирует их по texture через map. Между разными текстурами глобальный порядок теряется. Нужен либо stable batching с ограничениями, либо OIT.

24.5. Explosion impulse

```
J = J0 · (1 - d/R)²
```

Текущая модель не проверяет line of sight, поэтому импульс проходит через стену. Для engine-level explosion query нужны physics overlap + ray/occlusion и точка приложения силы.

25. Скелетная анимация

25.1. Local и global bone transforms

Position/scale интерполируются линейно, rotation — SLERP. Local matrix:

```
M_local = S · R · T
M_global = M_local · M_parent
```

Skin matrix в row-vector convention:

```
M_skin = M_offset · M_animatedGlobal
```

25.2. Vertex blending

До четырёх bone influences:

```
p_skinned = Σ weight_i · (p · M_skin_i)
```

Weights нормализуются importer-ом.

25.3. Ограничения

- HLSL buffer рассчитан на 128 bones;
- importer не всегда выдаёт явную ошибку при превышении;
- bone ID ≥128 пропускается;
- shadow/refraction/transparent passes не везде получают current bone buffer;
- в `assimpmodelclass.cpp` остаётся legacy hierarchy path с другим порядком matrices.

Проверка должна одновременно показывать персонажа напрямую, в reflection/refraction и shadow.

Часть V. Bullet и симуляция мира

26. Bullet Physics и синхронизация

26.1. World settings

```
gravity = (0, -9.81, 0) m/s²
fixed step = 1/60 s
solver iterations = 50
split impulse = enabled
ERP ≈ 0.2
```

26.2. Типы тел

Тип	Кто задаёт transform
static	создаётся один раз; мир не двигает
kinematic	ECS/editor задаёт каждый кадр, Bullet только сталкивает
dynamic	Bullet — источник истины; ECS получает результат

Нельзя одновременно каждый кадр телепортировать dynamic Transform из ECS и ожидать честную физику. В проекте есть детектор внешнего изменения: editor teleport обновляет Bullet и сбрасывает velocities.

26.3. Collision shapes

Поддерживаются box, sphere, cylinder, capsule, plane, convex hull, triangle/GImpact mesh, compound, StairRamp и объявленный VHACD.

Правила:

- static concave: triangle mesh допустим;
- dynamic concave: GImpact дорог, convex decomposition предпочтительнее;
- capsule: cylinder height не может быть отрицательной;
- sphere при non-uniform scale либо использует max scale, либо перестаёт быть sphere;
- compound child должен учитывать local center/rotation.

> **ОГРАНИЧЕНИЕ.** VHACD присутствует в enum/editor, но основной path не создаёт полноценную shape во всех ветках. Функция выглядит доступной, хотя физически может отсутствовать.

26.4. Настройки компонента

`RigidBodyComponent` хранит friction, damping, CCD и angular factor, однако create path на срезе часть значений задаёт константами. UI-поле, которое не влияет на runtime, опаснее отсутствующего поля: разработчик верит ложной настройке.

26.5. Interior collider

Код создаёт `interiorBody`, но аудит не нашёл регистрацию `world->addRigidBody(interiorBody, ...)`. Выделенный объект без регистрации не участвует в collision detection.

26.6. Ownership

Безопасный teardown:

```
world.removeConstraint/removeAction/removeRigidBody
→ delete controller/constraint/body motion state
→ delete body
→ delete shapes/triangle meshes
```

Прямой reset Registry опасен для character ghost/action и constraints, потому что их destructors не все снимают registration.

27. Character controller

27.1. Геометрия

Персонаж — вертикальная capsule. Полная высота:

```
H_total = H_cylinder + 2R
```

Центр capsule смещён относительно feet Transform. Поэтому попытка «починить провал» случайным подъёмом всей capsule ломает ступеньки, camera anchor и reach.

27.2. Ground test

Поверхность walkable, если:

```
dot(normal, up) >= cos(maxSlope)
```

27.3. Auto-step

Последовательность sweep:

```
up → forward → down → validate slope
```

Это хорошая engine-функция: один controller используется игроком и NPC, поэтому они сталкиваются с миром одинаково.

27.4. Stabilization

Код содержит post-step ground correction и dead zone, появившиеся из реального бага дрожания. Важный урок: прежде чем добавлять smoothing, определить источник:

- conflict ECS/Bullet ownership;
- contact normal noise;
- capsule center offset;
- camera reading pre-physics pose;
- animation root motion.

27.5. Прыжок

Сейчас `wantJump` формируется из `GameInputFrame::Down(GameAction::Jump)`. После приземления удержанная кнопка может запустить новый прыжок. Если нужен прыжок только по новому нажатию, замени это место на `Pressed` и проверь поведение при паузе и F2.

28. Давление, затопление и плавучесть

28.1. Гидростатическое давление

```
P = P0 + ρgh
```

Где:

- `ρ ≈ 1000 kg/m³` для воды;
- `g = 9.81 m/s²`;
- `h` — глубина в метрах;
- результат — Pa.

`PressureSystem` следует этой модели.

28.2. Скорость через пробоину

Из Bernoulli/Torricelli:

```
ΔP = ρgh
v = sqrt(2ΔP/ρ) = sqrt(2gh)
Q = Cd · A · v
```

Если `size` — радиус:

```
A = πr²
```

28.3. Подтверждённая ошибка единиц

`FloodingSystem` вычисляет:

```
pressure = depth · 9.81
velocity = sqrt(2 · pressure / 1000)
```

В первой строке отсутствует `ρ`, а во второй оно делится ещё раз. Полученная скорость примерно в `sqrt(1000) ≈ 31.6` раза меньше Torricelli.

Пример при `h=10m`:

```
правильно: sqrt(2·9.81·10) ≈ 14.0 m/s
текущий код: sqrt(2·9.81·10/1000) ≈ 0.443 m/s
```

28.4. Water volume

```
V_next = clamp(V + (Q_in - Q_pump)dt, 0, V_room)
waterLevel = (V / V_room) · roomHeight
```

Нужен guard `V_room > epsilon`, иначе division by zero.

28.5. Архимедова сила

```
F_b = ρwater · g · V_displaced
```

Текущая модель:

```
submersion = clamp((waterLevel - bottom)/height, 0, 1)
F_b = ρgV · submersion · multiplier
F_net_y = F_b - ρwater·V_inside·g
```

28.6. Двойной учёт воды

`FloodingSystem` уменьшает `buoy.volume` на внутренний объём воды. `PhysicsSystem` одновременно оставляет отдельный вес внутренней воды. Физически затопление должно либо:

- уменьшать эффективную герметичную displacement volume;
- либо добавлять массу воды;
- либо использовать подробно моделируемый flooded volume,

но не применять упрощённые эффекты дважды без вывода. На срезе потеря плавучести и дополнительный вес суммируются.

28.7. Центр плавучести

Сила сейчас прикладывается в центре массы, поэтому не создаёт восстанавливающий момент. Более выразительная модель использует несколько sample points или вычисленный center of buoyancy:

```
τ = (centerBuoyancy - centerMass) × F_b
```

Асимметричное затопление тогда создаёт крен.

29. Комнаты и атмосфера

29.1. Текущая игровая модель

`RoomComponent` хранит normalized oxygen/pressure, temperature, bounds и связи, но не массу газа и физический volume. Поэтому это **игровая диффузионная модель**, не CFD и не thermodynamics solver.

29.2. Парный перенос

Для двух комнат:

```
transfer = (A - B) · rate · doorFlow · dt
```

Transfer ограничивается половиной разности, а изменения сначала собираются в список и применяются затем. Это даёт два плюса:

- A→B и B→A не зависят от порядка обхода;
- закрытая пара сохраняет сумму величины до clamp/источников.

29.3. Leak

Пробоина использует fraction loss:

```
lost = clamp(leak · dt, 0, 0.95)
pressure *= (1 - lost)
oxygen   *= (1 - lost)
```

Это понятно для gameplay, но не учитывает physical volume и нелинейный массовый расход.

29.4. Breathing и generators

Breathing вычитает normalized oxygen на entity, а generator равномерно раздаёт output активным комнатам. Комната в 10 м³ и 1000 м³ реагирует одинаково. Если понадобится физичность, сначала добавить `volumeM3` и хранить moles/mass, затем вычислять fractions.

29.5. Conservation test

Для двух закрытых комнат без source/sink:

```
sumBefore = amountA + amountB
simulate 10 minutes
abs(sumAfter - sumBefore) / sumBefore <= 0.001
```

Если используются normalized fractions, test должен весить комнаты по volume.

30. Энергетика

30.1. Power grid

Каждый ship root имеет свою сеть. Генераторы дают MW, consumers требуют MW, priority определяет порядок. Внутри одного priority используется water-filling: available power делится поровну, полностью насыщенные consumers удаляются, остаток перераспределяется.

30.2. Battery units

Power — MW, energy — MWh:

```
ΔE_MWh = P_MW · dt_seconds / 3600
```

Код `EmergencyBatterySystem` использует именно это преобразование для charge/discharge. Это пример хорошего явного unit contract.

30.3. Brownout

```
powerRatio = suppliedMW / requiredMW
hasPower = powerRatio >= 0.999
brownout = totalSupplied < totalDemand
```

Gameplay subsystem не должен самостоятельно забирать мощность «мимо» grid; он читает `powerRatio`.

30.4. Engine-level развитие

- stable consumer IDs;
- event на переход powered↔brownout, а не каждый кадр;
- граф dependency/isolated grids;
- telemetry history;
- deterministic allocation независимо от unordered order.

31. Damage, cutting и repair

31.1. Damage flow

```
Weapon/raycast
→ DamageEvent
→ DamageSystem
→ health / hull panel / decals / particles / world event
→ breach or destroyed state
```

31.2. Runtime cutting

F6 строит picking ray и вызывает `MeshCuttingTest::CutByRayAsync`. Старое руководство, где указан синхронный `CutByRay`, устарело.

Концептуальные этапы cutting:

1. ray/triangle intersection;
2. выбор области/плоскости разреза;
3. clipping polygons;
4. interpolation новых attributes;
5. triangulation;
6. rebuild CPU/GPU mesh;
7. rebuild physics shape;
8. запись operation ID/history;
9. создание breach/damage representation.

31.3. Интерполяция атрибутов

Для точки на ребре:

```
p(t) = a + t(b-a)
uv(t) = uvA + t(uvB-uvA)
n(t) = normalize(nA + t(nB-nA))
```

Если используется barycentric interpolation внутри triangle:

```
p = uA + vB + wC,  u+v+w=1
```

31.4. Асинхронность

В текущем коде async cutting уже разделён на две фазы. Main thread выбирает цель и снимает CPU-snapshot вершин и индексов. Worker выполняет clipping и triangulation только на этих копиях. Completion возвращается на main thread, проверяет `ActiveComponent` и то, что entity всё ещё ссылается на ту же model, и только после этого меняет D3D buffers, history и Bullet collider.

Async repair добавляет более сильную optimistic validation: worker запоминает `expectedRevision`, а main-thread completion до первой мутации проверяет revision и наличие repair operation у всех целей. Это защищает от применения устаревшей геометрии.

Scene/entity generation token пока не стал общим engine-контрактом. Локальные проверки model pointer и revision закрывают текущие cutting/repair paths, но новая async-система должна либо получить generation token, либо доказать эквивалентную проверку lifetime. Общая модель worker/completion разобрана в главе 35.

31.5. Repair invariants

- одна operation repair не применяется дважды;
- удалённая entity не получает completion;
- GPU buffer меняется только main thread;
- collider перестраивается после mesh;
- save во время repair имеет определённое состояние;
- visual crack, breach simulation и geometry history согласованы.

Часть VI. NPC, данные и инструменты

32. NPC AI pipeline

Фактический порядок:

```
NpcUtilitySystem
→ CrewTaskCoordinator
→ StateSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
```

32.1. Utility AI

Idle score примерно `0.1`. Repair breach score сочетает severity и distance; equipment repair — damage, importance и distance. Utility выбирает не только пробоины, как говорит старый NPC guide, но и повреждённое оборудование.

Общая нормализация:

```
score = Σ wi · normalizedCriterion_i
Σ wi = 1
```

Distance utility, например:

```
distanceScore = 1 - clamp(distance / detectionRange, 0, 1)
```

32.2. Hysteresis

Без hysteresis две близкие задачи меняются местами каждый кадр:

```
switch only if newScore > currentScore + threshold
```

Также полезны minimum commitment time и cooldown.

32.3. Manual command

Debug/manual command имеет приоритет. Utility продолжает считать scores для UI, но не перехватывает управление. Это хорошее разделение decision telemetry и authority.

32.4. Чего пока нет как законченной системы

Старая документация описывает fatigue, fear, перенос инструментов и общий World Task Board так, будто они реализованы. На текущем срезе это всё ещё целевая модель, но reservation ремонта уже появился.

`CrewTaskCoordinator` собирает активные `RepairableComponent`, сортирует NPC по `Entity`, исключает hostile/off-duty/manual-command персонажей и закрепляет цель через `CrewTaskClaimComponent`. `CrewMemberComponent` хранит роль, навык, текущую работу и счётчик завершённых работ. Это уже предотвращает ситуацию, когда два дружественных NPC одновременно выбирают одну пробоину или одно оборудование.

Важно точно различать два уровня:

- `NpcUtilitySystem` оценивает индивидуальную полезность ремонта и оставляет диагностический score;
- `CrewTaskCoordinator` выполняет межагентный арбитраж и назначает уникальные цели;
- общего task board для пожаров, лечения, переноски ресурсов и охраны пока нет;
- claim существует только для ремонтируемой цели и пересоздаётся из состояния мира, а не является полноценной долговременной задачей.

Поэтому новый тип коллективной работы следует добавлять либо расширением координатора, либо будущим единым task board. Нельзя создавать второй скрытый `unordered_map<Entity, Entity>` рядом: тогда UI, save/load и другие AI-системы перестанут видеть единственный источник истины.

33. StateSystem и исполнение

Utility отвечает «что делать», StateSystem — «как выполнить».

Типичный repair state:

```
Acquire target
→ build room route
→ build/follow NavMesh path
→ request/open door
→ approach work zone
→ execute timed repair
→ validate target
→ success/fail/cancel
```

State должен иметь:

- enter/exit;
- timeout;
- reason code;
- target validation;
- cancellation cleanup;
- debug-readable progress.

Файл `StateSystem.h` вырос примерно до 1845 строк. Разделение по state handlers уменьшит coupling и compile time.

34. Recast/Detour NavMesh

34.1. Параметры build

```
cellSize = 0.20 m
cellHeight = 0.10 m
agentHeight = 1.8 m
agentRadius = 0.42 m
agentClimb = 0.30 m
maxSlope = 45°
```

Они должны соответствовать capsule controller. Если radius NavMesh меньше физической capsule, найденный путь может не пройти.

34.2. Pipeline

```
input triangles
→ rasterized heightfield
→ compact heightfield
→ erode by agent radius
→ regions
→ contours
→ polygon mesh
→ detail mesh
→ Detour nav data
```

34.3. Query

```
findNearestPoly(start/target)
→ findPath (A*)
→ findStraightPath (funnel-like corners)
```

Лимиты на срезе: около 256 polygon refs и 64 straight points. Длинный путь может быть обрезан.

34.4. Ограничения

- obstacle footprint строится из world AABB, а не oriented geometry;
- doors исключаются из baked geometry;
- нет tile cache/dynamic carving;
- нет явных off-mesh door links;
- fallback может построить только пол без препятствий;
- локальное avoidance выбирает ближайший dynamic AABB, но это не RVO crowd.

34.5. Debug colors

NavMesh/path/obstacles рисуются one-frame debug lines. Цвет fallback-сетки должен отличаться от полноценной, чтобы «путь сквозь мебель» не выглядел загадочным AI-багом.

35. JobSystem и асинхронные конвейеры

> ФАКТ. Эта глава актуализирована после engine-maturity коммита 9913264. Асинхронность в этом движке — не «запустить всё в другом потоке», а явно разделить CPU-подготовку и применение результата к живому engine state.

35.1. Thread affinity: почему нужны две очереди

Worker безопасен только тогда, когда работает с независимыми данными. Сам факт переноса функции в другой поток не делает ECS, Direct3D или Bullet thread-safe.

Работа	Поток	Почему
чтение/parse файла, clipping, frustum test, CPU mesh processing	worker	работа идёт с immutable snapshot или локальным result
публикация CPU-result в completion queue	worker	передаёт владение результатом, но не меняет engine state
ECS structural change	main	component pools и entity lifetime принадлежат main thread
D3D immediate context/GPU buffer creation	main	immediate context и render state имеют одного владельца
Bullet world mutation	physics/main owner thread	world хранит внутренние ссылки на bodies, actions и constraints

Работа	Поток	Почему
ImGui	main	UI живёт в main-frame lifecycle

Отсюда следует главный контракт:

```
main snapshot → worker CPU result → main-thread validation → ECS/D3D/Bullet apply
```

35.2. Владение и lifecycle JobSystem

`ApplicationClass` хранит `JobSystem` как value member и запускает его с четырьмя workers. Число «4» — это конфигурация приложения, а не ограничение класса: `Initialize(workerCount)` принимает любое положительное число.

Важные гарантии lifecycle:

1. Новый или уже остановленный `JobSystem` имеет `stop=true`, поэтому отвергает job, а не кладёт его в очередь без workers.
2. `Initialize(0)` и отрицательное число workers отвергаются.
3. Повторный валидный `Initialize` сначала выполняет `Shutdown`, затем сбрасывает stop/active count и создаёт новые потоки.
4. Если создание одного из потоков бросит исключение, уже созданные workers останавливаются и join-ятся, а `Initialize` возвращает `false`.
5. Деструктор вызывает idempotent `Shutdown`; копирование и перемещение запрещены.

Концептуальная машина состояний:

```
Stopped/rejecting
      | Initialize(N > 0)
      ▼
Running/accepting ── Shutdown ──► Draining queued jobs ── join ──► Stopped/rejecting
```

35.3. Как worker извлекает job

Worker ждёт condition variable по предикату `stop || !jobs.empty()`. Очередь `jobs` и флаг `stop` защищены одним `queueMutex`. Job извлекается по FIFO, счётчик `activeJobs` увеличивается, после чего мьютекс отпускается. Тяжёлая функция выполняется вне lock, иначе все workers фактически стали бы одним потоком.

```
lock → wait → pop → activeJobs++ → unlock
                        │
                        └→ execute job
                                │
lock ← notify Wait if idle ← activeJobs-- ←┘
```

`Shutdown` ставит `stop=true` и будит workers. Worker завершается только при `stop && jobs.empty()`: уже принятые CPU-jobs дренируются, но новые `SubmitJob` отвергает.

35.4. Wait не равен completion fence

В движке две независимые очереди:

- `SubmitJob` кладёт CPU-работу в worker queue;
- `SubmitMainThread` кладёт completion в main-thread queue.

`Wait()` ждёт только `jobs.empty() && activeJobs == 0`. Он не запускает main-thread callbacks. Поэтому «`Wait` вернулся» означает «CPU-фаза закончена», но не «результат уже применён к сцене». Вызывать `Wait()` из worker job нельзя: текущий job сам учтён в `activeJobs`, поэтому будет ждать собственного завершения.

`ExecuteMainThreadJobs()` под lock меняет shared queue на локальную, затем выполняет callbacks без удержания mutex. Callback, добавленный во время этого прохода, останется до следующего вызова. Это убирает re-entrancy и долгое удержание `mainThreadMutex`.

35.5. Фактические async-конвейеры

Потребитель	Worker-фаза	Main-thread применение	Защита от устаревшего результата
`AssetLoadSystem`	Assimp/file parse и CPU mesh	completion ставит `CpuLoaded`, `Update()` создаёт GPU buffers	tracker и state; completion захватывает raw `this`, поэтому важен shutdown-порядок
async ship import	Assimp читает GLB, готовит nodes, collision proxies и panel batches	ECS/Bullet/GPU создаются фазами с бюджетом 4 ms на кадр	state живёт в `shared_ptr`; `Clear()` отцепляет старый state от новой сцены
`RenderCandidateSystem`	frustum test по snapshot transform/bounds	completion публикует cache visible entities	cache используется только при близких view/projection; иначе fail-open считает entity кандидатом
async cutting	clipping копии CPU mesh	проверка entity/model, GPU apply, history и collider rebuild	active entity + тот же model pointer
async repair	копирование baseline и replay оставшихся cuts	валидация всех целей, GPU apply и collider rebuild	`expectedRevision`, operation ID/state и per-model pending lock

`AssetLoadSystem::RequestMeshLoad` реализован, но в текущем editor spawn path флаг `useAsyncMesh` выключен. Поэтому generic mesh loader — готовый, но не основной runtime path. Активные async-пути сегодня — ship import, render candidates, cutting и repair.

35.6. Порядок внутри кадра и shutdown

В начале `ApplicationClass::Frame` порядок такой:

1. `ExecuteMainThreadJobs()` применяет готовые completions.
2. `AssetLoadSystem::Update(device)` создаёт GPU buffers для состояния `CpuLoaded`.
3. `UpdateAsyncShipImport(..., 4.0f)` тратит ограниченный main-thread budget на текущую фазу импорта.

Порядок shutdown ещё важнее:

```
JobSystem.Shutdown()          // reject new jobs, drain accepted jobs, join workers
ExecuteMainThreadJobs()       // apply every completion produced before join
SceneManager.Clear()          // release ECS/Bullet runtime owners
PhysicsWorld.Shutdown()
AssetLoadSystem.Shutdown()
```

Так raw `this` в asset completion не становится dangling в штатном `ApplicationClass` lifecycle: workers уже join-нуты, completions выполнены, и только затем уничтожаются scene/assets. Вынос `AssetLoadSystem` в другой lifetime потребует новой гарантии.

35.7. Проверяемые контракты

`tests/CoreContractsTests.cpp` теперь доказывает базовый lifecycle `JobSystem`:

- job отвергается до `Initialize`;
- неположительное число workers отвергается;
- запущенные workers выполняют все принятые jobs;
- после `Shutdown` новая работа отвергается;
- повторный `Initialize` снова принимает и выполняет job.

Контракты запускаются для Debug и Release x64 через:

```
.\scripts\verify_engine.ps1 -Configuration All
```

Это уже автоматический test target, но не полное async test suite и не внешний CI.

35.8. Текущие ограничения

1. Исключение из worker job не перехватывается `JobSystem`: обычно это завершит process, а `activeJobs--` не будет выполнен.
2. Исключение из main-thread completion также не изолировано и выйдет в `ApplicationClass::Frame` или `Shutdown`.
3. Нет cancellation token: принятый job при shutdown дренируется, а не отменяется.
4. Нет priority queue, dependency graph, task handle/future и общего канала ошибок.
5. Приложение всегда запускает четыре workers и не учитывает `hardware_concurrency` и нагрузку на render thread.
6. `SubmitMainThread` не проверяет stop-state и не возвращает status; вне штатного application lifecycle completion может остаться без consumer.
7. Нет универсального scene/entity generation token. Cutting и repair имеют локальную валидацию, ship import изолирует state через `shared_ptr`, но новый async-consumer не получает эту защиту автоматически.
8. Static cache `RenderCandidateSystem` не имеет scene generation и не сбрасывается в `SceneManager::Clear`; матричный fail-open снижает риск невидимого объекта, но не заменяет lifecycle token.

Целевая форма completion для новых систем:

```
RequestId + sceneGeneration + entityGeneration/revision + immutable CPU result + error
```

36. Scene save, runtime save и Entity remap

36.1. Текущее scene save

Schema `version=1`, объекты собираются вручную из нескольких pools. `SerializeObject` покрывает много компонентов, затем `LoadScene` создаёт новые Entity и строит map:

```
oldEntityId → newEntityId
```

После создания выполняется `RemapEntityReferences`.

36.2. Подтверждённые пробелы

- loader читает `world`, saver его не пишет;
- `WorldAreaComponent` не сохраняется;
- `SerializeMaterial` — пустой stub;
- sprites не входят в полноценный round trip;
- одна entity может попасть в objects через несколько component pools;
- JSON exception не перехватывается;
- текущая сцена сбрасывается до полной проверки новой;
- прямой Registry reset обходит специальный Bullet teardown;
- version не мигрируется.

36.3. Транзакционная загрузка

```
read bytes
→ parse JSON
→ validate schema/version
→ build temporary scene/registry
→ validate references/resources
→ atomically swap current scene
→ destroy old scene safely
```

36.4. Атомарная запись

```
write save.tmp
→ flush + close
→ rename/replace save.json atomically
```

Повреждение `save.tmp` не уничтожает предыдущий рабочий save.

36.5. Scene и runtime save

Scene asset хранит authoring data. Runtime save дополнительно хранит:

- world tick/time;
- velocities;
- health/damage;
- atmosphere/water/power;
- AI tasks/memory/reservations;
- in-progress operations;
- random seeds.

Смешивать два формата без явной границы опасно.

37. Editor, picking и диагностика

37.1. Редактор

`EditorUI.cpp` — крупнейший hotspot. Он показывает components, создаёт requests, управляет inspectors и debug panels. Хороший принцип уже используется: тяжёлая операция из UI оформляется request и исполняется application/scene layer, а не внутри ImGui draw.

37.2. Gizmo и physics

Editor меняет Transform. Если entity dynamic, physics layer должен распознать teleport, обновить body/AABB, сбросить или сохранить velocities по явной политике. Изменение scale требует rebuild collision shape.

37.3. Диагностический минимум объекта

```
Entity + generation/alive
Name + Active
Transform + units
world bounds
material/shader/textures
Bullet body/shape/type/mass/velocity
room/ship membership
AI state/target/path/reason
save support yes/no
last system that modified critical value
```

37.4. Debug UI не должен быть логикой

UI может вызвать command, но правило симуляции должно жить в system. Иначе headless test и NPC не смогут использовать ту же механику.

Часть VII. Срез качества и диагностика

38. Сильные стороны движка

1. Порядок кадра виден в одном координаторе.
2. ECS отделяет данные от большинства систем.
3. Bullet использует fixed step и accumulator clamp.
4. Character controller общий для player/NPC.
5. Async render culling работает по snapshot, не по живому ECS.
6. CPU asset loading отделено от GPU upload.
7. Editor/Play — явные режимы.
8. PBR shader содержит реальную Cook–Torrance математику.
9. Render pipeline уже многопроходный.
10. Entity references remap-ятся после load.
11. Корабельные systems образуют причинные цепочки.
12. Debug overlays существуют для physics/NavMesh/objects.

39. Реестр слабых мест

Приоритет	Слабое место	Симптом	Главная проверка
P0	Release x64 не настроен	C1083/link/subsystem	clean Debug+Release matrix
P0	partial init raw pointers	случайный crash в Shutdown	fail Initialize после каждого шага
P0	scene reset обходит Bullet teardown	crash/dangling после load	100 load/unload + counters
P0	FileSystem path traversal	delete вне assets	`..`/absolute/symlink tests
P0	refraction uninitialized lights/camera	случайный PBR	deterministic render capture
P0	Euler order render/Bullet	collider не совпадает	random compound rotation test
P0	flooding units/double count	слишком медленный/тяжёлый flood	analytical Torricelli/buoyancy tests
P1	transparent depth/order	исчезающее стекло	creation-order invariant image
P1	forces outside fixed tick	FPS-dependent physics	30/60/144 comparison
P1	incomplete serialization	потеря world/material/area	round-trip + corruption test
P1	Entity without generation	stale ID becomes valid	reuse test
P1	normal matrix	неверный свет при scale	inverse-transpose reference
P1	shadow at origin/no skinning	тени исчезают/старая поза	long-distance animated test
P1	current-directory assets	EXE не запускается отдельно	launch from output/temp

Приоритет	Слабое место	Симптом	Главная проверка
P1	jobs without exception/cancel	terminate/hanging Wait	throwing/cancel stress test
P2	monolithic coordinators	дорогие правки/merge conflicts	module dependency budget
P2	duplicate vendored ImGui	drift/size	single dependency source
P2	mixed encoding	C4566/mojibake	UTF-8 compile policy
P2	узкое test coverage / no CI	сложные регрессии всё ещё узнаются вручную	расширять contracts/integration tests + CI gate

39.1. Самые важные пять

Если времени только на пять исправлений:

1. воспроизводимый Release/runtime package;
2. безопасный scene/Bullet teardown;
3. тестовый target;
4. единые rotation/time/fixed-step contracts;
5. refraction/transparency/render-state correctness.

40. Производительность без угадывания

40.1. CPU budgets

Измерять отдельно:

- input/editor;
- AI/NavMesh;
- fixed physics;
- scene systems;
- render preparation/culling;
- draw submission;
- UI;
- main-thread asset upload.

40.2. GPU budgets

- shadow;
- reflection/refraction;
- opaque;
- transparent;
- particles;
- first person;
- postprocess.

40.3. Counters

```
entities/components
Bullet bodies/actions/constraints
render candidates/culled/draw calls/triangles
jobs queued/active/failed
assets queued/ready/failed
GPU buffers/textures/SRV
```

40.4. Hotspot strategy

Сначала profiler, затем изменение. Например, `CULL_NONE` удваивает часть raster workload, но его нельзя просто включить глобально: импортированные winding/inside surfaces должны иметь явный material culling mode.

41. Как диагностировать симптом

Объект не виден

1. Active?
2. Transform finite и scale > 0?
3. model/buffers ready?
4. material/shader/SRV?
5. main/room/frustum culling?
6. transparent queue/depth?
7. camera near/far?

Объект дрожит

1. два владельца Transform?
2. force зависит от render FPS?
3. fixed pose не интерполируется?
4. camera читает pre-physics?
5. animation root motion спорит с controller?
6. collider/pivot offset?

NPC не идёт

1. Utility выбрала target?
2. manual command блокирует?
3. State reason?
4. start/target project on NavMesh?
5. path не обрезан?
6. door links/room route?
7. controller capsule проходит физически?
8. fallback mesh не игнорирует obstacles?

После load crash

1. JSON parse/version;
2. old scene destroyed слишком рано;
3. Bullet action/constraint остался;
4. async completion пришёл из старого generation;
5. Entity reference не remap;
6. material/model pointer уничтожен;
7. static render/nav cache не invalidated.

Значение зависит от FPS

Искать:

- константу «на кадр» без `dt`;
- `lerp(..., 0.3)`;
- силы вне fixed tick;
- random calls разное число раз на frame;
- clamp после интегрирования;
- unordered iteration, влияющий на решение.

42. Практикум: безопасное добавление engine-функции

42.1. Шаблон design note

```
Проблема:
API/данные:
Источник истины:
Владелец:
Поток:
Фаза кадра:
Единицы:
Инварианты:
Сериализация:
Диагностика:
Автотест:
Manual test:
Rollback:
```

42.2. Пример: Heat как engine subsystem

Не начинать с fire particle. Сначала:

1. `HeatComponent` с Kelvin/J/K;
2. `HeatTransferSystem` с pairwise order-independent transfers;
3. fixed/scaled time contract;
4. conservation test;
5. debug heat overlay;
6. serialization;
7. только затем fire gameplay и visual particles.

42.3. Definition of Done системы

- API минимален;
- ownership ясен;
- units названы;
- phase зарегистрирована;
- test падает при намеренной ошибке;
- debug UI объясняет состояние;
- save/load round trip;
- нет D3D/Bullet warnings;
- performance budget известен;
- known limitation записано.

43. Карта чтения кода

Маршрут «запуск и кадр»

```
src/Core/main.cpp
→ src/Core/SystemClass.cpp
→ src/Core/ApplicationClass.cpp::Initialize/Frame/Shutdown
→ src/Scene/SceneManager.cpp::UpdateAll
→ src/Render/Renderer.cpp::RenderFrame
```

Маршрут «сущность»

```
src/ECS/Components/ECSTypes.h
→ ComponentManager.h
→ ECSRegistry.h
→ Components.h
→ SceneManager::SpawnObject/RemoveObject/SerializeObject
```

Маршрут «физика персонажа»

```
CharacterMovementSystem
→ CharacterFacingSystem
→ CharacterControllerSystem
→ PhysicsWorld
→ PhysicsSystem post-step sync
```

Маршрут «NPC»

```
NPCComponents.h
→ NpcUtilitySystem.h
→ NpcCommandSystem.h
→ StateSystem.h
→ NpcNavMeshSystem.cpp
→ CharacterControllerSystem
```

Маршрут «PBR frame»

```
Renderer::RenderFrame
→ SceneManager::RenderAllECS
→ RenderMesh.h
→ MaterialClass
→ PBRShaderClass
→ pbr.vs / pbr.ps
```

Маршрут «корабельная авария»

```
WeaponSystem/DamageSystem
→ HullPanel/Breach
→ FloodingSystem/RoomEnvironmentSystem
→ PhysicsSystem/SubmarineSystem
→ ShipPowerGrid/OxygenGenerator
→ ShipAlert/NpcUtility/RepairSystem
```

Часть VIII. Работа с кодом и практикум по NPC

44. Как изучать собственный движок, а не просто читать файлы

Эта часть превращает справочник в рабочий учебник. Её задача — научить не запоминать тысячи строк, а восстанавливать путь данных и находить правильное место изменения.

44.1. Пять вопросов перед чтением любой системы

Перед тем как разбирать метод, ответь на пять вопросов:

1. **Какие данные он читает?** Например, `NpcUtilitySystem` читает позиции, повреждения, пробоины и настройки мозга.
2. **Какие данные он меняет?** Например, он записывает `repairScore`, `repairTarget`, `state` и `status`.
3. **Кто вызывает метод?** Система сама по себе не работает; важна её позиция в `ApplicationClass::Frame` или `SceneManager::UpdateAll`.
4. **Кто использует результат?** `StateSystem` читает решение utility, а `CharacterControllerSystem` читает уже готовое движение.
5. **Что является источником истины?** Для движения это `CharacterMovement` до физического шага и Bullet-transform после него; для враждебности — `NPCComponent::isHostile`; для назначения ремонта — claim на цели.

Если на эти вопросы нет ответа, изменение пока рано писать. Большинство трудноуловимых багов движка возникает не внутри формулы, а между двумя владельцами одного состояния.

44.2. Чтение по вертикальному пути

Не нужно читать каталог подряд. Выбирай одно наблюдаемое поведение и иди от входа до результата.

Пример для движения NPC:

```
NPCComponent / NpcBrainComponent
→ NpcUtilitySystem или NpcCommandSystem
→ StateSystem
→ CharacterMovement
→ CharacterFacingSystem
→ CharacterControllerSystem
→ Bullet fixed step
→ PhysicsSystem sync/interpolation
→ TransformComponent
→ AnimationController
→ Renderer
```

Пример для ремонта:

```
DamageSystem создаёт повреждение
→ RepairableComponent описывает работу
→ CrewTaskCoordinator закрепляет цель
```

```
→ NpcUtilitySystem считает полезность
→ StateSystem ведёт NPC к WorkZone
→ RepairSystem применяет прогресс
→ WorldEventLog записывает результат
→ HUD/debug объясняет состояние
```

Такой маршрут даёт контекст. Отдельный метод без вызывающей и принимающей стороны почти всегда вводит в заблуждение.

44.3. Метод изучения за 30 минут

Для каждой новой подсистемы используй один и тот же цикл:

1. Найди типы данных через `rg -n "ComponentName|SystemName" .`.
2. Найди место вызова `Update`, `Spawn`, `Destroy` или `Render`.
3. Выпиши producer и consumer каждого изменяемого поля.
4. Включи debug UI и сопоставь поля с наблюдаемым поведением.
5. Измени один параметр через инспектор, не меняя код.
6. Предскажи результат до запуска.
7. Запусти и сравни предсказание с фактом.
8. Верни параметр и запиши вывод в `DEV_NOTES_OFFLINE.md`.

Этот способ полезнее пассивного чтения: ты строишь причинную модель движка и сразу проверяешь её.

44.4. Как отличать реализованное от проектируемого

В документации встречаются три типа утверждений:

- **факт текущего кода** — можно найти тип и место вызова;
- **ограничение** — код работает, но имеет известную границу;
- **целевая архитектура** — идея для следующего этапа, которой ещё нет в runtime.

Пример: `CrewTaskClaimComponent` — реализованный reservation ремонта. Общий task board пожаров, лечения и снабжения — целевая архитектура. Нельзя писать новую систему, исходя из того, что целевая часть уже существует.

44.5. Минимальная design note перед новой механикой

```
Наблюдаемое поведение:
Компоненты данных:
Система, принимающая решение:
Система, исполняющая решение:
Источник истины:
Место в кадре:
Время: real / simulation / fixed:
Что происходит при pause:
Как обрабатывается удаление target:
Что сохраняется в JSON:
Что видно в debug UI:
Минимальная тестовая сцена:
Критерий готовности:
Что намеренно не входит в задачу:
```

Если этот список заполнен, код обычно получается меньше и понятнее.

45. Анатомия персонажа в DX Framework

Игрок, дружественный NPC и враг используют общую физическую основу. Различие задаётся набором компонентов и тем, какая система пишет намерение движения.

45.1. Слои сущности персонажа

Слой	Компоненты	Ответственность
Идентичность	`NameComponent`, `NPCComponent`, `PlayerGameTag`	кто это и какую роль играет
Представление	`MeshRendererComponent`, `AnimationComponent`	модель, материал, скелет и клипы
Положение	`TransformComponent`	отображаемая позиция, поворот и масштаб
Намерение	`CharacterMovement`	куда и с какой скоростью персонаж хочет двигаться
Физика	`CharacterController`, `CharacterPhysicsTuningComponent`	Bullet capsule, земля, ступени, прыжок
NPC-настройки	`NPCMovementComponent`	скорость, бег, дистанции прибытия и дверей
AI runtime	`NpcBrainComponent`	состояние, команда, цели, маршрут и debug
Экипаж	`CrewMemberComponent`	роль, смена, ремонтный навык и assignment

Mesh не определяет игровую принадлежность. Одна и та же модель может быть дружественной или враждебной. Источником истины служат gameplay-компоненты.

45.2. `NPCComponent`

`NPCComponent` уже содержит:

- `name` и `description` для инспектора и взаимодействия;
- `health` и `maxHealth` как базовое здоровье NPC;
- `isHostile` как текущий признак враждебности.

Флаг `isHostile` уже:

- редактируется чекбоксом `Hostile` в `EditorUI`;
- сохраняется как `npc_hostile`;
- восстанавливается при загрузке сцены;
- передаётся в `InteractionTargetInfo::isHostileNpc`;
- меняет цвет панели NPC в `GameHud`;
- исключает NPC из автоматического распределения ремонтных задач.

Поэтому отдельный `EnemyComponent` для первого врага не нужен. Он создал бы два источника истины: `isHostile == true`, но `EnemyComponent` отсутствует, или наоборот. Отдельный компонент станет оправдан только для самостоятельного набора боевых данных, например `HostileAiComponent`, а не как второй флаг принадлежности.

45.3. `NpcBrainComponent`

Brain хранит изменяемое состояние, но не обязан содержать всю будущую логику. Его важные группы полей:

- текущие `NpcState` и `NpcCommand`;
- цель ремонта и комнатный маршрут;
- активная дверь и состояние прохождения портала;
- utility-настройки и диагностические score;
- таймеры перестроения пути и detector застревания;
- состояние wander;
- debug-команды и объясняющие строки.

Поле `status` предназначено для ответа «что сейчас происходит», а `blockedReason` — «почему система не может продолжить». Они не должны быть условием gameplay-логики.

45.4. `CharacterMovement` как граница между AI и физикой

AI не должен напрямую двигать `TransformComponent` или вызывать Bullet `warp` для обычной ходьбы. Он записывает намерение:

```
walkDirection
currentSpeed
isMoving
wantsRun / isRunning
wantJump
locomotionState
```

После этого общие системы выполняют намерение. Благодаря такой границе:

- игрок и NPC используют одну collision-модель;
- анимация получает одинаковые состояния locomotion;
- враг не проходит сквозь препятствия особым способом;
- AI можно тестировать отдельно от Bullet.

45.5. Создание runtime-контроллера

Добавление `CharacterController` в ECS ещё не создаёт Bullet-объекты. `CharacterSpawnSystem::Spawn`:

1. проверяет `CharacterController` и `TransformComponent`;
2. уничтожает старый runtime controller при повторном spawn;
3. создаёт capsule shape;
4. создаёт ghost object и связывает его с Entity через `userIndex`;
5. создаёт `btKinematicCharacterController`;
6. применяет gravity, slope, jump и penetration settings;
7. регистрирует collision object и action в Bullet world.

Парный `Destroy` обязан удалить action и collision object из мира до освобождения памяти. Простое удаление ECS-компонента без teardown опасно.

45.6. Что делает `CharacterControllerSystem::Update`

Метод обслуживает все сущности с `CharacterController`:

- останавливает и временно снимает Bullet-объекты неактивной сущности;
- лениво добавляет безопасный `CharacterPhysicsTuningComponent` старым сценам;
- вычисляет опору геометрическим sweep, а не доверяет только `onGround()`;
- применяет auto-step;
- объединяет locomotion и внешнюю скорость декомпрессии;
- запускает прыжок;
- переводит неподвижного стоящего персонажа в стабильный physics idle;
- очищает runtime-map для удалённых сущностей.

Публичные диагностические методы `IsAutomaticStepActive`, `HasGroundSupport`, `IsIdlePhysicsLocked` позволяют UI показать внутреннее состояние без дублирования расчётов. `ForceReconfigure` сбрасывает runtime cache после изменения настроек в инспекторе.

45.7. Поворот и анимация

`CharacterFacingSystem` читает горизонтальное направление движения и плавно меняет visual rotation. Он не принимает решение, куда идти.

`AnimationController` читает locomotion-результат и выбирает Idle, Walk, Run или другой доступный клип. AI не должен напрямую запускать Walk каждый кадр: это связывает решение с конкретным ассетом и ломает fallback для моделей с другим набором клипов.

46. Дружественный NPC в текущей архитектуре

46.1. Состав дружественного NPC

Минимальная дружественная сущность содержит:

```
ActiveComponent
TransformComponent
MeshRendererComponent
AnimationComponent
NPCComponent { isHostile = false }
NpcBrainComponent
NPCMovementComponent
CharacterMovement
CharacterController
CharacterPhysicsTuningComponent
```

`CrewMemberComponent` может быть добавлен вручную или будет создан координатором с defaults. Для предсказуемой сцены лучше авторить его явно и сохранять роль, `onDuty` и `repairSkill`.

46.2. `NpcCommandSystem`

Это единая точка выдачи ручного приказа. Команды не исполняют движение сами: они валидируют/записывают `NpcCommand`, увеличивают revision и сбрасывают необходимые runtime-поля.

Назначение методов:

Метод	Смысл	Время жизни команды
`Stop`	запретить автоматике перехватить NPC	пока не будет Resume/другая команда
`Wander`	постоянно выбирать точки блуждания	постоянная
`MoveToPosition`	прийти в мировую точку	снимается по прибытии
`MoveToRoom`	прийти внутрь комнаты	снимается после безопасного прохода портала
`FollowEntity`	следовать за движущейся Entity	постоянная
`RepairEntity`	прийти и починить конкретную цель	снимается после завершения/невалидности
`ResumeAutomatic`	очистить ручной приоритет	немедленно возвращает authority автоматике
`ForceRepath`	инвалидировать маршрут	не меняет смысл текущего приказа

Главный инвариант: `command.type != None` означает, что automatic utility не имеет права менять задачу NPC. Debug UI не должен обходить этот контракт прямой записью `brain.state`.

46.3. `NpcUtilitySystem`

Utility проходит по NPC с brain, movement и transform, оценивает ремонтируемые пробоины и оборудование. При оценке учитываются валидность, расстояние, тяжесть повреждения и чужой claim.

Система всегда может вычислять score для диагностики, но при активной ручной команде не меняет решение. Это позволяет видеть, что автоматика выбрала бы, не отбирая управление у разработчика.

46.4. `CrewTaskCoordinator`

Координатор решает межагентную проблему: несколько NPC не должны чинить одну цель.

Последовательность `Update`:

1. очистить невалидные claims;
2. собрать и отсортировать ремонтируемые цели;
3. собрать активных NPC и отсортировать по Entity для воспроизводимости;
4. исключить hostile, off-duty и NPC с ручной командой;
5. сохранить действующее валидное назначение;
6. оценить свободные цели;
7. добавить `CrewTaskClaimComponent` цели;
8. записать assignment в `CrewMemberComponent` и brain;
9. обновить debug snapshot.

`IsRepairTargetValid` — общий контракт валидности цели. `IsClaimedByOther` используется utility, чтобы не выбирать чужую работу. `GetSnapshot` отдаёт UI количество доступного экипажа, занятых работников и открытых задач.

Роль влияет на score: инженер получает преимущество в ремонте, Operations — дополнительную близость к генераторам и кислородному оборудованию, Security/Medic имеют меньшую склонность к обычному ремонту. `repairSkill` влияет не только на выбор, но и на скорость исполнения ремонта.

46.5. `StateSystem`

`StateSystem` — исполнитель. Он не должен решать стратегическую полезность задачи.

Верхний `Update` проверяет обязательные компоненты и направляет NPC либо в manual-command handler, либо в automatic state. Для movement-задач используется общий `NavigateTo`:

```
определить текущую и целевую комнаты
→ построить маршрут по дверям
→ выбрать approach/exit точки портала
→ запросить открытие двери
→ пройти портал полностью
→ запросить локальную NavMesh waypoint
→ записать CharacterMovement
→ следить за фактическим прогрессом
```

`FollowEntity` уже является полезным прототипом преследования: target transform читается каждый кадр, target room обновляется, а NPC держит дистанцию 1.5 м. Но это ручная бесконечная команда, а не hostile perception/combat AI.

`UpdateRepairTask` унифицирует пробоину и оборудование через `RepairableComponent`. Для оборудования он ищет `WorkZoneComponent`, выбирает достижимую рабочую точку, подходит, учитывает `repairSkill` и вызывает общий ремонтный путь.

46.6. Двери и NavMesh

Комнатный маршрут и локальный NavMesh решают разные задачи:

- граф комнат говорит, через какие двери идти;
- NavMesh говорит, как пройти внутри комнаты вокруг статических препятствий;
- local dynamic avoidance даёт короткий объезд динамического AABB;
- Bullet controller окончательно решает физическую проходимость.

Успешный путь NavMesh не гарантирует, что capsule пройдёт: размеры агента и физического контроллера должны совпадать. При любом застревании сначала смотри `status`, `blockedReason`, door snapshot, current room, waypoint и ground support.

46.7. Что значит «почти идеальный дружественный NPC»

Для текущего масштаба проекта это не NPC с сотней реплик. Это персонаж, который:

- стабильно стоит, ходит, бегает и проходит допустимые пороги;
- не дрожит и не проваливается;
- проходит двери и восстанавливается после временной блокировки;
- выполняет manual-команды;
- самостоятельно получает уникальную ремонтную работу;
- корректно освобождает claim при отмене, off-duty, уничтожении цели или смене команды;
- имеет понятную роль и параметры в инспекторе;
- сохраняет authoring-данные и безопасно пересоздаёт runtime-данные;
- в каждый момент объясняет состояние через UI;
- не зависит от присутствия игрока или камеры.

47. Проектирование врага на существующем NPC pipeline

47.1. Что уже готово

Для врага уже можно переиспользовать:

- `NPCComponent::isHostile` как принадлежность;
- `health/maxHealth` как базовое здоровье;
- `NpcBrainComponent` как общий navigation/debug runtime;
- `NPCMovementComponent` как параметры движения;
- `CharacterMovement`, controller, facing и animation;
- `FollowEntity`/`NavigateTo` как доказанный путь преследования;
- `PlayerGameTag` как способ найти игрока;
- `DamageSystem` как общий путь применения урона;
- scene save для базовых NPC-полей.

Врагу не нужна отдельная физика или особая телепортация. Новая часть — принятие боевого решения и безопасная атака.

47.2. Почему одного `isHostile` недостаточно

`isHostile` отвечает только «враг ли это». Боевые runtime-данные имеют другой смысл и другое время жизни. Их лучше хранить в отдельном компоненте, например концептуальном `HostileAiComponent`:

```
// Учебный эскиз контракта, не готовая вставка.
struct HostileAiComponent
{
    Entity target = INVALID_ENTITY;
    float detectionRange = 12.0f;
    float loseTargetRange = 18.0f;
    float attackRange = 1.6f;
    float attackCooldownSeconds = 1.0f;
    float cooldownRemaining = 0.0f;
    // Состояние perception/combat добавь осознанно.
};
```

Этот компонент не дублирует `NPCComponent::isHostile`. Маркер отвечает за отношение, hostile AI — за настройки и runtime боя.

47.3. Минимальный автомат состояний

Первый враг должен иметь только наблюдаемые состояния:

```
Idle
→ обнаружил игрока
Chase
→ вошёл в attack range
Attack
→ игрок отошёл
Chase
→ игрок потерян или невалиден
Idle
```

Дополнительное `Blocked` можно переиспользовать из общего navigation runtime либо показать как причину, не превращая первую версию в огромный state machine.

47.4. Perception первой версии

Первый шаг — дистанция и валидность Entity. Затем добавляется line of sight через raycast.

Порядок проверки:

1. существует ли игрок;
2. активен ли он и имеет ли Transform/health;
3. находится ли он в detection/lose range;
4. не закрывает ли raycast стену или дверь;
5. нужно ли удержать уже найденную цель по hysteresis.

Два радиуса обязательны:

```
detectionRange < loseTargetRange
```

Иначе враг будет переключать Chase/Idle каждый кадр на границе одного радиуса.

47.5. Преследование

Не копируй навигацию из `StateSystem`. Вынеси или переиспользуй существующий путь так, чтобы hostile decision только сообщал:

```
target entity
desired stop distance
desired movement state
```

Для первого прототипа допустимо доказать маршрут существующей ручной командой `FollowEntity`. Но автоматическая hostile-система не должна каждый кадр заново выдавать command и увеличивать revision: это постоянно инвалидирует маршрут. Ей нужен стабильный combat state или отдельный вызов общего navigation helper.

47.6. Атака

Первая атака должна быть простой melee-атакой с cooldown. Условия применения урона проверяются в момент удара, а не только при входе в Attack:

- target всё ещё валиден;
- target всё ещё в attack range;
- при необходимости line of sight не потерян;
- cooldown завершён;
- урон проходит через общий `DamageSystem`/damage event, а не прямое изменение случайного поля HP.

Cooldown использует `simulationDeltaTime`. На паузе он не уменьшается. Один удар не должен зависеть от FPS и не должен наноситься каждый кадр анимации.

47.7. Порядок hostile-системы в кадре

Боевой decision producer должен работать до facing/controller/animation. Возможный контракт:

```
NpcUtilitySystem (friendly repair score)
→ CrewTaskCoordinator (friendly assignment)
→ Hostile decision/perception
→ StateSystem или общий navigation executor
→ CharacterFacingSystem
→ CharacterControllerSystem
→ AnimationController
→ Damage resolution / particles
```

Главное правило: в одном кадре только один слой имеет authority над `CharacterMovement` конкретного NPC. Для hostile NPC дружественный repair pipeline должен быть выключен — это уже частично обеспечивает `CrewTaskCoordinator`, но `NpcUtilitySystem` тоже нужно проверить на ранний выход для `isHostile`.

47.8. Смерть и потеря цели

Вражеский AI обязан безопасно переживать:

- удаление игрока;
- выход из Play Mode;
- загрузку новой сцены;
- деактивацию врага;
- смерть врага;
- смерть target;
- уничтожение двери/изменение маршрута.

Нельзя хранить raw pointer на компонент цели между кадрами. Храни `Entity`, а компонент получай заново после проверки `IsAlive` и наличия нужного типа.

47.9. Debug-контракт врага

Для выбранного врага показывай:

```
Hostile: yes/no
Combat state
Target Entity
Distance to target
Detection / lose / attack ranges
Line of sight
Cooldown remaining
Current room / target room
Nav status / waypoint
Movement intent
Blocked reason
Last damage result
```

Если враг «не работает», эта панель должна за минуту ответить: он не видит, не решил, не построил путь, физически застрял или не прошёл условие удара.

48. Практическое задание Первый полноценный дружественный NPC

Цель задания — довести одного члена экипажа до состояния, при котором его можно оставить в сцене без постоянного ручного контроля. Код выполняет разработчик; учебник задаёт архитектурные границы и проверяемый результат.

48.1. Сценарий

Создай NPC `Crew_Engineer_01`. Он стоит в жилой комнате, получает роль Engineer, самостоятельно выбирает одну из двух ремонтных целей, проходит в техническую комнату через дверь, чинит цель, освобождает assignment и возвращается в понятное Idle-состояние. Ручная команда временно перехватывает его и после Resume возвращает автоматике.

48.2. Этап 1. Authoring и сохранение

Настрой сущность через редактор:

- читаемые name/description;
- `isHostile = false`;
- корректный набор character/NPC-компонентов;
- роль Engineer, `onDuty = true`, осмысленный `repairSkill`;
- скорости walk/run и arrival distances;
- capsule и physics tuning под размеры модели;
- AnimationComponent с Idle/Walk/Run;
- ActiveComponent.

Сохрани сцену, перезапусти и проверь, что authoring-поля восстановились. Runtime pointer Bullet, текущий путь и временный claim не должны сериализоваться как адреса.

48.3. Этап 2. Физическая стабильность

Проверки:

1. 60 секунд Idle на ровном полу.
2. Проход порогов 0.05, 0.10, 0.20, 0.30 и 0.36 м.
3. Движение по склону до max slope и отказ на слишком крутом.
4. Остановка у цели без колебания вперёд-назад.
5. Исчезновение пола корректно снимает grounded/idle lock.
6. 30, 60 и 144 FPS не меняют заметно скорость и результат.

Запиши capsule bottom, ground Y, grounded, idle lock, auto-step и горизонтальную скорость.

48.4. Этап 3. Команды

Проверь все команды инспектора:

- Stop удерживает NPC;
- Wander выбирает достижимые точки;
- MoveToPosition завершается;
- MoveToRoom проходит портал до безопасной точки;
- Follow player держит дистанцию и не толкает capsule;
- Repair entity работает для пробоины и оборудования;
- Resume auto действительно снимает manual authority;
- Force repath не стирает смысл команды.

Для каждой ошибки сначала исправляй объясняющее состояние, затем поведение. Не оставляй silent failure.

48.5. Этап 4. Автономный ремонт и reservation

Размести две цели: пробоину и повреждённый генератор. Добавь второго простого crew NPC только для проверки arbitration.

Критерии:

- каждый NPC получает максимум одну работу;
- одна цель принадлежит максимум одному NPC;
- Engineer имеет предсказуемое преимущество при близких условиях;
- ручная команда освобождает прежний claim;
- off-duty освобождает claim;
- завершение ремонта увеличивает completed count один раз;
- исчезновение цели не оставляет вечный claim;
- hostile NPC не получает crew assignment.

48.6. Этап 5. Двери, препятствия и восстановление

Проверки:

- закрытая рабочая дверь открывается;
- NPC ждёт створки, а не толкает её;
- после прохода collider двери возвращается;
- временное препятствие вызывает repath/avoidance;
- обесточенная или непроходимая дверь даёт понятный `Blocked`;
- после исправления причины Force repath восстанавливает задачу.

48.7. Definition of Done дружественного NPC

- Debug и Release собираются.
- Нет дрожания в Idle и постоянного толкания двери.
- Все ручные команды имеют предсказуемое завершение.
- Автоматический ремонт работает без камеры и игрока рядом.
- Два NPC не чинят одну цель.
- Save/load сохраняет authoring и не сохраняет опасные runtime pointers.
- В debug UI видно состояние, цель, assignment, путь и причину блокировки.
- Есть отдельная regression-сцена и записанные шаги воспроизведения.
- Известные ограничения перечислены, а не спрятаны.

49. Практическое задание Первый преследующий и атакующий враг

Цель — создать `Enemy_Hunter_01`, который автоматически обнаруживает игрока, преследует его через комнаты и двери, останавливается на дистанции атаки, наносит дискретный урон и безопасно теряет цель.

49.1. Ограничение первой версии

В задачу входят только:

- один игрок;
- один melee-враг;
- Idle, Chase и Attack;
- distance perception и затем line of sight;
- существующая navigation/physics/animation pipeline;
- cooldown и урон;
- debug UI и сохранение настроек.

Не входят patrol graph, ranged weapon, cover, squad tactics, loot, ragdoll, hearing, stealth meter и сложная анимационная синхронизация. Они добавляются после стабильного vertical slice.

49.2. Этап 1. Сущность и данные

Создай NPC через существующий `NPC AI Components`, поставь `Hostile`, настрой mesh, animation, controller и movement. Проверь, что после save/load `npc_hostile` остаётся true и HUD показывает hostile-оформление.

Спроектируй отдельные combat-поля. Перед кодом письменно реши:

- где хранится combat state;
- кто владеет target Entity;
- чем отличаются detection и lose range;
- где хранится attack range, damage и cooldown;
- какие поля authoring, а какие runtime;
- какие из них сохраняются;
- кто имеет authority над `CharacterMovement` hostile NPC.

49.3. Этап 2. Доказать преследование без нового AI

Через debug нажми `Follow player`. Проверь путь на ровном полу, через дверь и между комнатами. Это отделяет проблемы существующей навигации от ошибок будущего combat decision.

Если ручное Follow уже не работает, hostile-код пока не писать. Сначала исправляется общий pipeline.

49.4. Этап 3. Автомат Idle и Chase

Добавь систему принятия hostile-решения. Она обрабатывает только активных NPC с `isHostile == true` и нужными combat-данными.

Требования:

- игрок ищется через `PlayerGameTag`;
- target хранится как `Entity`, не pointer;
- detection использует горизонтальную или трёхмерную дистанцию по явно выбранному правилу;
- захваченная цель теряется только за `loseTargetRange`;
- состояние не переустанавливается каждый кадр без причины;
- изменение target/состояния инвалидирует путь один раз;
- при отсутствии игрока движение полностью останавливается;
- friendly utility не перезаписывает hostile movement.

49.5. Этап 4. Line of sight

Добавь raycast от осмысленной точки врага к осмысленной точке игрока. Ray не должен начинаться в полу или центре capsule, если там мешает собственный collider. Игнорируй entity врага и корректно интерпретируй первое попадание.

Проверки:

- видит игрока в пустой комнате;
- не видит через стену;
- закрытая дверь перекрывает видимость;
- открытая дверь разрешает видимость;
- краткая потеря LOS не вызывает дрожание состояния — при необходимости добавь grace timer;
- debug line показывает реальный луч.

49.6. Этап 5. Attack

Когда дистанция меньше attack range, враг останавливает locomotion и входит в Attack. Удар происходит только по cooldown и повторно проверяет target/range/LOS.

Критерии:

- один удар уменьшает здоровье ровно один раз;
- частота удара одинакова при 30/60/144 FPS;
- pause замораживает cooldown и атаку;
- враг не наносит урон через стену;
- игрок, вышедший из attack range, переводит врага обратно в Chase;
- удалённый/мёртвый игрок очищает target;
- damage идёт через общий механизм, а не обходной прямой доступ к UI-полю;
- friendly NPC не становится жертвой, пока это явно не предусмотрено.

49.7. Этап 6. Анимация и обратная связь

Idle/Walk/Run остаются под управлением общего animation controller. Для первой атаки допустим отдельный trigger/state, но урон нельзя случайно привязать к render frame.

Минимальная обратная связь:

- имя и красный hostile accent;
- combat state в debug;
- attack cooldown;
- короткое событие/лог успешного удара;
- видимое изменение HP игрока;
- понятная причина потери цели.

49.8. Матрица тестов врага

Ситуация	Ожидаемый результат

Ситуация	Ожидаемый результат
Игрок вне detection	Idle, target invalid
Игрок входит в detection	target захвачен, Chase
Игрок между detection и lose после захвата	Chase продолжается
Игрок за стеной	нет захвата или срабатывает grace по выбранному правилу
Игрок в другой связанной комнате	путь идёт через дверь
Игрок ближе attack range	движение остановлено, Attack
Игрок отступил	возврат в Chase
Игрок удалён	target очищен, Idle
Враг деактивирован	Bullet/action и AI не продолжают работу
Игра на pause	движение, cooldown и урон заморожены
Save/load	authoring сохранён, runtime target безопасно сброшен/проверен

49.9. Definition of Done врага

- Враг определяется существующим `isHostile`.
- Combat-данные не дублируют brain/movement без причины.
- Преследование переиспользует общий navigation pipeline.
- Есть hysteresis обнаружения и cooldown атаки.
- Урон применяется дискретно и FPS-независимо.
- Friendly crew pipeline не управляет врагом.
- Невалидная target Entity никогда не разыменовывается.
- Debug UI объясняет perception, decision, navigation и attack.
- Есть тестовая сцена с комнатой, стеной и дверью.
- Все сценарии матрицы проверены и записаны.

50. План одного рабочего дня и порядок сдачи

Если на практику есть один длинный вечер, не пытайся закончить обоих персонажей сразу.

Блок 1. 30 минут — baseline

- собрать Debug;
- сохранить копию тестовой сцены;
- проверить текущего NPC через Follow player и Repair entity;
- записать исходные баги отдельно от новых.

Блок 2. 90 минут — дружественный NPC

- полный authoring;
- роль Engineer;
- команды;
- одна автономная ремонтная задача;
- debug snapshot;
- save/load.

Если на этом этапе есть баг движения или двери, остановись и локализуй его. Враг использует тот же фундамент.

Блок 3. 60 минут — данные врага и Idle/Chase

- `isHostile`;
- отдельный combat runtime;
- target игрока;
- detection/lose hysteresis;
- автоматическое преследование;
- отсутствие конфликтов с crew utility.

Блок 4. 60 минут — Attack

- attack range;
- остановка;
- cooldown;
- общий damage path;
- pause/FPS проверки;
- debug-поля.

Блок 5. 30 минут — фиксация результата

Запиши:

```
Дата:
Сцена:
Изменённые файлы:
Что умеет Crew_Engineer_01:
Что умеет Enemy_Hunter_01:
Какие состояния проверены:
Какие значения ranges/cooldown использованы:
Что происходит при pause:
Что происходит при потере/удалении target:
Известные проблемы:
Следующий маленький шаг:
```

Для code review подготовь короткое видео или последовательность скриншотов и список шагов воспроизведения. Хороший результат — не «вроде бегает», а повторяемый сценарий, в котором каждое состояние можно объяснить данными.

Часть IX. Боевой NPC как учебный проект

Эта часть продолжает практикум ровно с текущего состояния проекта. Враг уже обнаруживает игрока по дистанции, переходит между `Chasing` и `Attacking`, поворачивает корпус к цели и запрашивает одноразовый клип `Punch`. Наблюдаемая проблема: ноги и корпус смотрят на игрока, но траектория удара уходит вбок. Цель следующих глав - не получить готового врага из одного большого патча, а последовательно построить и доказать собственную боевую архитектуру.

Главный принцип обучения: каждый этап имеет гипотезу, минимальную тестовую сцену, наблюдаемые данные и критерий сдачи. Если критерий не выполнен, следующий слой не добавляется. Так ошибка в анимации не маскируется NavMesh, а ошибка видимости не маскируется сложным utility score.

Статус реализации на 28 октября 2026 года. В этой части групповые роли и координация описаны как учебный проект и возможное развитие. В текущем игровом кадре NpcSquadCombatSystem::Update отключён в ApplicationClass.cpp. Проверяй работающий индивидуальный цикл через NpcCombatSystem, NpcCoverSystem, StateSystem и NpcRangedCombatSystem.

51. Текущее состояние врага и границы следующей версии

51.1. Фактический путь одного кадра

В текущем `ApplicationClass` боевой путь проходит в следующем порядке:

```
NpcUtilitySystem
-> CrewTaskCoordinator
-> NpcCombatSystem::Update
-> StateSystem::Update
-> CharacterFacingSystem::Update
-> выравнивание игрока по камере
-> CharacterControllerSystem::Update
-> PlayerPunchSystem::Update только для игрока
-> AnimationController::Update
-> DamageSystem::Update
```

Для понимания текущего бага особенно важны три факта. `StateSystem` меняет yaw NPC и отправляет `oneShotRequested = true`. `AnimationController` накладывает клип от `mixamorig:Spine`. А привязка кулака к лучу камеры реализована внутри `PlayerPunchSystem` и требует `PlayerComponent`; NPC через этот код не проходит. Поэтому фраза «удар привязан к прицелу» сейчас верна для игрока, но не для врага.

51.2. Что уже существует

Слой	Реализовано сейчас	Следующее доказательство
Данные	`NPCCombatComponent`: target, attackRange, seeRange, cooldown и таймеры	Отделить authoring от runtime и сохранить настройки

Слой	Реализовано сейчас	Следующее доказательство
Решение	`NpcCombatSystem` выбирает `Chasing` или `Attacking` по дистанции	Добавить LOS, память и устойчивые переходы
Исполнение	`StateSystem` строит путь, останавливает NPC и поворачивает его	Разделить facing корпуса и aim атаки
Анимация	`oneShotState = Punch` воспроизводится поверх locomotion	Доказать правильный forward клипа и контактную фазу
Навигация	Detour waypoints, двери и dynamic avoidance уже используются	Переиспользовать их для cover и flank destinations
Диагностика	Inspector показывает ranges, target и attack timer	Добавить LOS, yaw error, aim error, cover score и reason

51.3. Подтверждённые ограничения текущего кода

- `NpcCombatSystem` пока воспринимает игрока только по расстоянию; стены и направление взгляда не учитываются.
- В ветке, где цели ещё нет и игрок вне `seeRange`, состояние устанавливается в `Chasing`; это нужно проверить как логическую ошибку до усложнения автомата.
- `NPCCombatComponent` редактируется в инспекторе, но в текущем `SceneManager.cpp` не найдено сохранение его полей. После загрузки значения могут вернуться к defaults.
- В `StateSystem` удар пока описан как запрос анимации. Общая точка контактного урона NPC ещё должна быть спроектирована.
- Дистанция считается в 3D, а поворот - в плоскости XZ. Нужно явно решить, влияет ли разница высоты на detection и melee range.
- Константа разворота модели на PI повторяется в нескольких местах. Пока соглашение не централизовано, легко получить дополнительный поворот на 90 или 180 градусов.

51.4. Учебный контракт

Для каждого этапа создавай короткую design note:

```
Наблюдаемое поведение:
Гипотеза:
Producer данных:
Consumer данных:
Источник истины:
Минимальная сцена:
Что рисуется в debug:
Тест до изменения:
Тест после изменения:
Definition of Done:
Что намеренно не входит в этап:
```

При обращении за помощью присылай эту заметку, небольшой фрагмент кода и один конкретный наблюдаемый симптом. Тогда помощь останется объясняющей, а решение - твоим.

52. Практикум Удар направлен вбок

52.1. Теория трёх направлений

В одном кадре существуют минимум три независимых направления. `bodyForward` задаётся world rotation сущности. `clipForward` зашит в локальной системе скелета и в ключах таза, позвоночника и плеч. `attackDirection` идёт от точки атаки к выбранной цели. Исправление считается верным только тогда, когда эти направления согласованы осознанно.

```
toTarget = targetPosition - attackerPosition
horizontal = normalize(float3(toTarget.x, 0, toTarget.z))
desiredYaw = atan2(horizontal.x, horizontal.z) + modelForwardOffset
yawError = wrapPi(desiredYaw - currentYaw)
canStartAttack = abs(yawError) <= facingTolerance
```

Эта формула поворачивает корпус, но не гарантирует направление кисти. Верхний клип может вращать `Spine`, анимация может быть экспортирована с другой осью forward, а runtime IK может вообще не запускаться для NPC.

52.2. Четыре вероятные причины

7. Клип `Kevin_Punch.1` сам направлен боком относительно локального forward модели.
8. Клип корректен, но при смешивании от `mixamorig:Spine` его локальный поворот применяется поверх уже развёрнутого корпуса не в ожидаемом пространстве.
9. Корпус NPC смотрит правильно, но система aim и IK существует только для игрока, поэтому кисть NPC не получает цель.
10. В разных местах используется разный model forward offset: один слой добавляет PI, а другой считает, что модель смотрит по +Z без поправки.

Не выбирай причину по ощущению. Следующие проверки должны исключить три неверных варианта.

52.3. Минимальная диагностическая сцена

- Ровный пол без стен, один игрок, один `Enemy_Hunter_01`, одна камера сверху или сбоку.
- Отключи перемещение игрока и автоматическое изменение yaw камеры во время серии.
- Поставь врага в начало координат с yaw = 0. Игрока последовательно ставь в +Z, +X, -Z и -X.
- Используй один и тот же вариант удара и один и тот же кадр контакта.
- Сохраняй для каждого направления: target yaw, current yaw, yaw error, world position плеча, кисти и цели.

Позиция цели	Ожидаемое	Если кулак всё равно уходит вправо
+Z	корпус и кулак по +Z	постоянная локальная ошибка клипа или IK отсутствует
+X	корпус и кулак по +X	если ошибка повернулась вместе с NPC, проблема в model space

Позиция цели	Ожидаемое	Если кулак всё равно уходит вправо
-Z	корпус и кулак по -Z	если ошибка осталась в world space, ищи неверную world transform
-X	корпус и кулак по -X	сравни знак atan2 и повторное добавление offset

52.4. Сделай ошибку видимой

Добавь временную debug-визуализацию, а не новый gameplay-код:

- зелёная линия - `bodyForward` от позиции NPC;
- жёлтая линия - горизонтальный `toTarget`;
- синяя линия - направление от плеча к текущей кисти;
- красная линия - от кисти к желаемой contact point;
- текст: `currentYaw`, `desiredYaw`, `yawErrorDeg`, `oneShotTimeRemaining`, активный clip index.

Если зелёная и жёлтая линии совпадают, поворот ног уже исправен и трогать `TransformComponent` нельзя. Если синяя линия стабильно отклонена на одинаковый угол, проблема находится после facing: в ассете, смешивании или IK.

52.5. Развилка ассет или runtime

Сначала открой клип в Animation Studio без AI и проверь его на yaw 0. Затем поверни всю сущность на 90 градусов и повтори.

Результат проверки	Диагноз	Учебное решение
Клип уже в Studio бьёт боком	ошибка ориентации Action или ключей верхнего тела	исправить Action в Blender или локальную pose correction; не компенсировать AI углом
Studio верна, NPC неверен	ошибка runtime binding, layer root или отсутствующий aim	проверить binding `Punch`, `upperBodyRoot`, clip index и generic aim path
Только игрок верен	player-only camera aim и IK	вынести общую часть melee aim из `PlayerPunchSystem`
Ошибка меняется при повороте мира	смешаны local/model/world spaces	подписать пространство каждого вектора и добавить явные преобразования

Минимальный ассетный фикс допустим, если Action объективно экспортирован боком. Архитектурный фикс нужен, если враг должен направлять один и тот же удар к разным точкам, высотам и движущимся целям.

52.6. Проектирование общего melee aim

Не копируй весь `PlayerPunchSystem` внутрь NPC. Выдели нейтральный контракт, который не знает о клавиатуре, камере или `PlayerComponent`. Один из вариантов данных для обсуждения:

```
struct MeleeAimRuntime
{
    Entity target = INVALID_ENTITY;
    XMFLOAT3 aimOrigin{};
    XMFLOAT3 aimDirection{0, 0, 1};
    XMFLOAT3 contactPoint{};
    float contactPhase = 0.5f;
    float reach = 1.5f;
    bool active = false;
};
```

Это не готовое требование к имени компонента. Твоё задание - письменно ответить:

11. Кто создаёт aim для игрока и кто создаёт aim для NPC?
12. Кто переводит world contact point в model space перед IK?
13. Кто знает, какой рукой выполняется текущий вариант удара?
14. Кто сбрасывает runtime после окончания one-shot?
15. Какие поля сохраняются в сцену, а какие никогда не сериализуются?

Хорошая граница: player input и NPC decision формируют одинаковое намерение; общая melee-система вычисляет контакт, применяет IK и в contact window создаёт `DamageEvent`. Тогда прицел игрока и target врага различаются только producer-ом.

52.7. Контактная фаза и урон

Не связывай урон с моментом запуска клипа или с конкретным render frame. В текущем игроке уже есть идея `contactPhase`. Для NPC используй нормализованную фазу one-shot и флаг «урон уже применён в этом ударе».

```
phase = elapsed / duration
enteredContactWindow = previousPhase < contactPhase && phase >= contactPhase
if enteredContactWindow and not hitApplied:
    revalidate target, range, line of sight and fist contact
    emit DamageEvent once
    hitApplied = true
```

При смене FPS условие пересечения фазы остаётся устойчивым, даже если точный момент между кадрами был пропущен.

52.8. Definition of Done исправления удара

- Враг поворачивается к игроку с четырёх сторон и не начинает клип раньше допуска по yaw.
- На contact phase кулак находится в допустимом радиусе от целевой точки, а debug показывает ошибку в метрах.
- Один one-shot создаёт максимум один `DamageEvent`.
- При выходе игрока из reach во время замаха урон не проходит.
- Стена между плечом и contact point блокирует урон.

- Результат одинаков по смыслу при 30, 60 и 144 FPS.
- Исправление не ломает удар игрока и first-person view.
- В коде нет случайного `+90°` или `-90°` без имени и объяснения соглашения forward.

53. Восприятие и память цели

53.1. От расстояния к восприятию

Сейчас `seeRange` означает сферу знания: внутри неё враг знает игрока даже через стену. Настоящее восприятие лучше разделить на широкую фазу кандидатов и точную проверку. Сначала distance/FOV быстро отбрасывают лишних, затем raycast доказывает line of sight.

```
candidate -> alive and active
-> within detection range
-> inside horizontal field of view
-> eye-to-chest ray is not blocked
-> visible target observation
```

Для первого варианта достаточно одного игрока, но данные проектируй так, чтобы система могла сравнить несколько целей позднее.

53.2. Память без телепатии

При потере LOS сохраняй наблюдение, а не live transform невидимой цели:

```
lastSeenPosition
lastSeenVelocity optional
lastSeenTime
hasLineOfSight
visibilityGraceRemaining
memoryRemaining
```

Пока цель видима, эти поля обновляются. После потери LOS враг идёт к `lastSeenPosition`, но не читает текущую позицию игрока. Когда memory истекает, target очищается или начинается ограниченный поиск.

53.3. Самостоятельное задание

16. Добавь eye point врага и aim point игрока как вычисляемые world positions.
17. Сделай raycast с игнорированием собственного capsule врага.
18. Раздели `hasLineOfSight` и `targetEntity`: цель может быть известна, но временно невидима.
19. Добавь grace timer 0.15-0.35 секунды, чтобы край двери не дёргал состояние каждый кадр.
20. Покажи луч, первое попадание, last seen marker и возраст наблюдения в debug UI.
21. Исправь ветку без цели: вне detection враг должен оставаться в Idle/Patrol, а не самопроизвольно переходить в `Chasing`.

53.4. Проверки

- Пустая комната: цель видна.
- Стена: цель не видна, но последнее наблюдение ещё помнится.
- Закрытая дверь блокирует LOS; открытая разрешает.
- Краткое пересечение дверной рамой не сбрасывает target мгновенно.
- После истечения memory враг перестаёт знать текущую позицию игрока.
- Save/load не восстанавливает старую runtime-цель как валидный указатель или случайную Entity.

54. Преследование последней известной позиции

54.1. Состояния поиска

После стабильного LOS раздели `Chasing` на смысловые фазы. Имена могут отличаться, но данные должны объяснять поведение.

```
Idle or Patrol
-> Alerted
-> ChaseVisibleTarget
-> MoveToLastSeen
-> SearchArea
-> Return or Idle
```

`StateSystem` по-прежнему должен исполнять цель движения через существующий `NavigateTo`; perception не пишет `CharacterMovement` напрямую.

54.2. Поиск без сложной системы

Первая версия SearchArea может состоять из 3-5 достижимых точек вокруг last seen position. Кандидаты проецируются на NavMesh, отбрасываются при слишком коротком пути или отсутствии связности и посещаются по очереди. Это уже создаёт понятное поведение без behavior tree.

22. Прийти к last seen position.
23. Повернуться в последнее известное направление движения игрока.
24. Выбрать несколько NavMesh-кандидатов в радиусе поиска.
25. Остановиться на короткую scan pause в каждой точке.
26. Немедленно вернуться в ChaseVisibleTarget при восстановлении LOS.
27. После таймаута очистить цель и вернуться к исходному поведению.

54.3. Навигационные инварианты

- Новая perception-цель инвалидирует путь один раз, а не каждый кадр.
- Асинхронный результат проверяет revision/epoch перед применением.
- Если last seen point не проецируется на NavMesh, debug хранит конкретную причину.

- Дверной маршрут и local Detour waypoint остаются общей инфраструктурой.
- Ctuck recovery не меняет стратегическую цель: он только перестраивает способ достижения.

55. Укрытия как данные и запрос

55.1. Что такое укрытие для AI

Укрытие - не просто объект с тегом. Это позиция, из которой геометрия закрывает атакующего от угрозы, но рядом существует достижимая позиция для выхода и атаки. Поэтому система должна оценивать укрытие относительно конкретной угрозы.

Для первого варианта выбери один из двух путей:

- Авторские `CoverPointComponent` и `CoverSlot` на стенах и ящиках. Это проще отлаживать и хорошо подходит текущему редактору.
- Автоматические точки по границам NavMesh. Это масштабируется лучше, но требует устойчивого анализа полигонов и нормалей.

Для учебного проекта начни с авторских точек, а генерацию добавь после работающего поведения.

55.2. Минимальные данные cover slot

```
worldPosition
coverNormal
standOrCrouch
leftPeekPoint and rightPeekPoint
reservedBy Entity
reservationRevision or expiry
enabled
```

`coverNormal` должен смотреть от поверхности в свободное пространство. Для угрозы подходящее укрытие обычно находится с противоположной стороны нормали.

55.3. Оценка кандидатов

```
score =
    protectionWeight * protection
  - pathWeight * normalizedPathCost
  - distanceWeight * distanceToSlot
  + attackWeight * peekCanSeeThreat
  + spacingWeight * distanceFromAllies
  - occupancyPenalty
  - recentFailurePenalty
```

Сначала фильтруй невозможные точки: нет NavMesh path, slot занят другим NPC, точка находится слишком близко к угрозе, между угрозой и укрытием нет блокирующей геометрии. Только затем вычисляй score.

55.4. Практическое задание

28. Создай в редакторе 6-10 cover slots вокруг двух ящиков и одной стены.

29. Покажи normal, peek points и состояние reservation разными цветами.
30. Напиши read-only запрос `FindBestCover`, который возвращает кандидат и подробный score breakdown, но ещё не двигает NPC.
31. Сравни прямую дистанцию и реальную длину NavMesh path; в score используй выбранное правило осознанно.
32. Добавь reservation по образцу `CrewTaskClaimComponent`, чтобы два врага не выбирали одну точку.
33. Только после устойчивого выбора добавь состояния `MoveToCover` и `InCover`.

55.5. Definition of Done укрытия

- Враг выбирает точку, действительно закрытую от текущей угрозы.
- Путь к cover slot существует и завершается с предсказуемой дистанцией.
- Два врага не занимают один slot.
- Исчезновение или отключение укрытия освобождает reservation.
- В debug видны все кандидаты, причина фильтрации и итоговый score.

56. Атака из укрытия и смена позиции

56.1. Раздели melee и ranged ветки

Текущий `Enemy_Hunter_01` является melee-врагом. Укрытие с выглядыванием имеет полный смысл для ranged-врага, поэтому не смешивай обе модели в одном условии. Общая часть - perception, memory, navigation и reservation. Attack executor различается.

Свойство	Melee	Ranged
Желаемая дистанция	в пределах reach	preferred range с min и max
Укрытие	путь сближения и разрыв LOS	позиция защиты и peek
Контакт	fist/contact window	muzzle ray или projectile
Позиционирование	обход препятствия и attack slot	cover, strafe, reposition
Анимация	one-shot punch плюс aim IK	aim pose, recoil, reload

56.2. Peek как временное состояние

```
InCover
-> DecidePeekSide
-> MoveOrLeanToPeekPoint
-> ConfirmLineOfFire
-> AttackBurst
-> ReturnToCover
-> cooldown or reposition
```

Сначала перемещай весь capsule к отдельной peek point. Настоящий procedural lean добавляй позже: он требует согласования collider, камеры, оружия и анимации. Простая позиция легче проверяется и не создаёт визуальный обман, когда модель вышла, а collider остался за стеной.

56.3. Условия смены укрытия

- Текущий slot больше не защищает от новой позиции игрока.
- Peek не даёт line of fire несколько попыток подряд.
- Игрок подошёл ближе минимальной дистанции.
- Укрытие стало занято, разрушено или выключено.
- NPC получил подавление или роль flanker.
- Прошёл минимальный dwell time, чтобы AI не менял решение каждый кадр.

57. Обход и фланг

57.1. Фланг как задача выбора позиции

Фланг - не специальная анимация и не поворот вокруг игрока по окружности. Это выбор достижимой позиции, которая отличается углом атаки от текущей позиции союзника или последней линии огня.

```
candidateAngle = threatYaw + side * flankAngle
candidate = threatPosition + direction(candidateAngle) * flankRadius
project candidate to NavMesh
validate path, exposure and line of fire
score candidate
```

Начни с углов 45-90 градусов и двух сторон. Не генерируй десятки точек до появления debug-визуализации.

57.2. Проверки кандидата

34. Точка проецируется на тот же связный NavMesh-остров.
35. Путь не проходит через закрытую навсегда дверь или запрещённую область.
36. Кандидат не находится внутри минимального радиуса игрока.
37. Угол атаки действительно отличается от текущего, например минимум на 35 градусов.
38. Точка не зарезервирована другим flanker.
39. Ожидаемая линия огня не проходит через союзника.
40. Score содержит штраф за слишком длинный или открытый путь.

57.3. Самостоятельное задание

- Сначала вычисляй flank candidates read-only и рисуй их без движения.
- Покажи зелёным валидные, красным отклонённые и подпиши главную причину отказа.
- Добавь один state `MoveToFlank`, который использует обычный `NavigateTo`.

- При восстановлении хорошей позиции атаки разрешай прервать длинный фланг.
- Добавь cooldown на смену стороны и hysteresis score, иначе NPC будет метаться слева направо.

58. Несколько врагов и распределение ролей

58.1. Почему два хороших одиночных AI могут вести себя плохо

Без координации оба врага выберут лучший cover, один проход или одинаковый фланг. В движке уже есть полезный образец: `CrewTaskCoordinator` сортирует NPC и цели, создаёт claim и сохраняет воспроизводимость. Боевой координатор может использовать тот же принцип.

58.2. Минимальные роли

Роль	Обязанность	Ограничение первой версии
Pressure	держит видимость и атакует из безопасной позиции	не меняет cover без причины
Flanker	ищет другой угол и резервирует flank destination	только один flanker на маленькую группу
Chaser	сокращает дистанцию при потере ranged pressure	не обгоняет всех через один дверной проём
Searcher	проверяет last seen и search points	не получает live position невидимой цели

Не делай role отдельной жёсткой сущностью на всё время жизни. Это runtime assignment с revision и минимальным временем удержания. Роль меняется при смерти агента, исчезновении цели или существенном изменении ситуации, но не из-за малой разницы score.

58.3. Задание на координацию

41. Запусти двух врагов без coordinator и запиши конфликт в видео или лог.
42. Создай общий snapshot цели: visible agents, occupied cover slots и reserved flank sides.
43. Сортируй участников по Entity для воспроизводимости.
44. Сохраняй действующее валидное назначение до перераспределения.
45. Назначай максимум одну роль и одну destination на NPC.
46. Освобождай reservation при смерти, deactivation, scene reload и потере задачи.
47. Покажи role, assignment age и причину смены в debug UI.

Часть X. Атлас классов и маршруты данных

Этот атлас описывает проектные классы, компоненты и сообщения по их фактической роли в текущем коде. Он не перечисляет внутренние типы ImGui, Assimp, DirectX, Bullet, Recast/Detour и tinygltf, а также однотипные constant-buffer структуры: это чужие или локальные детали реализации. Семейства однотипных шейдеров и простых ECS-компонентов объединены, но все главные точки управления, данные gameplay и системы исполнения разобраны отдельно.

Удобная модель чтения: компонент хранит факт, система читает факты и создаёт изменение, координатор задаёт порядок систем, а renderer только превращает подготовленное состояние в изображение. Если класс одновременно выбирает цель, двигает Entity, включает анимацию и наносит урон, границы уже размыты и тестировать его будет трудно.

60. Как читать атлас и где проходят границы

60.1. Пять ролей типа

Роль	Главный вопрос	Примеры
Данные	Какой факт должен пережить кадр?	`TransformComponent`, `NPCCombatComponent`, `RoomComponent`
Решение	Что нужно сделать и почему?	`NpcUtilitySystem`, `NpcCombatSystem`, `CrewTaskCoordinator`
Исполнение	Как изменить мир безопасно?	`StateSystem`, `CharacterControllerSystem`, `DamageSystem`
Оркестрация	В каком порядке идут подсистемы?	`ApplicationClass`, `SceneManager`, `Renderer`
Представление	Как показать состояние человеку?	`GameHud`, `EditorUI`, debug renderers

60.2. Карточка класса

Для каждого типа фиксируй четыре вещи: владелец времени жизни; входные данные; выход или изменяемые компоненты; следующий потребитель результата. Эта карточка лучше списка методов: она показывает контракт между классами и место ошибки в цепочке.

```
Тип: CharacterFacingSystem
Вход: CharacterMovement.desiredFacingDirection
Выход: Transform.rotation.y
Следующий потребитель: CharacterController / Animation / Renderer
Инвариант: нулевое направление не создаёт NaN; yaw-конвенция едина
```

61. Ядро, цикл приложения и инфраструктура

Класс или тип	За что отвечает	Связи и границы
`ApplicationClass`	Создаёт сервисы, переключает Editor/Game, задаёт порядок update и render.	Владеет большинством систем; не должен становиться местом gameplay-правил.
`SystemClass`	Поднимает Win32-окно и главный цикл приложения.	Передаёт ввод и кадры в `ApplicationClass`; платформенная оболочка.
`TimeManager`	Предоставляет кадровое время и общие timing-данные.	Питает цикл; gameplay должен получать уже выбранный simulation delta.
`GameTimer`	Управляет simulation time, pause и скоростью игрового времени.	Используется перед gameplay-системами; cooldown NPC должен жить в этом времени.
`JobSystem`	Выполняет фоновые задачи и очередь возврата на main thread.	Используется загрузкой ресурсов, ремонтом и NPC; ECS/Direct3D нельзя бездумно менять из worker.
`FileSystem`	Сканирует assets и предоставляет проектный каталог файлов.	Связан с редактором, сценой и загрузчиками; хранит пути, а не GPU-объекты.
`GameSettings`	Собирает настройки окна, fog, physics debug и визуальных режимов.	Читается приложением и renderer; это authoring/config, не runtime state.
`PhysicsWorld`	Инкапсулирует создание и владение миром Bullet.	Базовый сервис для `PhysicsSystem`, raycast и controller; удаление тел должно быть согласовано с ECS.
`InputClass`	Собирает низкоуровневое состояние клавиш, мыши и окна.	Источник для `InputManager`; не должен знать игровые действия.
`InputManager`	Нормализует переходы кнопок и состояние устройств на кадр.	Подаёт данные в `GameplayInputSystem`, editor camera и drag/drop.
`CameraController`	Управляет свободной камерой редактора и её перемещением.	Читает input, меняет `CameraClass`; не равен first-person camera персонажа.
`WorldEventLog`	Хранит человекочитаемые события мира и диагностику.	Получает записи от gameplay; полезен для причин решения NPC и тестов.

61.1. Главная опасность ApplicationClass

Сейчас `ApplicationClass` является composition root и одновременно знает детали оружия, NPC, HUD, загрузки, редактора и подлодки. Это допустимо для прототипа, но при росте лучше выделить `SimulationPipeline`, `EditorPipeline` и `RenderPipeline`. Тогда порядок систем останется видимым, а главный класс перестанет быть единственным местом, где можно добавить механику.

62. ECS, сцена, редактор и сериализация

Класс или тип	За что отвечает	Связи и границы
`Entity` / `ECSTypes`	Идентификатор объекта и базовые ECS-типы.	Идентификатор не гарантирует существование; перед доступом нужна проверка registry.
`IComponentPool`	Нетипизированный интерфейс пула компонентов.	Позволяет registry хранить разные `ComponentManager<T>` единообразно.
`ComponentManager<T>`	Плотно хранит компоненты одного типа и сопоставление Entity-index.	Используется `ECSRegistry`; специализация `ActiveComponent` задаёт быстрый фильтр активности.
`ECSRegistry`	Создаёт и уничтожает Entity, добавляет, ищет и перебирает компоненты.	Общий источник состояния для почти всех систем; не содержит gameplay-решений.
`TransformComponent`	Позиция, вращение, масштаб и пространственная иерархия.	Пишут movement/physics/editor; читают render, AI, interaction и picking.
`NameComponent` / `ActiveComponent`	Имя для человека и признак участия Entity в системах.	Читаются редактором и фильтрами; не заменяют устойчивый save-id.
`MeshRendererComponent`	Ссылка на mesh/material и параметры отображения.	Готовится сценой/assets, читается render systems и culling.
`RigidBodyComponent` / `StaticColliderComponent`	Связь Entity с Bullet-телом или статическим collider.	Создаются physics layer; transform sync должен иметь одного владельца направления.
`SceneManager`	Владеет registry, сценовыми ресурсами, import/save/load и world settings.	Связывает editor, assets и render; runtime-only ссылки нельзя сериализовать как обычные числа.
`GameObject`	Старый объектный путь рендера и служебных light-сфер.	Сосуществует с ECS; новые gameplay-объекты лучше не возвращать в двойное владение.

Класс или тип	За что отвечает	Связи и границы
`TransformSystem`	Обновляет иерархические/world transforms.	Должен выполняться после локальных изменений и до culling/render/picking.
`EditorUI`	Рисует editor panels и формирует запросы spawn/import/file actions.	Командует `SceneManager`; UI не должен напрямую владеть runtime-ресурсами.
`ObjectDebugWindow`	Показывает диагностические данные выбранной Entity.	Читает scene/registry; подходящее место для NPC reason, LOS и yaw error.
`PickingSystem`	Строит луч экрана и выбирает объект, свет или sprite.	Использует camera и scene data; результат потребляет editor selection.
`DragDropSystem`	Перетаскивает выбранный объект в мире редактора.	Связывает input, picking, camera и scene transform.

62.1. Семейства компонентов сцены

`InteractionComponent`, `EditorSelectionComponent`, `ShipRootComponent`, `ShipPartComponent`, `BoneAttachmentComponent`, `PlayCameraComponent`, `FirstPersonViewComponent`, `WorldAreaComponent` и `SpotLightComponent` — декларативные метки или настройки. Они не выполняют работу сами: их смысл возникает только в системе-потребителе. При добавлении поля всегда записывай, какая система его пишет, какая читает и сохраняется ли оно в сцену.

62.2. Save/load контракт

- Authoring: диапазоны, скорости, ссылки через устойчивый id, флаги и конфигурация должны сохраняться.
- Runtime: текущая цель, временный путь, cooldown, reservation и указатели Bullet/GPU после загрузки пересоздаются.
- Каждый новый компонент получает schema version, default для старой сцены и round-trip тест save -> load -> save.
- Для `NPCCombatComponent` отдельно решить судьбу `attackRange`, `seeRange`, cooldown и lose-target delay; runtime target не сохранять.

63. Ресурсы и рендеринг

Класс или семейство	За что отвечает	Связи и границы
`AssetLoadSystem`	Асинхронно готовит CPU mesh и создаёт GPU-ресурсы на главном потоке.	Работает через `JobSystem`; состояние запроса отражают tracker/snapshot типы.

Класс или семейство	За что отвечает	Связи и границы
`MeshAsset` / `MeshCpuData` / `MeshGpuData`	Разделяют декодированные вершины и готовые D3D buffers.	CPU-часть допустима в worker; GPU-часть создаётся с device на main thread.
`D3DClass`	Владеет Direct3D device, context, swap chain, depth и базовыми render states.	Нижний слой для всех render-классов; gameplay не должен обращаться к нему.
`Renderer`	Оркестрирует проходы кадра, тени, материалы, debug и итоговую статистику.	Читает scene/camera/settings; не принимает gameplay-решений.
`RenderCandidateSystem`	Готовит и фильтрует список кандидатов на рендер.	Получает ECS snapshot, выдаёт result renderer; удобная граница для многопоточности.
`CameraClass`	Хранит view-состояние и строит лучи из экрана.	Используется renderer, picking, оружием и камерными системами.
`ViewPointClass`	Представляет дополнительную точку зрения/матрицы.	Нужен для специальных проходов; не заменяет gameplay camera ownership.
`FrustumCulling`	Проверяет видимость bounds относительно камеры.	Потребляет world bounds, помогает `RenderCandidateSystem` и renderer.
`ModelClass`	Базовое статическое mesh-представление и buffers.	Низкоуровневый ресурс, используемый render path.
`AssimpModelClass`	Загружает сложные модели, skeleton, clips и bone data.	Источник данных для `AnimationComponent` и character rendering.
`TinyGltfModelClass`	Загружает glTF-модели через tinygltf.	Альтернативный import path; форматные детали не должны течь в gameplay.
`ModelManager`	Кэширует и выдаёт модели по ключу.	Снижает повторные загрузки; политика владения должна быть едина с assets.
`MaterialClass`	Хранит параметры материала и ссылки на текстуры.	Читается shader/renderer; damage visual может передаваться отдельным параметром.
`TextureClass` / `RenderTextureClass`	Загружает обычную texture и предоставляет render target texture.	Используются материалами и off-screen проходами отражения/постобработки.
`LightClass` / `SpotLightComponent`	Представляют старый light-объект и ECS-настройки spot light.	Renderer должен получать единый подготовленный light list.
`ShaderManager`	Создаёт и выдаёт набор shader-программ.	Централизует lifetime; renderer выбирает подходящий shader.

Класс или семейство	За что отвечает	Связи и границы
`ShaderBaseClass`	Общий код компиляции, buffers и базовых параметров.	Родитель специализированных shader classes; не знает сцену целиком.
PBR/light/normal/fog shaders	Освещение, материалы, normal maps, fog и damage-параметры.	Получают подготовленные constants от renderer/material.
Water/glass/reflection/clip shaders	Прозрачность, отражение, refraction и clip plane.	Зависят от `RenderTextureClass` и порядка специальных проходов.
Terrain/sky/shadow shaders	Рисуют terrain blend, sky и depth shadow map.	`ShadowMap` создаёт ресурс, основной lighting pass читает его.
`GrassRenderer`	Группирует instances травы и рисует их батчами.	Читает camera/light/settings; отдельный specialised render path.
`BitmapClass` / `WorldSpriteClass`	2D bitmap и sprite, привязанный к миру.	Используются UI/scene helper rendering; picking умеет возвращать sprite.
`DWriteTextSystem`	Рисует текст DirectWrite поверх кадра.	Получает готовые строки HUD/debug, не вычисляет gameplay-значения.

64. Игрок, ввод и перемещение персонажа

Класс или тип	За что отвечает	Связи и границы
`GameInputBinding`	Связывает игровое действие с клавишей или кнопкой.	Конфигурация `GameplayInputSystem`, полезная и для подписей HUD.
`GameInputFrame` / `GameActionState`	Снимок игровых действий одного кадра.	Производится input system, одинаково читается controller, inventory и weapon.
`GameplayInputSystem`	Преобразует device input в смысловые действия и session-команды.	Не двигает Entity; результат получает `GamePlayController` и системы действий.
`GamePlayController`	Переводит действия игрока в intent персонажа.	Пишет movement/controller state; физическое перемещение оставляет controller system.
`CharacterSpawnSystem`	Гарантирует набор компонентов и начальное состояние персонажа.	Используется игроком и `StateSystem`; один prefab-contract уменьшает расхождения NPC/player.
`CharacterMovementSystem`	Обновляет данные locomotion и движение по intent.	Связывает input/AI с controller и animation.

Класс или тип	За что отвечает	Связи и границы
`CharacterFacingSystem`	Поворачивает персонажа по desired facing direction.	Пишет yaw в transform; должен разделять body facing и attack aim.
`CharacterControllerSystem`	Выполняет sweep, ground checks, ступени, slope и безопасное displacement.	Читает movement/tuning и Bullet world; пишет transform/velocity.
`CharacterExternalVelocitySystem`	Добавляет импульсы извне: взрыв, отдачу, течение.	Суммируется с обычным movement до controller integration.
`FirstPersonViewSystem`	Вычисляет camera anchors и first-person представление.	Связывает character bones/transform, camera и view weapon.
`PlayerVitalsSystem`	Обрабатывает показатели жизни игрока.	Читает damage/environment, результат показывает `GameHud`.
`PlayerComponent` / `PlayerGameTag`	Помечают управляемого игроком персонажа.	Их проверяет player-only код, в том числе текущий `PlayerPunchSystem`.
`CharacterController` / `CharacterMovement`	Хранят runtime controller state и намерение движения.	Пишутся input/AI, потребляются controller, facing и animation.
`CharacterPhysicsTuningComponent`	Хранит authoring физики персонажа: скорости, slope, шаги и ground tolerances.	Читается controller system; пригоден для профилей NPC.

Ключевой контракт: игрок и NPC должны сходиться на общем уровне intent (`CharacterMovement`, attack request), но иметь разные источники intent. Игрок получает его из input, NPC — из решения AI. Тогда движение и удар можно тестировать без дублирования физики и анимации.

65. NPC: решение, команда и исполнение

Класс или тип	За что отвечает	Связи и границы
`NpcState`	Перечисляет исполняемые режимы NPC.	Читается `StateSystem` и debug UI; состояние не должно подменять причину перехода.
`NpcCommandType` / `NpcCommand`	Описывает приоритетный ручной приказ и его параметры.	Выдаётся `NpcCommandSystem`, временно перекрывает automatic utility.
`NPCComponent`	Базовая идентичность и общие параметры NPC.	Маркер для AI pipeline и editor inspection.
`NPCMovementComponent`	Настройки скорости и runtime движения NPC.	Решение читает параметры, `StateSystem` пишет common character intent.

Класс или тип	За что отвечает	Связи и границы
`NPCCombatComponent`	Хранит цель, дистанции, cooldown и память контакта.	Пишется combat/perception, читается state и debug; target — runtime only.
`NpcBrainComponent`	Хранит utility scores, выбранную задачу, причину и командный режим.	Пишется utility/command, читается coordinator и `StateSystem`.
`NpcCommandSystem`	Единственная точка выдачи и отмены приоритетных команд.	Не строит путь; `StateSystem` исполняет команду и возвращает automatic mode.
`NpcUtilitySystem`	Оценивает потребности и выбирает автоматическую задачу.	Читает ремонт, энергию, кислород и world state; не двигает NPC.
`CrewTaskCoordinator`	Разрешает конкуренцию экипажа за задачи через claim.	Работает между utility и исполнением; предотвращает толпу у одной цели.
`CrewMemberComponent`	Параметры члена экипажа для координации задач.	Читается coordinator; не содержит маршрут.
`CrewTaskClaimComponent`	Фиксирует владельца задачи и срок claim.	Должен очищаться при смерти, отмене, timeout и удалении цели.
`NpcCombatSystem`	Выбирает combat target и переводит между chasing/attacking по текущим условиям.	Передаёт решение `StateSystem`; следующий этап — вынести perception/LOS отдельно.
`StateSystem`	Исполняет команду или выбранную задачу: путь, двери, ремонт, facing, animation request.	Центральный исполнитель; не должен сам становиться perception и damage system.
`NpcNavMeshSystem`	Строит/использует Detour path, выдаёт waypoint и dynamic avoidance.	Вызывается `StateSystem`; принимает destination, не выбирает игровую цель.
`NpcNavMeshDebugInfo`	Снимок маршрута, статуса и причин навигации.	Показывается debug UI/rendering; не влияет на решение.

65.1. Целевая цепочка NPC

```
Perception -> TargetMemory -> CombatDecision / Utility
           -> NpcBrain intent + reason
           -> CrewTaskCoordinator / CoverReservation
           -> StateSystem executor
           -> CharacterMovement + AttackRequest
           -> Controller + Facing + Animation + HitDetection
           -> DamageEvent -> DamageSystem
```

В этой цепочке каждая стрелка — тестируемый контракт. Например, если NPC смотрит верно, но бьёт вбок, сначала сравниваются body forward, attack aim и fist/contact direction. NavMesh к этой ошибке не относится.

66. Бой, анимация, инвентарь и взаимодействие

Класс или тип	За что отвечает	Связи и границы
`AnimationComponent`	Хранит bindings, clip state, one-shot, blend и first-person offsets.	Пишется gameplay request, читается animation controller/render model.
`CharacterAnimationControllerSystem`	Выбирает locomotion/one-shot, смешивает верх тела и обновляет pose.	Читает movement и requests; не решает, прошёл ли урон.
`AnimationSystem`	Низкоуровневые операции обновления анимации моделей.	Используется контроллером и model data; держит математику pose отдельно.
`TwoBoneIK`	Решает двухкостную IK-цепь по target и pole.	Используется руками/ногами; требует корректных coordinate spaces.
`WeaponIKSystem`	Вычисляет grip/attachment преобразования оружия.	Связывает weapon component, skeleton attachment и first-person view.
`MeleeActionConfig` / `MeleeActionJson`	Описывает клип, contact window, reach, damage и варианты удара.	Загружается из данных; одинаковый контракт нужен игроку и NPC.
`PlayerPunchSystem`	Обрабатывает player-only punch: aim ray, contact phase, hit и IK.	Читает camera и `PlayerComponent`; NPC сейчас через этот путь не проходит.
`WeaponComponent`	Хранит характеристики оружия, grip transforms и runtime параметры.	Читается weapon/IK/inventory и сериализуется как authoring.
`WeaponSystem`	Обрабатывает экипированное оружие, прицеливание, атаку и запрос damage.	Получает input/camera, выдаёт `DamageEvent` через `DamageSystem`.
`DamageEvent`	Сообщение о нанесении урона: источник, цель, величина и контекст.	Создаётся melee/weapon/explosion, потребляется damage pipeline.
`DamageSystem`	Применяет события к `DamageableComponent` и запускает effects.	Единственная точка изменения здоровья; вызывается после генераторов damage.
`InventoryComponent` / `InventorySlot`	Хранят содержимое и вместимость инвентаря Entity.	Меняются только через inventory operations/system.
`InventorySystem`	Проверяет и выполняет add/remove/equip операции.	Связан с pickup, gameplay input, weapon ownership и HUD.
`WeaponPickupSystem`	Находит доступный pickup, переносит weapon и формирует feedback.	Читает interaction/input, вызывает inventory, отдаёт текст HUD.
`InteractionSystem`	Raycast-выбор интерактивной Entity, prompt и opened entity.	Получает camera/input, результат используют pickup, panels и HUD.

Класс или тип	За что отвечает	Связи и границы
`EventQueue`	Очередь gameplay-событий, например открытие двери и NPC interaction.	Развязывает producer и consumer в пределах кадра.
`TriggerZoneComponent` / `TriggerZoneSystem`	Описывает зону и отслеживает enter/stay/exit overlaps.	Порождает trigger events; не исполняет содержательное действие.
`TriggerActionComponent` / `TriggerActionSystem`	Преобразует trigger event в настроенное действие и feedback.	Потребляет события зон, меняет gameplay через явные команды.
`DoorSystem`	Обрабатывает door events и плавно меняет open progress/проходимость.	Вызывается после events; `StateSystem` ждёт результат, а не анимирует дверь сам.

67. Физика подлодки, помещения, энергия и ремонт

Класс или семейство	За что отвечает	Связи и границы
`PhysicsSystem`	Создаёт rigid bodies, синхронизирует ECS/Bullet и обновляет физику объектов.	Использует `PhysicsWorld`; порядок sync должен быть явно определён.
`PhysicsRaycast`	Даёт общий raycast и callback с исключением Entity.	Используется interaction, оружием, perception и picking physics.
`SubmarineSystem`	Обновляет физическое состояние корпуса подлодки.	Читает hull/buoyancy/pressure data и работает рядом с physics.
`SubmarineJointSystem`	Обновляет физические соединения частей подлодки.	Опирается на `SubmarineJointComponent` и Bullet constraints.
`JointConnectionSystem`	Согласует логические соединения частей и их состояние.	Работает до joint physics; удаление частей должно чистить обе стороны.
`PressureSystem`	Вычисляет влияние внешнего давления и повреждения корпуса.	Порождает изменения hull/breach, которые затем влияют на flooding.
`FloodingSystem`	Обновляет воду через breaches и compartments.	Читает hull breach, room/compartment и water level.
`RoomSystem`	Определяет membership объектов в комнатах и готовит snapshot/result.	База для environment, culling, oxygen и gameplay queries.
`RoomEnvironmentSystem`	Переносит среду между комнатами и обновляет атмосферу.	Читает room graph/openings; питает oxygen, hazards и AI.

Класс или семейство	За что отвечает	Связи и границы
`RoomCullingSystem`	Скрывает комнаты, не нужные текущей камере.	Читает room topology и camera; влияет только на представление.
`ShipAlertSystem`	Сводит проблемы корабля в alert snapshot.	Читает flooding/power/oxygen/damage; HUD и AI используют результат.
`PowerGridComponent` / `PowerConsumerComponent`	Хранят сеть, спрос, питание и приоритет потребителя.	Пишутся power systems, читаются устройствами и UI.
`PowerSystem` / `ShipPowerGridSystem`	Собирают grid, распределяют доступную мощность и выставляют powered state.	Должны выполняться до систем устройств, зависящих от питания.
`GeneratorComponent` / `GeneratorSystem`	Хранят и обновляют генератор, топливо/выход и режим работы.	Поставляет энергию в grid; `GeneratorHud` отображает и управляет.
`OxygenGeneratorComponent` / system	Хранят и обновляют производство кислорода.	Зависит от power, влияет на room environment; имеет отдельный HUD.
`EmergencyBatterySystem`	Подхватывает критические потребители при потере основного питания.	Работает с grid priorities и alert state.
`RepairableComponent` / `WorkZoneComponent`	Описывают повреждение и позицию безопасной работы.	Используются utility AI, coordinator и `RepairSystem`.
`RepairSystem`	Применяет ремонт с учётом времени, доступа и mesh damage.	Вызывается игроком/NPC executor; пишет repair state и event log.
`RuntimeMeshDamageComponent`	Хранит операции вырезания, snapshot геометрии и связь breach-repair.	Связан с mesh cutting, damage, repair и асинхронными jobs.
`ExplosionSystem`	Создаёт радиальный damage/impulse и эффекты взрыва.	Выдаёт damage/external velocity и запускает `ParticleSystem`.
`ParticleSystem` / `ParticleRenderer`	Симулирует emitters/particles и рисует подготовленные render items.	Gameplay создаёт emitter, renderer только отображает.

68. Сквозные маршруты вызовов

68.1. Один игровой кадр

```
InputClass -> InputManager -> GameplayInputSystem
-> Interaction / Inventory / GamePlayController
-> Utility + Coordinator + Combat decision
-> StateSystem -> Facing -> CharacterController
-> Weapon/Punch -> DamageSystem -> ParticleSystem
-> Transform/room/power/physics updates
-> RenderCandidateSystem -> Renderer -> D3DClass
```

68.2. Удар NPC

```
Perception says target visible/in reach
-> NpcCombatSystem selects Attack intent
-> StateSystem writes body facing + AttackRequest
-> Animation controller starts one-shot and exposes phase
-> shared melee hit detection checks contact window + reach + arc/LOS
-> DamageEvent -> DamageSystem -> DamageableComponent/effects
```

Именно shared melee hit detection сейчас является недостающим мостом. Переносить весь `PlayerPunchSystem` в NPC нельзя: camera aim и player input должны остаться адаптером игрока, а общий contact/damage код — стать независимым сервисом.

68.3. Загрузка сцены

```
Editor/File request -> SceneManager parses JSON
-> ECS entities + authoring components
-> AssetLoadSystem CPU jobs -> main-thread GPU upload
-> Physics bodies / runtime caches rebuilt
-> TransformSystem -> render candidates
```

68.4. Авария подлодки

```
Pressure/Damage -> HullBreach
-> Flooding + RoomEnvironment
-> ShipAlert snapshot
-> NpcUtility score -> CrewTaskCoordinator claim
-> StateSystem navigation/work zone
-> RepairSystem -> breach/mesh state -> alert recalculation
```

69. Правила зависимостей и кандидаты на разделение

Правило	Почему	Проверка
UI читает snapshot и отправляет command	Иначе интерфейс становится скрытым владельцем gameplay state.	Закрытие окна не меняет симуляцию само по себе.
AI пишет intent, не Transform	Физика и collision должны иметь единый путь исполнения.	NPC и игрок проходят общий controller.
Animation сообщает phase, но не наносит damage	Клип — представление времени действия, а не владелец здоровья.	Damage появляется только через event/system.
NavMesh выбирает путь, не цель	Навигация не должна знать тактику.	Одну destination можно подать из repair, cover или flank.
Renderer не читает brain rules	Изображение не должно влиять на решение.	Headless simulation сохраняет смысл.
Runtime handles не сериализуются	Entity/Bullet/GPU ссылки протухают после load.	После reload все caches восстановлены.
Один writer на физическое состояние	Двойная запись transform создаёт дрожание и рассинхрон.	Для каждой фазы известен владелец position/rotation.

Правило	Почему	Проверка
Reason хранится рядом с решением	Без причины debug показывает только итоговый state.	Inspector объясняет transition и score.

69.1. Кандидаты на аккуратное разделение

48. Выделить из `ApplicationClass` явные pipeline-функции с фиксированным порядком, не меняя поведения.
49. Разделить `StateSystem` на маленькие executors задач после появления characterization tests.
50. Выделить `NpcPerceptionSystem` и `TargetMemoryComponent` из distance-only combat выбора.
51. Выделить общий `MeleeExecutionSystem`; player camera и NPC aim оставить отдельными producers intent.
52. Ввести `SceneSerializationRegistry`, чтобы новые компоненты не забывались в save/load вручную.
53. Свести старые `GameObject` и ECS render paths к одному контракту владения ресурсами, постепенно, без большого переписывания.

Часть XI. Практика разработки систем и NPC

Эта часть дополняет карту классов конкретными трассировками. Слово «сейчас» означает состояние исходного кода на 28 октября 2026 года. Предложенные улучшения помечены как проектные задачи; они не выдают будущую механику за уже работающую. Для поиска реализации используй указанные пути, а при изменении кода проверяй порядок вызовов в `src/Core/ApplicationClass.cpp`.

70. Как читать одну систему от входа до результата

У каждой механики найди пять точек: источник события, данные ECS, правило решения, исполнитель и наблюдаемый результат. Для NPC источник может быть пробоина, приказ или увиденный враг. Данные лежат в `NPCComponents.h`, `RepairableComponent`, `DamageableComponent`, оружии и тактическом компоненте. Решение принимает Utility AI, координатор экипажа или боевой слой. Исполнение делает `StateSystem`, `CharacterControllerSystem`, `RepairSystem` либо `WeaponSystem`. Результат виден в мире, журнале события и debug UI.

Текущий порядок в Play: `WeaponSystem::UpdateTimers` → `NpcUtilitySystem` → `CrewTaskCoordinator` → `NpcCombatSystem` → `NpcCoverSystem` → `StateSystem` → `CharacterFacingSystem` → `CharacterControllerSystem` → `AnimationController` → `DamageSystem`. После обновления позы в другой части кадра вызывается `NpcRangedCombatSystem`. Так видно, почему стрелковый слой может брать положение дула после анимации, а Utility AI не двигает капсулу напрямую. `NpcSquadCombatSystem::Update` в `ApplicationClass.cpp` закомментирован. Файл групповой тактики существует, но считать её действующим поведением сейчас нельзя.

Перед изменением механики нарисуй такую цепочку и подпиши владельца каждого поля. Изменение одного состояния в двух системах одного кадра может дать скрытую борьбу: последняя запись победит, а в debug panel причина будет непонятна. Старайся иметь одну точку записи решения и отдельную точку физического исполнения.

71. Дружественный NPC ремонтирует пробоину

71.1. От урона к задаче

`DamageSystem.cpp` может создать сущность пробоины после попадания: на ней появляются `HullBreachComponent`, `RepairableComponent` и связь с операцией повреждения меша. Пробоина содержит мировую точку, комнату, признак запечатывания и интенсивность утечки. У оборудования путь иной: это `DamageableComponent` с `health < maxHealth`, общий `RepairableComponent`, Transform и желательно рабочая зона. Эти два типа цели нельзя сводить к одному визуальному признаку «сломано»: пробоина влияет на герметичность, оборудование — на функциональность устройства.

`NpcUtilitySystem::Update` пропускает выключенных NPC, бойцов и экипаж вне смены. Для доступного члена экипажа он оценивает ремонтные цели и записывает `repairScore` для диагностики. Активная `NpcCommand` сохраняет ручной приоритет: score вычисляется, но автоматический выбор не перезаписывает приказ. `CrewTaskCoordinator::Update` затем чистит недействительные заявки,

оценивает доступных работников и ставит CrewTaskClaimComponent на выбранную цель. Один claim не даёт нескольким членам экипажа бежать к одной работе. Если работник умер, выключен, ушёл со смены или цель исчезла, claim должен освобождаться.

У координатора score строится из серьёзности задачи (вес 0,65), близости (0,35), коэффициента роли и repairSkill. Пример без ограничивающего clamp: пробоина с leakRate = 0,12 даёт severity = 0,8; при distanceScore = 0,5, инженерном коэффициенте 1,25 и skill = 1,5 score = (0,8·0,65 + 0,5·0,35)·1,25·1,5 ≈ 1,303. Для медика при том же расстоянии и skill = 1 score ≈ 0,431. Это объясняет предпочтение инженера. Но самый высокий score не гарантирует физическую достижимость: NavMesh, дверь и рабочая зона проверяются дальше. Именно поэтому в плане развития есть предоценка маршрута и явный отказ задачи.

71.2. Дойти до места работы

StateSystem::UpdateRepairTask сначала перечитывает RepairableComponent: цель могла уже исчезнуть или быть завершена. Для пробоины берёт worldPosition и room. Для оборудования берёт Transform, RoomMembership и по возможности достижимую точку WorkZoneComponent; если подходящей зоны нет, используется позиция оборудования. Дистанция рабочего взаимодействия считается по горизонтали, так как персонаж стоит на палубе. До входа в repairDistance состояние MoveToRepair вызывает NavigateTo: маршрут может включать комнаты, двери и NavMesh. Ожидание створок выражается отдельным WaitingForDoor; недоступная дверь должна давать Blocked, а не бесконечное топтание.

StateSystem выставляет намерение движения в CharacterMovement. Затем CharacterFacingSystem задаёт взгляд, CharacterControllerSystem двигает Bullet капсулу с коллизиями, а анимация выбирает ходьбу или покой. Запись Transform.position = target здесь обошла бы двери и физику. Если NPC дошёл по визуальной линии, но не чинит, сравни расстояние до рабочей точки, repairDistance, текущую комнату и статус маршрута. Неправильный pivot оборудования может сделать точку Transform недоступной; для такого объекта добавляй рабочую зону.

71.3. Накопить прогресс и завершить

В состоянии Repairing NPC останавливает движение и передаёт прирост в RepairSystem::ApplyRepairProgress. Формула NPC в текущем коде: deltaProgress = requiredProgress · clamp(repairSkill, 0,25, 2) · dt / max(repairDuration, epsilon). Если requiredProgress = 1, repairDuration = 8 с, skill = 1,5 и dt = 0,1 с, за тик добавится 0,01875, а полный ремонт займёт приблизительно 5,33 с. Проверяй итог на реальном коде с учётом уже накопленного progress. У игрока в RepairSystem::Update прирост идёт от repairRate · dt: две механики используют общую точку завершения, но разные настройки скорости.

При достижении порога CompleteRepair герметизирует пробоину или восстанавливает оборудование. Для повреждённого runtime mesh может запускаться обратная операция; её завершение нужно прокачать на главном потоке. Код после завершения не должен хранить ссылку на компонент пробоины: её Entity может быть удалена. Убери claim, обнови мир и журнал события, затем разреши NPC выбрать следующую работу. Критический сценарий: цель удалена во время движения или асинхронной операции. Проверяй Entity и актуальность операции снова перед применением результата.

71.4. Ручной ремонт и отладка

Команда `NpcCommandSystem::RepairEntity` передаёт конкретную Entity и имеет приоритет над Utility AI. `StateSystem` исполняет её тем же `UpdateRepairTask`, но по завершении снимает команду. `ResumeAutomatic` очищает приказ и маршрут, чтобы старый путь не использовался для новой автоматической цели. При поиске ошибки смотри по порядку: Active/onDuty, фракция и боевой компонент, валидность Repairable, score, claim, target, комнату, дверь, рабочую точку, дистанцию, progress, ожидание async. Debug status нужен для каждого отказа, иначе «NPC стоит» не является диагнозом.

72. Вражеский NPC обнаруживает и атакует

72.1. Фракция и цель

Текущий бой использует `FactionComponent` и `AreEnemies(a, b)`: одинаковые стороны дружественны, Neutral не атакуется, разные ненейтральные стороны считаются врагами. Старый `NPCComponent::isHostile` остаётся ради совместимости некоторых сцен, но не заменяет отношение пары субъект–цель. Для нового NPC обязательно задавай фракцию; без неё дальний бой возвращает статус `Shooter has no faction`. После изменения правил дипломатии замени функцию отношений и проверь также выбор цели, линию огня и поведение нейтральных.

`NpcCombatSystem` собирает кандидатов, проверяет дальность, горизонтальный FOV и физические лучи до частей цели. Личная видимость обновляет `targetVisible`, точку прицеливания и последнюю известную позицию. Сообщение другого NPC не должно автоматически ставить `targetVisible = true`. Когда контакт потерян, `Searching` опирается на память и план поиска; использование текущего скрытого Transform игрока сделало бы врага всеведущим. Отдельно храни «где видел», «когда видел» и «куда предположительно пошёл».

72.2. Решение и движение

После наблюдения мозг выбирает `Chasing`, `Attacking` или `Searching`. Для вооружённого NPC дистанция огня ограничена минимумом `combat.rangedAttackRange` и `weapon.range`. Без оружия используется дистанция melee и отдельная устойчивость стойки. `NpcCoverSystem` может удержать ранее выбранное укрытие, чтобы простая проверка видимости не сбивала маршрут каждый кадр. `StateSystem` строит путь и выставляет движение; физический контроллер разрешает столкновения. Тактический компонент отвечает за индивидуальные паузы очереди, движение с огнём и укрытие. В текущем кадре групповой `NpcSquadCombatSystem` отключён, поэтому описывать подавление отряда как работающую систему неверно.

72.3. Разрешение выстрела

`NpcRangedCombatSystem` работает после обновления позы и получает дуло из модели и socket. Он требует живого активного NPC с мозгом, фракцией, оружием и неактивной ручной командой. Далее проверяет перезарядку, патроны, состояние `Attacking` либо допустимый огонь на ходу, личную видимость, время прицеливания, существование и фракцию цели, ориентацию, свободный выход из

дула и центральную линию огня. Если на ней живой союзник, статус становится `Holding fire: ally in firing lane`. Это не абсолютная гарантия отсутствия friendly fire при разбросе: проверяется центральный луч, а итоговая пуля может отклониться.

Только после этих условий вызывается общий `WeaponSystem::TryFire`. Он управляет кулдауном, разбросом, физическим лучом и передачей попадания в `DamageSystem`. Следовательно, «цель видна» не означает «выстрел должен быть». Диагностируй первый отказ по `rangedStatus`: `Unarmed`, `Reloading`, `Aiming`, `No visible target in firing stance`, союзник на линии или cooldown. Если пуля летит из головы, проверь socket и результат позы до запуска системы выстрела. Если попадает сквозь дверь, проверь collision shape, фильтр луча и то, кто первым пересекается.

72.4. Попадание и последствия

`DamageSystem` обрабатывает событие урона позже в кадре: снижает здоровье, сообщает тактике о входящем огне, может создать повреждение корпуса и мировой event. Для новой реакции на попадание не делай её в pixel shader или анимации: визуальный эффект должен читать уже принятое игровое событие. При гибели NPC отменяй цель и огонь, освобождай занятое укрытие или задачу и не допускай повторного выстрела от уже мёртвого объекта.

73. Добавление новой работы дружественного NPC

Пример будущей задачи — тушение пожара. Сначала определи правила мира: `FireComponent` хранит интенсивность, комнату и источник; атмосфера влияет на горение; огонь повреждает оборудование; тушение требует ресурса и безопасной рабочей точки. Не начинай с нового значения `NpcState`, пока не решено, что должно происходить при отсутствии кислорода, закрытой двери, занятой задаче и смерти работника.

Шаг 1. Добавь данные пожара и сохранение настроек сцены. Runtime поля вроде текущего таймера эффекта отдели от authoring полей. Шаг 2. Создай `FireSystem` в папке Gameplay и поставь его в понятный момент симуляции относительно атмосферы и damage. Шаг 3. Расширь `CrewTaskType`, сбор кандидатов и score координатора. Например, severity должна учитывать интенсивность, риск для соседнего отсека и ценность оборудования; роль и запас огнетушителя влияют на пригодность работника. Шаг 4. Claim оставь единым механизмом: один пожар — один назначенный NPC, если дизайн не допускает совместной работы. Шаг 5. Добавь `MoveToFire`, `Extinguishing` и имена состояний, ветку StateSystem с навигацией через двери, радиусом взаимодействия, тайм-аутом и отказом. Шаг 6. Исполнитель уменьшает интенсивность через `rate · skill · dt`, создаёт расход ресурса и сообщает событие. Анимация реагирует на состояние, но не владеет расчётом огня.

Для первого вертикального среза сделай одну комнату, одного NPC и один очаг. Затем проверь две заявки на один очаг, недоступную комнату, кончившийся ресурс, отмену ручным приказом, сохранение и загрузку, смерть исполнителя, пожар вне камеры. Если новую задачу нельзя объяснить по цепочке «данные → решение → claim → путь → действие → событие», система ещё не готова к расширению.

74. Добавление нового боевого поведения

Пример — осмысленное отступление при низком здоровье. Настройка `retreatHealthFraction` и время повторного выбора живут в `NpcTacticalComponent`; редактор показывает их, а SceneManager сохраняет настройки. Координатор или индивидуальная система оценивает кандидаты: точка должна быть достижима по NavMesh, дальше от последней известной позиции врага, иметь защиту и не быть занята. Сравни оценку отступления с удержанием укрытия и преследованием, задай гистерезис, чтобы NPC не менял цель каждый кадр.

Если существующие `TakingCover` и `Chasing` с тактической точкой выражают поведение, переиспользуй их. Новое состояние нужно только когда его исполнитель действительно иной. У приказа обязательно есть: условие входа, цель, срок действия, условие завершения, отмена при смерти/потере маршрута и запасное действие при отсутствии точки. Для проверки используй известную, а не скрытую позицию врага. Ручная команда Stop должна оставаться выше боевой автоматики. Трассируй переходы в NPC monitor, затем проверь одиночного врага, двоих, узкую комнату, исчезнувшее укрытие, закрытую дверь и повторное обнаружение цели.

75. Bullet Physics как часть симуляции

75.1. Кто владеет телом

`PhysicsWorld` содержит Bullet world. ECS компоненты и системы связывают физические объекты с игровыми Entity. У каждого `btRigidBody`, `btCollisionShape`, action и constraint должен быть явный владелец и порядок удаления. Сначала удали действие или constraint из мира, затем тело, затем связанные ресурсы; не освобождай shape, на который ещё ссылается живое тело. При смене сцены и shutdown этот порядок особенно важен. `ApplicationClass` прямо отмечает, что ECS владеет Bullet actions, constraints, bodies и shapes и очищает их до разрушения мира.

Статическое тело представляет неподвижную геометрию. Динамическое получает скорость, силы, импульсы и ответ столкновений. Kinematic управляется кодом, а Bullet использует его положение при контактах; для движущейся двери важно согласовать обновление motion state и broadphase. Не делай объект одновременно динамическим и принудительно переписываемым Transform каждый кадр. Для каждого типа запиши направление синхронизации: gameplay → Bullet или Bullet → gameplay.

75.2. Collision shape и render mesh

Render mesh отвечает за картинку, collision shape — за контакты и raycast. Точная треугольная сетка может быть дорогой и неудобной для динамического тела. Коробки, капсулы, наборы примитивов и выпуклые оболочки дают более стабильный и дешёвый контакт. Вогнутую геометрию интерьера обычно используют как статическую. Если NPC визуально проходит через стену, сначала включи debug draw shape: возможно, стена нарисована, но её collision shape отсутствует или смещён. Если персонаж застревает на пороге, сравни высоту капсулы, размер шага, форму двери и NavMesh, а не увеличивай случайно скорость.

75.3. Шаг физики и время

Физика должна получать секунды, а не миллисекунды. Fixed step упрощает стабильность контактов: накопи real dt в accumulator и выполни несколько шагов фиксированной длительности, например 1/60 с, с лимитом подшагов на случай зависшего кадра. Лимит защищает приложение, но означает, что при слишком длинном кадре часть реального времени может не быть просимулирована; это нужно явно учитывать. Системы, производящие силы и перемещение до Bullet step, должны отработать раньше потребителей результата. После step обнови Transform тех тел, где владельцем положения является физика. Рендер может интерполировать предыдущую и текущую позу, но интерполяция не должна менять физическое состояние.

75.4. Капсула NPC и ручной Transform

`StateSystem` записывает желание идти, `CharacterControllerSystem` двигает капсульный контроллер и учитывает ground test, step-up и стабилизацию. Сам мозг NPC не должен менять мировую позицию, иначе маршрут не уважает коллизии и расходится с Bullet. Если в редакторе двигаешь объект с физикой, явно переведи действие в корректный режим и синхронизируй физическое тело. Симптом «визуальная модель дрожит на месте» часто означает двух владельцев Transform или борьбу корректировки контроллера с ручной записью позиции.

75.5. Raycast для зрения и оружия

Bullet raycast возвращает первое пересечение физической геометрии по отрезку. Он не знает, дружественна ли Entity, чинится ли объект и должен ли урон пройти через стекло. Эти правила находятся выше, в игровых системах. Для луча зрения исключи собственную капсулу и проверь допустимый объект попадания; для оружия отдельно проверь дуло, союзника, дистанцию и маски. После runtime cutting физическая форма должна соответствовать новой геометрии или явно оставаться упрощённой; иначе игрок видит пробоину, а луч всё ещё встречает старую сплошную стену. Это важная проверка качества ремонта и повреждений.

75.6. Численный пример импульса

Импульс J имеет размерность Н·с = кг·м/с. При массе 20 кг и импульсе 40 Н·с вдоль X изменение скорости Δv = J/m = 2 м/с, если не учитывать ограничения и столкновения. Сила 40 Н за 0,1 с дала бы только J = 4 Н·с и Δv = 0,2 м/с. Не заменяй `applyImpulse` на `applyForce` с тем же числом: это разные величины. Точку приложения выбирай осознанно — импульс вне центра масс добавляет вращение. Для взрыва сначала вычисли нормализованное направление от центра, ослабление по расстоянию и допустимый радиус, затем передавай физике; нулевую дистанцию обработай отдельно.

76. Вершинный и пиксельный шейдеры

76.1. Путь одного треугольника

CPU создаёт vertex/index buffer, выбирает input layout, shader, constant buffers, текстуры, sampler, rasterizer и depth/blend state. Вершинный шейдер вызывается для вершин и выдаёт `SV_POSITION` в clip

space плюс данные для интерполяции. Rasterizer превращает треугольник в покрытые пиксели и интерполирует TEXCOORD, normal и worldPos. Пиксельный шейдер вычисляет цвет кандидата фрагмента. Затем depth/stencil и blending определяют итог в render target. Pixel shader не запускается один раз на объект и не должен хранить gameplay-состояние.

76.2. Что делает PBR vertex shader проекта

В `src/Render/Shader/Vs/pbr.vs` вход содержит позицию, UV, нормаль, tangent, binormal, до четырёх bone ID и веса. Если валидных костей нет, статичная модель остаётся в исходной позе: нельзя подставлять boneTransforms[0], потому что слот b8 мог остаться от предыдущего draw. Для skinned модели вершина смешивается по весам костей; затем `world → view → projection`. Отдельно формируется tangent basis и worldPos для освещения. `SV_POSITION` используется растеризатором, а `TEXCOORD` и другие семантики соединяют выход VS со входом PS. Если имена/типы/слоты не совпадают, получится ошибка компиляции или неверные данные.

76.3. Что делает PBR pixel shader проекта

`src/Render/Shader/Ps/pbr.ps` читает albedo, normal, metalness, roughness, shadow map, crack mask и environment map. Он работает с интерполированными normal/worldPos/UV, светом и камерой. Перед dot с направлением света нормализуй векторы; для энергии в BRDF ограничивай значения, следи за линейным цветовым пространством и conversion sRGB на границах текстур/вывода. Текстура цвета в sRGB не равна линейному освещению: например, значение 0,5 sRGB соответствует приблизительно 0,214 linear. Если перемножить свет прямо с 0,5 и затем ещё раз закодировать sRGB, материал станет неверно ярким. Normal map хранит направление, её не следует обрабатывать как цвет albedo.

76.4. Constant buffers, регистры и 16 байт

VS и PS могут иметь разные `b0`: в `pbr.vs` это MatrixBuffer, в `pbr.ps` — LightColorBuffer. Регистры привязаны к стадии. В C++ соответствующие структуры должны совпадать с HLSL по размеру, порядку и выравниванию; constant buffer выделяется с ByteWidth, кратной 16. `float3` рядом с `float` может делить один 16-байтный регистр, а массивы и матрицы имеют дополнительные правила упаковки. Не добавляй поле в HLSL, забыв C++ upload и выбор слота. Изучай `PBRShaderClass.cpp`: CPU транспонирует world/view/projection перед загрузкой, и это часть фактического контракта.

76.5. Как добавить новый эффект на практике

Пример — индикатор перегрева оборудования. Gameplay компонент хранит температуру и порог; система тепла обновляет её в секундах и создаёт события. Рендер читает нормализованное `heat01` из снимка состояния, передаёт в отдельный параметр материала или constant buffer; PS смешивает emissive цвет. Шейдер только отображает уже вычисленное состояние. Для реализации: добавь поле в C++ структуру и HLSL с одинаковой упаковкой, обнови upload и bind, назначь register, проверь статичный и skinned PBR draw, тени, прозрачность и случай отсутствующего параметра. После draw восстанови или явно задай состояния следующего прохода, чтобы эффект не «утёк» на соседний материал.

76.6. Типичные поломки

Объект исчез: проверь clip position, winding, cull и depth. Чёрная модель: проверь SRV, sampler, normal, NaN и линейность цвета. Эффект есть на предыдущем объекте: не очищен слот или constant buffer. Свет движется вместе с камерой: worldPos или lightPos оказались в разных пространствах. При неравномерном масштабе странный свет: normal/TBN преобразованы как позиция. Прозрачное стекло выглядит непрозрачным: проверь blend state, порядок рисования и запись depth. Исправляй первое неверное промежуточное значение, а не подбирай итоговый цвет на глаз.

77. Рецепт добавления системы в DX Framework

1. Запиши игроку видимый результат, входные события, инварианты и пять сценариев отказа. Для системы тепла: «работающий реактор нагревает отсек, вентиляция охлаждает, перегрев наносит урон и виден на приборе».

2. Выбери владельца данных и единицы. `HeatComponent` содержит ёмкость, температуру, источник и границы; не хранит указатель на Renderer. Раздели настройки сцены и runtime состояние.

3. Сделай одну систему, которая читает явные входы и пишет ограниченный набор выходов. Определи, нужен variable dt или fixed tick. Температура от потока энергии может считаться как `ΔT = (P_in − P_out) · dt / C`, где P — ватты, C — Дж/К. Проверь размерности: Вт·с/(Дж/К) = К.

4. Поставь вызов в `ApplicationClass.cpp` после производителей данных и до потребителей. Если Heat зависит от PowerGrid и влияет на DamageSystem, порядок должен это отражать. При циклической зависимости используй явный снимок прошлого тика либо разбей шаг на фазы.

5. Добавь отображение в EditorUI/debug HUD, сохранение authoring параметров в SceneManager и осмысленное событие для звука/эффектов. Отдельно реши, нужен ли runtime save текущей температуры.

6. Проверь минимальную сцену и переходы: нет энергии, авария, смена сцены, пауза, повторная загрузка, объект вне камеры. Профилируй только после правильной логики. Не объединяй AI, физику и render в одном классе ради удобства первого прототипа.

Этот рецепт нужен и для NPC: новый компонент описывает данные, координатор выбирает работу, StateSystem исполняет путь, отдельная gameplay система меняет мир, а UI объясняет отказ. Если новую механику приходится вписывать в десятки несвязанных switch, проверь, не потеряна ли граница ответственности.

Часть XII. План развития космического симулятора на 14 дней

Это единственная дорожная карта в этом издании. Каждый день завершается маленьким работающим срезом, а не только изменением кода. Приоритет — враги и дружественный экипаж; дополнительные системы выбраны потому, что делают их решения осмысленными в живом корабле. Если день требует больше времени, перенеси незавершённое в следующий день, сохранив критерий готовности.

78. День 1. Базовая сцена и измеряемое поведение

Собери одну тестовую палубу: две комнаты, дверь, генератор, пробоина, два члена экипажа разных ролей, два врага, укрытия и оружие. Запиши исходное поведение на видео или в журнале NPC monitor. Для каждого NPC видны State, target, route, claim, visible, fire status и причина отказа. Готово, когда один и тот же сценарий можно повторить после перезапуска и сравнить результат.

79. День 2. Надёжность ремонтных задач

Пройди полный цикл пробоины и оборудования. Исправь потерянные claims, повторное назначение завершённой цели и застревание у рабочей зоны. В UI показывай, кто назначен и почему задача недоступна. Готово, когда два ремонтника не теснятся у одной цели, а смерть или ручной приказ первого освобождает работу.

80. День 3. Доступность маршрута до назначения

Добавь недорогую проверку комнаты, двери и NavMesh перед окончательным claim либо явный быстрый отказ с повторным выбором. Не оценивай только прямую дистанцию через стену. Готово, когда дальняя, но доступная авария выигрывает у близкой за запертой дверью, а NPC не бесконечно пытается взять недостижимую задачу.

81. День 4. Роли и ресурсы экипажа

Разведи инженера, оператора, медика и охрану по пригодности к задачам. Добавь расход материала для ремонта и видимый отказ «нет ресурса». Готово, когда инженер чинит корпус быстрее, оператор осмысленно обслуживает устройство, а неподходящий работник не блокирует жизненно важную задачу при наличии специалиста.

82. День 5. Пожар как системная авария

Сделай один очаг пожара, который нагревает отсек и повреждает устройство. Атмосфера или вентиляция влияет на интенсивность, но не создаёт бесконечную энергию. Добавь debug показания.

Готово, когда пожар продолжает развиваться вне кадра и состояние сохраняется либо явно сбрасывается по описанному правилу.

83. День 6. NPC тушит пожар

Включи новый тип работы в координатор: score, claim, путь через дверь, рабочая зона, расход огнетушителя, завершение и отказ. Для начала один NPC и один очаг, затем несколько NPC. Готово, когда игрок может приказать тушить конкретный очаг и вернуть автоматический режим, а другой работник не дублирует занятую задачу.

84. День 7. Восприятие врага без всеведения

Раздели личное наблюдение, последнюю известную точку и предположение. Проверь FOV, высоту луча, стекло, двери и потерю цели. Готово, когда враг ищет там, где видел игрока, и не меняет маршрут по скрытому перемещению игрока.

85. День 8. Надёжный огонь и перезарядка

Проверь muzzle socket после анимации, короткие очереди, задержку прицеливания, кулдаун, перезарядку и союзника на линии огня. Добавь отдельный debug ray зрения и выстрела. Готово, когда каждый отказ стрелять объясняется статусом, а выстрел из-за укрытия не проходит через собственную стену.

86. День 9. Укрытия и выход из тупиков

Проверь достижимость укрытия и точки выглядывания, освобождение резерва, смену плохой точки и тайм-аут застревания. Готово, когда NPC не перебегает между двумя укрытиями каждый кадр, не занимает точку мёртвого напарника навсегда и имеет запасное действие без подходящего cover.

87. День 10. Простой командный бой

Верни `NpcSquadCombatSystem` только после проверки индивидуального цикла. Передавай между NPC последнее наблюдение, но не личный флаг видимости. Введи две роли: один удерживает огневой угол, другой меняет позицию. Готово, когда одиночка сохраняет поведение, а два NPC не выбирают один и тот же проход и не читают скрытую текущую позицию игрока.

88. День 11. Повреждения корабля как причина решений

Свяжи пробоину с давлением, энергией и эвакуацией. Дружественные NPC приоритизируют жизненно важный отсек; враги могут воспользоваться открытым проходом, но не обязаны знать скрытое

состояние всех помещений. Готово, когда одна авария меняет выбор экипажа и имеет объяснимые последствия для атмосферы и оборудования.

89. День 12. Звук и сигналы тревоги

Добавь мировые события для выстрела, пробоины, пожара и ремонта. Звук имеет позицию и радиус; NPC слышит событие и получает приблизительную область поиска, а не точную Entity за стеной. Тревога на корабле сообщает известный тип аварии и комнату. Готово, когда реакция на звук отличается от реакции на зрительный контакт и не нарушает приватность скрытой цели.

90. День 13. Сохранение и производительность

Сохрани authoring параметры, фракции, роли и постоянное состояние аварий; не сериализуй сырые Bullet pointers, временные claims и GPU handles. После загрузки восстанови связи по Entity remap и пересобери runtime кэши. Измерь стоимость AI, NavMesh, физики и рендера на сцене с несколькими NPC и авариями. Готово, когда повторная загрузка не создаёт второй claim или дубликат пожара и мир работает вне камеры.

91. День 14. Вертикальный срез и выпуск

Сыграй один сценарий целиком: внезапная пробоина и пожар, экипаж делит задачи, враги атакуют и ищут игрока, энергия и атмосфера меняют условия, игрок отдаёт ручной приказ, всё сохраняется и восстанавливается. Запиши обнаруженные причины отказов и один короткий список следующего цикла. Готово, когда сценарий проходится без debug вмешательства, а наблюдатель может объяснить каждое решение NPC по состоянию и журналу событий.

Приложение A. Горячие клавиши

Клавиша	Действие
F5	Editor ↔ Play
F1	object debug window
F2	курсор ImGui в Play / возврат camera capture
F6	async mesh cut по picking ray
Space (Editor)	пауза/продолжение отладочного TimeManager
K	одиночный шаг отладочного TimeManager только в Editor
P	пауза/продолжение симуляции в Play
Space (Play)	прыжок
Esc	закрыть открытую игровую панель

Текущие игровые привязки и пример добавления действия — в разделе 7.4 и конструкторе `GameplayInputSystem`. Для клавиш редактора проверяй `ApplicationClass` и `CameraController`.

Приложение B. Формулы на одной странице

```
length(v) = sqrt(dot(v,v))
normalize(v) = v / length(v)
W = S·Rx·Ry·Rz·T                    [текущий Transform]
p_clip = p_local·W·V·P              [row-vector]
N' = normalize(N·transpose(inverse(W3x3)))
t_exponential = 1 - exp(-lambda·dt)
fixed alpha = accumulator / h
P_water = P0 + rho·g·h
v_breach = sqrt(2·g·h)
Q = Cd·A·v
F_buoyancy = rho·g·Vdisplaced
torque = r × F
DeltaE_MWh = P_MW·dt_seconds/3600
Fresnel = F0 + (1-F0)(1-cosTheta)^5
specular = D·G·F / (4(N·V)(N·L))
particle uniform radius = R·cbrt(u)
```

Приложение C. Словарь

Термин	Смысл
ECS	Entity IDs + data components + systems
source of truth	единственное авторитетное состояние
RAII	ресурс освобождается временем жизни объекта
fixed step	симуляция одинаковыми шагами времени
interpolation alpha	доля между двумя physics poses
frustum	объём видимости камеры
BRDF	модель отражения света поверхностью
PBR	physically based shading assumptions
IBL	освещение от environment map
PCF	усреднение shadow comparisons
NDC	координаты после perspective divide
skinning	деформация vertices matrices костей
NavMesh	polygon graph проходимой поверхности
remap	old Entity ID → new Entity ID после load
transaction	либо операция завершена целиком, либо старое состояние сохранено
soak test	длительный автоматический прогон на leaks/drift

Приложение D. Что открыть через год

1. Прочитать главы 1, 5, 7 и 12.
2. Собрать Debug/Release по главе 6 и записать новый baseline.
3. Открыть `ApplicationClass::Frame` и сравнить timeline с главой 7.
4. Запустить regression scene и debug overlays.
5. Проверить реестр слабых мест главы 39: что уже исправлено.
6. Только после этого добавлять новую систему.

> Главный вывод: DX Framework уже содержит устройство настоящего движка — composition root, ECS, fixed physics, multi-pass renderer, asset pipeline, editor и связанные simulation systems. Чтобы он стал «конфеткой», следующая ступень — не максимальное число функций, а единые контракты, безопасное владение, воспроизводимая сборка и тесты, которые доказывают математику.